

Research on Automated Code and Test Assertion Generation with LLMs

Jiaming, Huang
student ID: 224040352

A Thesis Submitted in
CSC6032 Advanced Operating System

School of Data Science
The Chinese University of Hong Kong, Shenzhen
April 2026

Abstract

The software development process includes requirement analysis, design, coding, testing, delivery, acceptance, and maintenance, among which coding and testing are the stages with the largest development cost. In recent years, automatic code generation technology has developed rapidly, demonstrating extensive application value and significance in improving software development efficiency and enhancing system maintainability; at the same time, with the continuous increase in software scale and complexity, software quality assurance has become a crucial task in software engineering. Automatic unit test case generation tools have emerged accordingly, and can help developers detect and fix problems in code in time during the early stages of software development, gradually becoming an important tool in the software development process.

With respect to code generation and unit test case generation, on the one hand, academia and industry have proposed code generation large language models such as Codex, PanGu-Coder, and StarCoder. Using test cases to evaluate the accuracy of the code generated by large language models has received wide attention. However, current code generation benchmarks and code large language models both have limitations. Specifically, they focus more on generating self-contained functions (i.e., functions that can be successfully executed only with the given programming language environment) and do not pay attention to how to more effectively generate non-self-contained functions, which does not meet

the needs of real development scenarios. On the other hand, although automatic unit test case generation technology has shown significant advantages in improving testing efficiency and software quality, it still faces challenges in practical applications. How to generate efficient, comprehensively-covering test cases that reflect real scenarios is an urgent problem to be solved. How to generate assertions that can find logical errors while generating as few incorrect assertions as possible is a necessary condition for automatic test case generation methods to be practically usable. In addition, for complex systems and projects with complex business logic, how to ensure that the generated test cases do not cause additional performance burdens, and how to conduct precise testing for complex inputs and outputs, also require further research.

To address the above issues, this paper conducts research on automatic generation of code and unit test case assertions based on large language models. The main contributions and innovations include: 1) CoderEval, a function-level code generation benchmark for real development scenarios. CoderEval is the first benchmark supporting project-level code generation accuracy validation. CoderEval first performs statistics based on the tag information on GitHub, selects the top 30 tags with the highest frequency, and then selects projects based on star counts and tags. Finally, CoderEval uses program analysis techniques to extract functions and docstrings from the projects. 2) Unit test case generation based on large language models and a combination of information retrieval and deep learning. This paper first uses ChatGPT to directly generate unit test case assertions. Then it proposes a unit test case assertion generation method based on a combination of information retrieval and deep learning. Specifically, it first proposes a Jaccard-based information retrieval method to retrieve assertions, and finds that simple information retrieval methods can already surpass the deep-learning-based assertion generation method ATLAS. Next, this chapter proposes a program-analysis-

and-neural-network-based adaptation method to modify the retrieved assertions, further improving the accuracy of assertion generation. Finally, through observation, it is found that the large-language-model-based unit test case assertion generation method has strong complementarity with the information-retrieval-and-deep-learning-based unit test case assertion generation method. This paper points out the advantages of non-model-based, lower-resource-consuming methods in unit test case assertion generation. 3) Incorrect unit test case assertion detection based on large language models, by generating postconditions to detect false positives in unit test cases. Although the unit test case assertion generation methods based on large language models and based on combining information retrieval and deep learning can achieve high accuracy in assertion generation tasks, they still generate incorrect assertions. Incorrect assertions would judge correct programs as incorrect, which is unacceptable for developers. Therefore, this paper proposes a method DeCon that detects incorrect unit test case assertions generated by large language models by generating postconditions of the program. The experimental results show that DeCon can detect more than 64% of the incorrect assertions generated by LLM.

Contents

Abstract

List of Figures	vii
-----------------	-----

List of Tables	ix
----------------	----

1 Related Research Work	1
--------------------------------	----------

1.1 Research on Key Technologies of Code Generation	1
1.1.1 Traditional Code Generation Methods	1
1.1.2 Large-Language-Model-Based Code Generation	7
1.1.3 Project-Level Code Generation Benchmarks	13
1.2 Research on Key Technologies of Unit Test Case Generation . . .	15
1.2.1 Traditional Unit Test Case Generation Research Methods	15
1.2.2 Unit Test Case Generation Based on Deep Learning and Large Language Models	19
1.3 Limitations of Existing Related Research Work	21
1.4 Summary of This Chapter	22

2 Introduction	23
-----------------------	-----------

2.1 Research Background and Significance	23
2.2 Research Questions and Difficulties	30

2.2.1	Large-Language-Model-Based Code Generation	30
2.2.2	Large-Language-Model-Based Unit Test Case Assertion Generation	33
2.3	Research Content of This Paper	34
2.4	Thesis Organization	36
3	Function-Level Code Generation Evaluation Benchmark for Real Development Scenarios	38
3.1	Introduction	38
3.2	Construction of the CoderEval Benchmark	42
3.2.1	Project and Function Selection	42
3.2.2	Data Review	44
3.2.3	Test Case Construction	48
3.2.4	Experimental Environment Construction	48
3.3	Experimental Evaluation	50
3.3.1	Experimental Models	50
3.3.2	Experimental Setup	51
3.3.3	Evaluation Metrics	51
3.3.4	Experimental Results	53
3.4	Summary of This Chapter	60
4	Unit Test Case Assertion Generation Based on Information Retrieval and Deep Learning	61
4.1	Introduction	61
4.2	Large-Language-Model-Based Unit Test Case Assertion Generation	65
4.2.1	Experimental Model	65
4.2.2	Experimental Setup	65
4.2.3	Evaluation Metrics	65

4.2.4	Experimental Dataset	66
4.2.5	Experimental Results	66
4.2.6	Limitations of Directly Generating Unit Test Case Assertions Based on Large Language Models	68
4.3	Unit Test Case Assertion Generation Based on the Combination of Information Retrieval and Deep Learning	68
4.3.1	Assertion Generation Method Based on IR	70
4.3.2	Adaptation Model Based on Heuristic Search and Neural Networks	72
4.3.3	Combination of IR and DL Methods Based on Fusion Neural Networks	80
4.3.4	Experimental Dataset	81
4.3.5	Evaluation Metrics	81
4.3.6	Experimental Results	82
4.4	Comparison Between Large-Language-Model-Based Methods and Methods Combining Information Retrieval and Deep Learning . .	91
4.5	Summary of This Chapter	92
5	Large-Language-Model-Based Detection of Incorrect Unit Test Case Assertions	94
5.1	Introduction	95
5.2	Large-Language-Model-Based Method for Detecting Incorrect Unit Test Case Assertions	98
5.2.1	Introduction to the Overall Method Flow	98
5.2.2	Generation and Filtering of Postconditions	100
5.2.3	Incorrect Assertion Detection	104
5.3	Experimental Evaluation	105
5.3.1	Experimental Datasets	105

5.3.2	Experimental Setup	106
5.3.3	Evaluation Metrics	107
5.3.4	Experimental Results	108
5.4	Summary of This Chapter	119
6	Summary	120
	Bibliography	123

List of Figures

1.1	AlphaCode workflow diagram [80]	8
1.2	PanGu- architecture [23]	11
1.3	PanGu-Coder two-stage training diagram [23]	12
1.4	TestPilot workflow diagram [91]	18
1.5	ChatTester workflow diagram [124]	20
3.1	CoderEval construction flow chart	42
3.2	Distribution of runnable levels in CoderEval	47
3.3	Number of problems solved by CodeGen, PanGuCoder, and Chat- GPT on CoderEval respectively	54
3.4	Accuracy of three models in generating oracle_context tokens . .	57
4.1	Assertion generation method based on information retrieval and its combination with deep learning	69
4.2	Distribution of lengths of correctly generated assertions by ATLAS and IR_{ar}	85
4.3	Complementarity between information-retrieval-based methods and deep-learning-based methods on the old dataset	89
4.4	Complementarity between information-retrieval-based methods and deep-learning-based methods on the new dataset	90

4.5	Complementarity between large-language-model-based methods and methods combining information retrieval and deep learning	92
5.1	Specific example of DeCon detecting incorrect unit test case assertions	98
5.2	Overall flow chart of DeCon	99
5.3	Example of formalization processing of I/O instances	102
5.4	Example of DeCon filtering incorrect postconditions	102
5.5	prompt used to generate postconditions.	103
5.6	Example of DeCon detecting incorrect assertions	104
5.7	Number and proportion of different types of assertions generated by different models	109
5.8	Distribution of the number of assertions generated by different models on HumanEval problems	110

List of Tables

3.1	Statistical Comparison among CoderEval, HumanEval, and Concode	42
3.2	Dependencies belonging to each contextual dependency type . . .	45
3.3	Performance of three models on CoderEval and HumanEval . . .	52
3.4	Comparison of effectiveness of different models in generating self-contained and non-self-contained functions in CoderEval	56
3.5	Similarity between original docstring and manually labeled docstring in CoderEval	59
3.6	Effectiveness of two prompt versions of different models on CoderEval	59
4.1	Distribution of assertion categories and numbers in two datasets .	66
4.2	Accuracy of ChatGPT method on each type of assertion	67
4.3	Accuracy of different methods	82
4.4	BLEU values of different methods	82
4.5	Accuracy of different methods on various types of assertions . . .	82
4.6	Accuracy of retrieval using different similarity coefficients	84
4.7	Edit distance between assertions that ATLAS and IR_{ar} cannot successfully generate and labeled assertions	86
4.8	Number of different token categories when the edit distance is 1 .	88
4.9	Accuracy of RA_{adapt}^H and RA_{adapt}^{NN} on each assertion type	88
4.10	Accuracy of the Integration method on each type of assertion . . .	90

5.1	Overall effectiveness of DeCon	112
5.2	Effectiveness of DeCon without I/O filtering of incorrect postcon- ditions	114
5.3	Effectiveness of DeCon in improving code generation effectiveness	116
5.4	Effectiveness of DeCon on fault finding tasks	117

Chapter 1

Related Research Work

This chapter elaborates in detail on the related research work of code generation and unit test case generation from the two parts of code generation benchmarks and code generation methods, respectively.

1.1 Research on Key Technologies of Code Generation

1.1.1 Traditional Code Generation Methods

In daily development work, the most valuable way of code generation is to generate code through natural language description. However, traditional code generation methods (based on deep learning but with model parameter scales not exceeding 100M) have poor understanding of natural language, making it very difficult to directly generate code through natural language. Therefore, traditional code generation work has turned its target to some simpler codes that are applied only in one domain.

The first deep-learning-based code generation method is the relatively simple

and intuitive method IFTTT (If-This-Then-That) [6, 30, 69, 84], which directly applies the network architecture of machine translation to the code generation task. Compared with common programming languages (such as Python, Java, C/C++, etc.), the code generated by IFTTT is relatively simpler and the code scale is usually smaller, so IFTTT is easier to learn the syntactic rules of these codes and complete the learning tasks of these code generation to a certain extent.

The second type of deep-learning-based code generation is for the query languages of relational databases (such as SQL). In 2017, Zhong et al. [127] proposed a model called Seq2SQL, which can translate natural language descriptions into SQL query statements. To address the issue that SQL query conditions are unordered and thus not suitable for optimization using traditional cross-entropy, Zhong et al. adopted a reinforcement-learning-based approach to generate query conditions. In 2018, Cai et al. [13] proposed combining deep learning methods with traditional query parsing techniques to improve the accuracy of generated SQL.

The third type of deep-learning-based code generation method focuses on generating relatively complex programming languages, such as Java and Python. Yin and Neubig [121] proposed an encoder-decoder-based model structure to generate Python code from natural language descriptions. Similarly, Dong and Lapata [30] proposed a model based on code syntax structure. The model achieves the generation of Lambda calculus programs from functional descriptions.

The fourth type of code generation work is sequence-model-based code generation. Sequence models are widely used in various natural language processing problems, such as machine translation, speech recognition, question answering systems, and so on. Hindle et al. [52] first applied the N-gram model to the code completion task. Subsequently, many works have applied the N-gram model to more program generation tasks. However, the N-gram technique itself has ob-

vious shortcomings: it only considers the statistical relationships between local words and cannot consider the overall feature information of the code, nor can it understand the semantic information of code segments. Therefore, the SLAMC model proposed by Nguyen et al. [79] adds global information to the N-gram model. SLAMC combines semantic information and models the rules of semantic annotation. This method has a significant improvement in the accuracy of code recommendation compared to ordinary N-gram. Raychev et al. [88] combined the N-gram model with recurrent neural networks, realizing code completion tasks at the API call level, and improved the performance of related models. At the same time, Tu et al. [103] and Hellendoorn [120] et al. found through statistical research that most codes have local characteristics, and identifiers will repeatedly appear within a relatively close range. Therefore, utilizing this feature, they added a caching mechanism to the N-gram model and achieved significant results in code completion. Compared with N-gram, RNN (Recurrent Neural Network) can capture the sequential relationships in code sequences, and can also learn the semantic information in code. In order to further solve the long-distance dependency problem, many RNN-based variants, such as Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU), have been proposed and widely used. Nguyen et al. [78] and Gu et al. [42] use RNN to learn API vector representations of Java and C# respectively, and apply the learned vector representations to the API migration tasks of these two languages, thereby generating code at the API level. Many scholars apply the RNN model to code Token prediction completion and achieve significant results. Pointer Networks were proposed by Vinyals et al. [109] in 2015. It was originally used to solve combinatorial optimization problems and is a variant of the neural machine translation model (i.e., the sequence-to-sequence model, Seq2Seq model). In the pointer network, attention is simpler: it does not consider input elements but rather points to them

probabilistically. At the same time, some studies have also proposed anonymizing identifiers, which can to some extent alleviate the problem of locally declared variables being reused. In 2016, Bhoopchand et al. [7] proposed a Sparse Pointer Network. The network mainly solves the problem of custom identifier prediction (e.g., class names, method names, variable names, etc.). The paper defines the concept of "Memory" (M), which is used to store the first k identifiers. In addition, the method maintains a vector $m = [id1, \dots, idk]$ to record the position of the identifier in the global vocabulary, i.e., a pointer to the global vocabulary. At each moment of prediction, the network combines the global language model probability distribution, the pointer probability distribution of custom identifiers in the local environment, and the current code input to decide the output word. Compared with traditional recurrent neural networks (without attention mechanism), this method improves the prediction accuracy of custom identifiers in Python by about 25%. Li et al. [66] in 2018 proposed a model combining language models with attention mechanisms and pointer networks for code completion.

The fifth type of traditional code generation method is abstract-syntax-tree-based code generation. Different from natural language generation, programming language is a kind of syntax sequence with strong structural constraints. Every code segment can be parsed into a corresponding abstract syntax tree according to the predefined grammar. Programs generated based on Token sequences usually find it difficult to fully satisfy syntactic constraints, so their accuracy will be very limited. Therefore, many researchers are committed to modeling code structure and syntax semantics and implementing code generation. Different from sequence-based modeling, structured modeling usually describes the probability distribution of the generation process of structures (e.g., abstract syntax trees). Therefore, many works perform program generation on program structures from how structures are generated and using semantics guidance. Using deep learn-

ing to model program structures to generate programs is mainly divided into three types: networks based on abstract syntax tree sequences, networks based on abstract syntax trees. Networks based on abstract syntax tree sequences are the most common network structures when modeling program structures. Researchers convert the nodes of the syntax tree into sequence form through preorder traversal and combine serialized models to encode abstract syntax tree sequences. Traverse the abstract syntax tree into a sequence, and model it using sequence-based models. This method is much simpler than designing a tree network. The difficulty lies in how to ensure that one sequence uniquely corresponds to one program, i.e., ensuring that the sequence is unambiguous. In Liu et al.’s [70] work, they regard the code completion problem as predicting the type and value of the next node of the tree given part of the abstract syntax tree. They obtain the sequence by traversing the abstract syntax tree using preorder depth-first traversal. Li et al. [66] also use preorder depth-first traversal to traverse the abstract syntax tree to obtain a sequence in their work, and learn the sequence through a language model with a pointer network. Different from Liu et al.’s [70] work, in order to ensure that the sequence uniquely corresponds to a code segment, they add two additional symbols when encoding nodes to indicate whether the node has a child node and whether it has a right sibling node. Their results compared with Liu et al.’s [70] results show an improvement of about 4% in the accuracy of predicting the Type and Value of nodes. Similarly, Sun et al. [97] proposed using special backtracking symbols to represent tree structures, which is half the representation space of Liu et al.’s work, achieving a certain degree of performance improvement in program generation tasks. When using tree networks to generate code, the child nodes of the tree are usually generated in a top-down, left-to-right order. Generally speaking, deep learning models that generate tree structures are difficult to learn. First, the computational cost of such

models is very high. Second, context modeling for tree generation requires more complex data structures (compared to sequence models). At the same time, trees often have some ambiguity, and deep neural networks find it difficult to learn such program ambiguity, resulting in poor learning effects. Some researchers use machine learning methods to model abstract syntax trees. Bielik et al. [109] and Raychev et al. [89] use context-free grammar and decision trees respectively to model abstract syntax trees and predict syntax rules. Deep learning methods have more complex network structures than machine learning, requiring new networks to be customized according to the structure of the abstract syntax tree. Rabinovich et al. [85] proposed Abstract Syntax Networks (ASNs), which is an extended form of the standard encoder-decoder framework. Different from the sequence model, its decoder is composed of many submodules, each corresponding to a syntactic structure of the abstract syntax tree. During decoding, the state of the recurrent neural network is passed from one module to another, thereby achieving the transfer of information between modules. In the process of tree generation, different modules are selected to execute different networks according to the current state, thereby achieving the prediction of tree structure. Different from Rabinovich et al. [85] who design different modules according to the syntactic structure of the tree, Dong and Lapata [30, 31] propose a hierarchical tree decoder SEQ2TREE. After the decoder obtains the results from the encoder, it uses the recurrent neural network to generate Tokens with tree depth of 1. When predicting as a non-terminal node, the state of the non-terminal node is used as input to predict the next layer of the tree, until there are no non-terminal nodes. Sun et al. [97] proposed a tree network based on convolutional neural networks, effectively solving the long-dependency problem faced in program generation, and using abstract syntax trees for program generation. On this basis, Sun et al. [98] introduced Transformer into program generation and proposed a

tree structure framework, further solving the problems of long-range dependency and tree structure encoding.

1.1.2 Large-Language-Model-Based Code Generation

Recently, large language models have demonstrated impressive code generation capabilities and can now accomplish simple programming tasks. However, when evaluated on more complex and subtle problems, these models still perform poorly. Such problems require problem-solving skills, not just converting instructions into code. For example, programming competition problems that require understanding algorithms and complex natural language are still extremely challenging. This section details several existing representative works on code large language models.

Codex. Codex [19] is a model based on GPT3 and trained using publicly available code on GitHub. The authors of Codex focused on studying Codex’s Python code writing capability. Codex is the kernel model of the first generation Copilot. In addition to training a series of models of different parameter sizes based on GPT3 using GitHub code, the authors of Codex also released a new evaluation benchmark, HumanEval, for evaluating the correctness of code generated by large language models through natural language descriptions. In the case of generating only one solution per problem, Codex can generate correct code for 28.8% of the problems, GPT-J can generate correct code for 11.4% of the problems, while GPT-3 cannot generate any correct code. In addition, Codex also found that repeatedly sampling from the model is a surprisingly effective strategy that can generate effective solutions for difficult problems. Using this method, Codex generates 100 solutions per problem and successfully solves 70.2% of the problems. Then the authors of Codex conducted a detailed analysis of Codex’s generated results, revealing the limitations of Codex. Mainly,

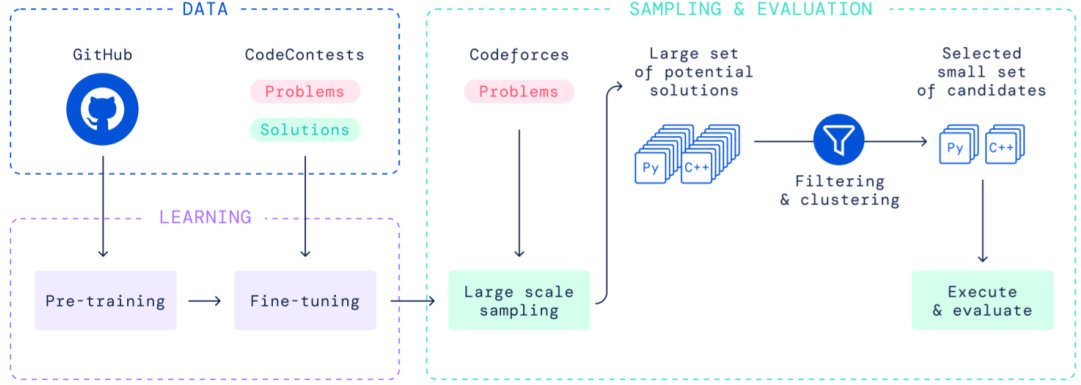


Figure 1.1: AlphaCode workflow diagram [80]

Codex cannot generate correct solutions for problems with descriptions that are too long and complex, and Codex has difficulty generating special variables for certain code to be generated. Finally, Codex discusses the potentially broader impacts of deploying powerful code generation models, including safety, security, and economic value.

AlphaCode. AlphaCode [68] is a code generation model proposed by Google in November 2021, which can generate new solutions for these problems that require deeper reasoning. In simulated evaluations of recent programming competitions on the Codeforces platform, AlphaCode averaged in the top 54.3% in competitions with more than 5,000 participants. AlphaCode found that three key modules are crucial for achieving good code generation results: (1) a dataset with high diversity and containing only programming competition problems for training and evaluation, (2) a large-scale and efficient model architecture, and (3) large-scale model sampling to explore the search space, then clustering based on program behavior to group candidate solutions into a small number of submissions. Figure 1.1 shows the overall workflow of AlphaCode.

CodeGen. CodeGen [80] points out that although the popularity of large language models has driven the development of automatic code generation, the

training data resources of current large language models are limited and the models are not sufficiently open-sourced. To address the above limitations, CodeGen trained and released a series of large language models with up to 16.1B parameters for natural language and programming language data, and open-sourced the training library JAXFORMER. Compared with the SOTA work on code generation at that time, CodeGen achieves better Zero-shot Python code generation on HumanEval, proving the effectiveness of CodeGen in code generation work. Interestingly, after CodeGen released the above version in March 2022, another version was updated at the end of September 2022, adding a multi-turn feedback generation function. CodeGen further studied the multi-step paradigm of program synthesis, where a program is decomposed into multiple prompts specifying subproblems. To this end, CodeGen constructed an open evaluation benchmark, Multi-Turn Programming Benchmark (MTPB), consisting of 115 different problem sets that are decomposed into multi-turn prompts. The authors of CodeGen’s analysis of CodeGen’s performance on MTPB shows that compared with single-turn generation, the multi-turn approach provides the same intent to CodeGen and significantly improves the code generation effect.

InCoder. InCoder [39] was proposed by researchers from Facebook and UCB. InCoder can perform code generation (by generating code from left to right) as well as code completion (through infilling techniques). InCoder is trained to generate code from a large amount of permissively-licensed code, where code regions are randomly masked and moved to the end of each file, allowing bidirectional context for code infilling. InCoder is the first code large language model capable of filling arbitrary code regions. It is evaluated on challenging tasks (such as type inference, comment generation, and variable renaming) in the Zero-shot setting. InCoder found that the capability under bidirectional context conditions greatly improves the performance of these tasks, while the code generation effect

on HumanEval is on par with other code large language models.

CodeGeeX. CodeGeeX [126] is a multi-programming-language code generation pre-trained model with 13 billion parameters. CodeGeeX is implemented using Huawei’s MindSpore framework and trained on 192 nodes (a total of 1,536 domestic AI processors) in Peng Cheng Laboratory’s ”Peng Cheng Cloud Brain II”. As of June 22, 2022, CodeGeeX has been pre-trained for two months on a code corpus (>850 billion tokens) of more than 20 programming languages. CodeGeeX is a large-scale pre-trained programming language model based on Transformers. It is a left-to-right autoregressive decoder that takes code or natural language identifiers (tokens) as input and predicts the probability distribution of the next identifier. CodeGeeX contains 40 Transformer layers, with a hidden layer dimension of 5,120 for each layer’s self-attention block, a feed-forward layer dimension of 20,480, and a total of 13 billion parameters. The maximum sequence length supported by the model is 2,048. The training corpus of CodeGeeX consists of two parts. The first part is the open-source code dataset The Pile [40] and CodeParrot. The Pile contains some open-source repositories on GitHub with more than 100 stars, and CodeGeeX selected code from 23 programming languages from it. The second part is supplementary data, directly crawled from GitHub open-source repositories for Python, Java, and C++ code; to obtain high-quality data, CodeGeeX selects code repositories according to the following criteria: 1) having at least 1 star; 2) total size $<10\text{MB}$; 3) not in previously open-sourced code datasets. CodeGeeX also removed files that meet any of the following conditions: 1) average line length greater than 100 characters; 2) automatically generated by automatic generation tools; 3) containing less than 40% of letters in the alphabet; 4) greater than 100KB or less than 1KB. To let the model distinguish different languages, CodeGeeX adds a prefix at the beginning of each sample in the form of [comment symbol] ”language: [language]”, for example: ”language: Python”.

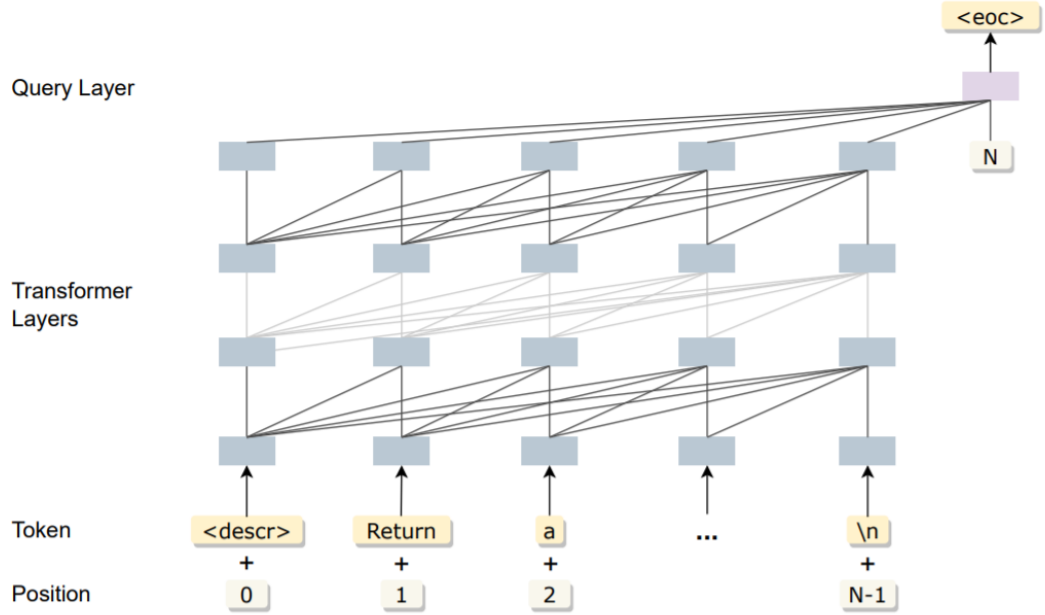


Figure 1.2: PanGu- architecture [23]

CodeGeeX uses the same tokenizer as GPT-2 [8], and treats spaces as special identifiers. The vocabulary size is 50,400. The entire code corpus contains 23 programming languages and a total of 158.7 billion tokens (excluding padding).

PanGu-Coder. PanGu-Coder [23] is a pre-trained large language code model that contains only the decoder part, adopting the PANGU- architecture to generate text-to-code, i.e., synthesizing programming language solutions given natural language problem descriptions. Figure 1.2 shows the architecture diagram of the PANGU- architecture. PanGu-Coder is trained using a two-stage strategy: the first stage uses Causal Language Modeling (CLM) to pre-train on raw programming language data, while the second stage uses a combination of Causal Language Modeling and Masked Language Modeling (MLM) training objectives, focusing on the downstream task of text-to-code generation, and training on semantically equivalent natural language descriptions and their corresponding code function implementations. Figure 1.3 shows the two-stage training diagram

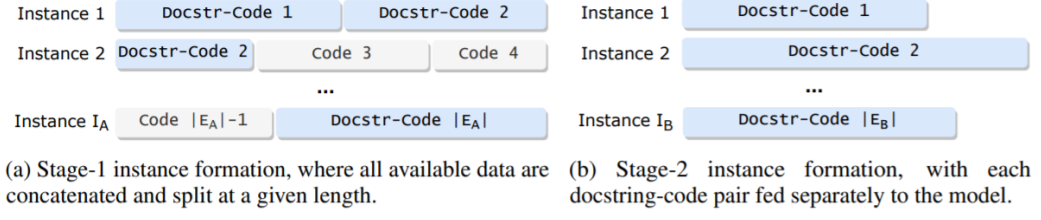


Figure 1.3: PanGu-Coder two-stage training diagram [23]

of PanGu-Coder. Finally, PanGu-Coder discusses PanGu-Coder-FT, which is fine-tuned on algorithmic programming problems and self-contained functions covered by tests through continuous integration testing. The authors evaluated PanGu-Coder, focusing on whether it generates functionally correct code, and demonstrated that it has comparable or better effects than similarly sized models (such as Codex). At the same time, compared with Codex, the context window of the model used by PanGu-Coder is smaller and the training data used is less.

StarCoder. StarCoder [67] was jointly launched by the HuggingFace community and ServiceNow, with a total of 69 authors from 38 units. All models, data, and code are open-sourced, and its Python code generation effect can reach the same level as the closed-source OpenAI code-cushman. Another contribution of StarCoder is to hire human labor to mark user-sensitive information that may exist in open-source code, use the labeled information to train a model for detecting sensitive information, and use this model to remove sensitive information from StarCoder’s training data. In addition, StarCoder also considers removing projects in GitHub whose licenses do not allow usage during training.

WizardCoder. WizardCoder [71] trained a more effective model based on StarCoder using instruct prompt fine-tuning. It uses 5 types of prompt templates, which not only gives the model interactive capabilities, but also surpasses StarCoder and other open-source code large language models in code generation effects.

1.1.3 Project-Level Code Generation Benchmarks

Benchmarks containing project-level functions are important, but constructing project-level code generation benchmarks is very difficult, mainly due to the following two reasons: 1) We need to ensure that the selected projects can be compiled and sandboxed to achieve successful execution of different projects. However, it is difficult to successfully build and run many existing open-source projects. 2) Project-level testing is too slow. To speed up testing, for the target function, it is usually necessary to reduce a large number of coverage test cases from the project to which it belongs. However, statically determining whether test cases cover the target function requires building function dependency graphs and coverage analysis. In addition, when the target function is not covered by the test cases of the project to which it belongs, before writing high-quality test cases, the constructor of the benchmark needs to have a deep understanding of the complex logic of the project to which it belongs. Most existing benchmarks (e.g., HumanEval [19], MultiPL-E [15], DS-1000 [63], and AiXBench [46]) use Pass@K to verify the correctness of the generated functions, and they only contain self-contained functions. Although DS-1000 can verify the correctness of generated non-self-contained functions, these non-self-contained functions are limited to calling API functions from seven specific widely-used third-party libraries in data science. Concode [56] is not a benchmark; it is a large dataset containing more than 100,000 functions from approximately 33,000 open-source Java repositories. In Concode, functions from the same repository are divided into training sets, validation sets, and test sets, so that deep learning models can be trained on training sets and validation sets and then tested on test sets. Each function contained in Concode is a pair: its natural language comment and code implementation.

Specifically, HumanEval is a benchmark for evaluating the functional correct-

ness of code generated by code generation models from docstrings. It contains 164 handwritten programming problems (each problem’s function is described by a natural language docstring) along with solutions (in Python). Each solution includes a function signature, function implementation, and multiple unit tests. Following HumanEval, researchers proposed AiXBench [46] to benchmark Java code generation models. AiXBench contains 175 problems for automatic evaluation and 161 problems for manual evaluation. The authors of AiXBench proposed a new metric for automatically evaluating the correctness of generated code, and a set of similarity criteria for manually evaluating the overall quality of generated code. MultiPL-E [15] is the first multilingual parallel benchmark for text-to-code generation. MultiPL-E extends HumanEval and MBPP [3], supporting 18 programming languages. MultiPL-E includes a set of compilers and an evaluation framework for converting code generation benchmarks (including unit test cases, docstrings, Python-specific programming language syntax, and type annotations) from Python to other programming languages. MultiPL-E contains two parallel benchmark parts (i.e., HumanEval and MBPP), which are used in the generated code of 18 programming languages, including various programming paradigms, language features, and popularity. While the aforementioned benchmarks contain only self-contained functions, another benchmark called DS-1000 [63] contains non-self-contained functions. In particular, DS-1000 contains 1,000 problems, covering seven widely-used Python data science libraries: NumPy, Pandas, TensorFlow, PyTorch, Scipy, Scikit-learn, and Matplotlib. The authors of DS-1000 alleviated the data leakage problem by manually modifying the implementation of functions, and emphasized using data from actual development scenarios to build DS-1000. Although DS-1000 contains non-self-contained functions, DS-1000 has two main limitations. First, functions in DS-1000 are only collected from seven data science third-party libraries in Python. Second, although some

functions in DS-1000 have contextual dependencies on third-party libraries, the development scenario reflected by DS-1000 is far from pragmatic code generation, because no function in DS-1000 has contextual dependencies on user-defined functions outside the target function. Concode [56] is a new large dataset containing more than 100,000 Java class problems from open-source projects. From a public Java project on GitHub, the authors of Concode collected functions each with at least one contextual dependency, along with natural language (i.e., javadoc-style method comments) and code tuples. The authors of Concode collected Java functions from approximately 33,000 repositories, and then divided them into training sets, validation sets, and test sets at the granularity of repositories (rather than at the granularity of functions). Although functions in Concode contain contextual dependencies, Concode only uses BLEU as its evaluation metric and does not evaluate the correctness of generated functions, because each problem in Concode does not include test cases.

1.2 Research on Key Technologies of Unit Test Case Generation

1.2.1 Traditional Unit Test Case Generation Research Methods

Traditional unit test case generation technology in this paper refers to non-deep-learning-based test case generation technology, which can be mainly divided into 5 categories: (1) search-based unit test case generation, (2) random-adaptation-based unit test case generation, (3) combination-based unit test case generation, (4) symbolic-execution-based unit test case generation, and (5) model-based unit test case generation.

Search-Based Software Testing (SBST) [48] generates test cases by using search-based algorithms and designing an objective function related to the object under test. This usually refers to the parameter input of the test cases and the construction of test sequences. SBST is widely used in multiple different test objectives, including structural [49, 73, 74, 101], functional [114], non-functional [115], and state-property-based [27] test objectives. The main issue that all SBST methods consider is how to design one or more objective functions related to the object under test. The objective function is mainly used to guide the search algorithm and better test the object under test by searching the space of test inputs. Since any test objective can in principle be reformulated as an objective function, this method is general. Although many literatures [47] tend to focus on evolutionary algorithms, there are still many different search-based algorithms available.

Random testing is one of the most basic and popular means in testing methods. The concept of random testing is concise, easy to implement, and can be used as a component in other testing methods. In cases where source code is not available, random testing may be the only relatively practical technology. Furthermore, random testing is one of the few methods whose error detection capability can be analyzed theoretically. Adaptive Random Testing (ART) has been proposed to enhance RT. Adaptive Random Testing (ART) [21, 22] has been proposed as an enhancement of RT. Some empirical studies have shown that fault-causing inputs tend to form continuous fault regions, so non-fault-causing inputs should also form continuous non-fault regions [117]). If previous test cases have not shown failures, the coverage area of the new test case should be far from the already executed non-fault-causing test cases. Therefore, test cases should be evenly distributed in the input domain. It is this concept of evenly distributing test cases in the input domain that constitutes the basic intuition of ART. Anti-random

testing [72] mainly aims to spread test cases in the input domain. However, a fundamental difference is that ART is a non-deterministic method. While except for the first test case being randomly selected, anti-random testing is essentially a deterministic method. Another difference is that anti-random testing requires testers to specify the number of test cases in advance, while ART has no such constraint.

Combinatorial testing [36, 60] has become a common technique in the toolbox of software testers. In combinatorial testing, the focus is on selecting samples of input parameters (or configuration settings) that cover a specified subset of the combinations of elements to be tested. The most common form of combinatorial testing is Combinatorial Interaction Testing (CIT). There has been a lot of work on the field of combinatorial testing in the past few years, including new techniques for generating CIT samples and their applications in new fields.

Symbolic execution [26, 32, 59] is a program analysis technique that analyzes the code of a program to automatically generate test data for the program. Extensive work exists that demonstrates the usefulness of this technique in a wide range of software engineering problems, including test data generation. However, this technique has at least three fundamental problems that limit its effectiveness on real-world software. First, in most applications of symbolic-execution-based test data generation (e.g., improving code coverage or exposing errors), ideally, the goal is to discover all feasible program paths. For example, to effectively expose errors, it is necessary to discover a large subset of all feasible paths and show that each of these paths is error-free, or many of them expose errors. Therefore, the system must be able to discover as many different feasible program paths as possible within the available time limit. Second, the human development cost required to apply symbolic execution to any program should be controlled within a user-acceptable range.

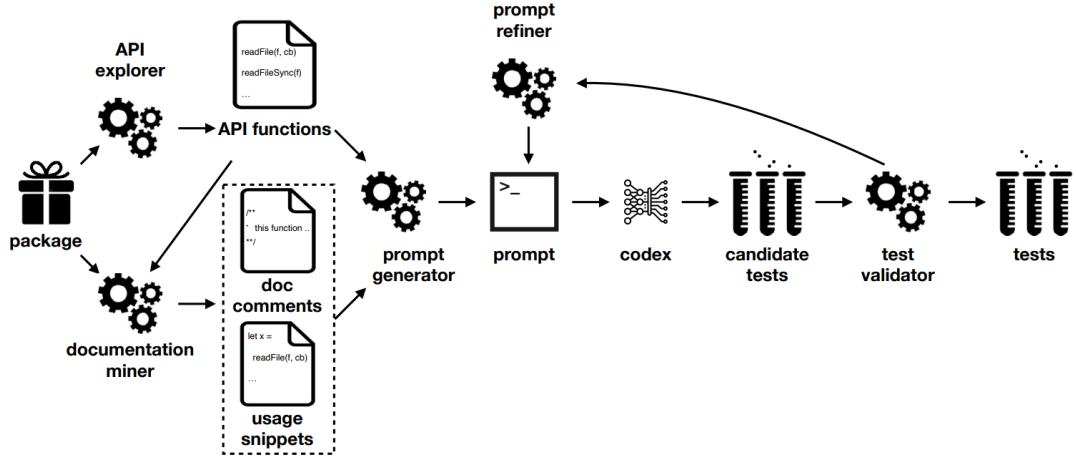


Figure 1.4: TestPilot workflow diagram [91]

Model-based testing (MBT) mainly has three schools: the axiomatic approach [33], the Finite State Machine (FSM) approach [10], and the Labeled Transition System (LTS) approach [96]. In behavioral MBT, the System Under Test (SUT) is given as a black box that accepts inputs and produces outputs. The SUT has an internal state that changes when processing inputs and producing outputs. The model describes possible input/output sequences at a selected abstraction level and is linked to the implementation through a conformance relation. The test selection algorithm derives test cases from the model by selecting a finite subset from the potentially infinite set of sequences specified by the model, using test criteria based on test hypotheses. Test selection can be performed by generating a test suite in a suitable language before test execution, called offline test selection, or by intervening in test execution, called online test selection.

1.2.2 Unit Test Case Generation Based on Deep Learning and Large Language Models

Traditional unit test case generation technology can achieve good results in coverage and in finding bugs that cause program crashes, but they have four main shortcomings: (1) The generated unit test cases have poor readability. (2) The composition ability for complex objects is relatively weak. (3) The coverage for complex programs is relatively low. (4) In non-regression testing scenarios, the generated test cases have very weak capability in finding logical bugs. Aiming at the shortcomings of traditional unit test case generation methods, in recent years, to solve the above limitations, researchers have explored applying deep learning techniques to address the shortcomings of traditional unit test case generation methods. Currently, large-language-model-based and deep-learning-based unit test case generation technologies can be mainly divided into the following two categories: (1) Generating complete unit test cases. TestPilot [91] was proposed by GitHub and Northwestern University. It directly uses OpenAI’s code-cushman model (the kernel of Copilot) to generate test cases. TestPilot focuses on test case generation for the JavaScript language, and generates test cases by constructing prompts for test case generation. Figure 1.4 shows the workflow diagram of TestPilot. Experimental results show that TestPilot can improve the code coverage of existing popular JavaScript third-party libraries. ChatTester [124] was proposed by the research team at Fudan University. It comprehensively evaluates ChatGPT’s ability to generate unit tests and proposes a high-quality unit test case generation framework based on ChatGPT. Figure 1.5 shows the workflow diagram of ChatTester. This work shows that large-language-model-based unit test case generation is a promising direction. The scenario where developers no longer need to manually write unit test cases may not be a dream in the near future. (2) Generating assertions in unit test cases. Tufano et al. [104] proposed

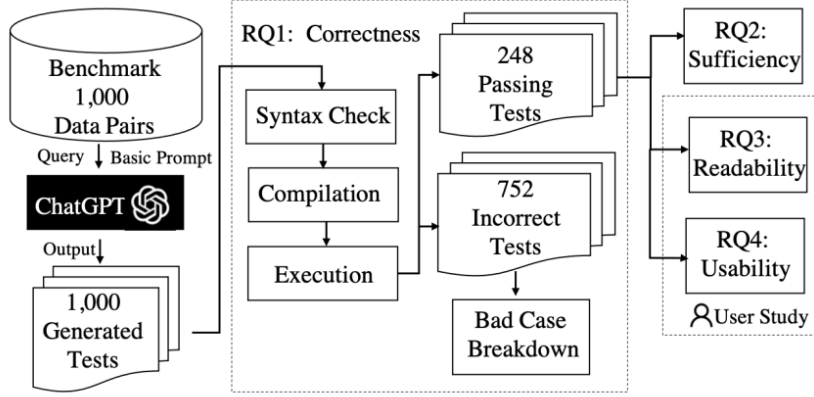


Figure 1.5: ChatTester workflow diagram [124]

AthenaTest, which uses the BART model pre-trained on natural language and source code corpora, and then fine-tunes it on the Methods2Test [105] dataset to generate entire unit test cases given the method under test and its context. They found that this method can correctly test 43% of the methods under test, and 16% of the candidate methods are correct. The authors of Methods2Test generated a test suite for Java projects by identifying the association between methods under test and their corresponding tests. They collected a large supervised dataset from GitHub, containing more than 780,000 pairs of methods under test and test methods using the JUnit Testing framework. Through their method, they can identify 27.41% of the test associations, and the parseability of test cases generated by this method is 86.69%. However, they did not evaluate the correctness or usefulness of the generated test cases in identifying errors. CODAMOSA [64] proposes a combination of search-based testing and LLM-based testing. They use search-based software testing until its coverage improvement reaches a limit, then they use the Codex model [19] to provide example test cases for functions with insufficient coverage. Compared with using SBST and LLM alone, CODAMOSA achieves significantly higher coverage on more test sets (173 and 279) than the reduction in coverage on (10 and 4). Their method can gen-

erate fault-reproducing test cases for 33% of the research cases in the Defects4j test set. Assertion generation aims to generate meaningful assertion statements given the context under test and the corresponding text prefix. The test prefix is the unit test case with assertions (assertion statements, try-catch clauses, etc.) removed. Due to the difficulty of generating entire unit test cases, researchers proposed generating useful assertions, such as assertions as a medium. ATLAS (AuTomatic Learning of Assert Statements) [113] is the first method to use deep neural models to generate assertions. They can generate assertions with a BLEU-4 score of 61.85% with a beam size of only one. Yu et al. [122] introduced a method that integrates information retrieval techniques, using Jaccard coefficient, Overlap coefficient, and Dice coefficient with the deep neural network model ATLAS. Through their method, they can improve the BLEU score to 78.86%. TOGA (Test Oracle Generation Assertion) [29] is proposed to use a unified Transformer-based neural model to generate try-catch statements and assertion statements for unit test case assertions. For generating try-catch clauses, their exact match accuracy is 86%, and the exact match accuracy of assertion statements is 69%. Tufano et al. [106] proposed using a BART pre-trained model trained on natural language and source code corpora, and then fine-tuning it on the ATLAS dataset. When considering the top 5 predicted assertion statements, ATLAS was able to predict exact matches in 50% of cases.

1.3 Limitations of Existing Related Research Work

Traditional code generation work, when directly given natural language, generates code of poor quality. Large-language-model-based code generation work focuses on how to better generate self-contained-type functions such as competition problems and algorithm problems, and does not focus on how to better generate code

for real development scenarios. The existing benchmarks for evaluating the code generation effect of large language models also only focus on self-contained-type functions such as competition problems and algorithm problems. Traditional unit test case assertion generation work generates assertions by first executing and then capturing the execution results. This method cannot generate assertion statements that find logical errors. Large-language-model-based unit test case assertion generation work generates assertions through the implementation of the function under test (without executing) and its corresponding functional description. Although this approach can generate assertions that find logical errors, it generates too many incorrect assertions, which would misjudge correct functions under test.

1.4 Summary of This Chapter

This chapter mainly introduced the research related to code generation technology and the research related to unit test case generation technology. First, it introduced traditional code generation technologies, and then introduced large-language-model-based code generation technologies (such as Codex, AlphaCode, CodeGen, and PanGu-Coder) and code generation benchmarks (such as HumanEval, AixBench, DS-1000, and Concode). Next, this chapter introduced traditional unit test case generation technologies and large-language-model-based and deep-learning-based unit test case and assertion generation technologies. Although large-language-model-based code generation and unit test case generation have made great progress, the effectiveness of large language models in generating code for real development scenarios still has great room for improvement, and at the same time, unit test case assertions generated by large language models have many false positives.

Chapter 2

Introduction

This chapter first elaborates on the research background and significance of automatic code generation, then explores the main issues and challenges faced by large-language-model-based automatic code generation, and finally discusses the research content of this paper and its organizational structure.

2.1 Research Background and Significance

The software development process mainly includes stages such as requirement analysis, design, coding, testing, delivery, acceptance, and maintenance. The coding and testing stages are those with the largest development cost and require the largest code volume. In modern software engineering, the scale and complexity of code continue to increase, and developers face the dual challenges of how to efficiently manage code quality and development speed [9, 14]. Especially in data-driven [2, 41] and service-oriented development modes [82, 87], developers often need to write large amounts of boilerplate code, such as database access layer code, object model code, and interface layer code. Writing such code is both time-consuming and error-prone, and the style, structure, and logic of the code may

differ significantly among different developers, affecting the overall consistency of the project and later maintenance costs. To effectively solve these problems, automatic code generation technology emerged. By defining a set of rules, models, or templates, developers can automatically generate relatively standardized code, greatly reducing the workload of manually writing repetitive code. Just as agile development emphasizes "rapid delivery of value", automatic code generation tools can help teams quickly build basic frameworks, allowing developers to invest more energy in implementing core business logic, thereby improving project development efficiency and quality. In academia and industry, the application of automatic code generation is becoming increasingly widespread, covering various aspects from data model generation, code framework generation to test case generation. Many large enterprises have integrated automatic code generation tools into their development workflows, such as Microsoft's Entity Framework Code First [65] and Hibernate's [4] ORM mapping tools, which are typical examples of automatic code generation technology in practical applications. These tools not only improve development efficiency but also bring significant improvements in cross-team collaboration, code consistency, and maintainability.

However, although the above tools can automatically generate boilerplate-type code to help developers improve development efficiency, they cannot generate correct programs based on natural language descriptions, nor can these tools generate code related to business logic. Automatically generating boilerplate-type code is still far from being the silver bullet in the field of software engineering. Starting from 2010, researchers have tried to help developers generate code related to business logic through deep-learning-based methods [50, 55], but it is difficult for the generated code to be functionally correct, and the code generation effect is still far from satisfactory. In 2021, OpenAI proposed Codex [19] based on a large language model, which greatly improved the effectiveness of generating

code through natural language. The introduction of automatic code generation technology can not only solve the inefficiency problem in the traditional manual code writing process, but also demonstrate its important significance and value in the following aspects: (1) Improve development efficiency and productivity. In modern software development processes, writing repetitive code is a major task of developers. Through automatic code generation, development teams can quickly generate high-quality basic code frameworks with less investment, greatly reducing the time for repetitive labor. Developers only need to focus on implementing core business logic rather than repeatedly writing boilerplate code for each new function or module. This not only improves development efficiency but also shortens the product development cycle. Even though the code related to business logic generated by existing large-language-model-based methods cannot guarantee fully correct functionality, because it can generate as correct code as possible in many cases, developers only need to modify a small amount of code to get the correct code they want. Therefore, large-language-model-based code generation work can help developers improve development efficiency in non-template-type code generation such as directly generating code from natural language, code completion, and business logic code generation. (2) Reduce human errors and improve code quality. Manually writing code often leads to errors due to oversight or excessive complexity, especially when the development team is large or the project complexity is high. Style differences between different developers may also lead to inconsistent code. Automatic code generation uses standardized templates to generate code, effectively reducing the risk of errors caused by human factors and ensuring code consistency and quality. This not only reduces potential technical debt in the project but also improves system stability and maintainability. (3) Promote consistency and standardization of software architecture. In large projects, maintaining consistency in code style, structure, and design patterns

is a daunting task. Automatic code generation can generate code according to preset architectural patterns and rules, ensuring that every part of the system follows the same design principles. This is of great significance in team collaborative development, code review, and later maintenance, greatly simplifying project management difficulty. (4) Support agile development and rapid prototyping. Agile development emphasizes rapid response to change, continuous delivery, and frequent release. In this context, automatic code generation tools can help developers quickly build basic frameworks and prototype systems, thereby verifying the feasibility of products and user needs in a short time. Through automatic code generation, development teams can flexibly respond to frequent changes in requirements and ensure efficient task completion within tight development cycles. (5) Improve system maintainability and scalability. Automatic code generation not only helps to quickly build the system in the early stage of the project, but also plays an important role in later maintenance and expansion of the project. By adjusting the templates or rules of the code generator, it is possible to adapt to new requirements and architectural changes without rewriting a lot of code. This greatly reduces the cost of code refactoring and provides strong technical support for the long-term evolution of the system. (6) Support continuous integration and continuous delivery. In modern software development processes, Continuous Integration (CI) and Continuous Delivery (CD) have become industry standards. Automatically generated code can be seamlessly integrated into CI/CD pipelines, ensuring that each code change can undergo automated testing and verification, thus reducing uncertainty and risk brought by code changes. Through automatic code generation technology, development teams can better support continuous integration and automated deployment processes, improving system stability and delivery efficiency.

Automatic code generation technology has demonstrated wide application

value and significance in improving development efficiency, reducing error risks, increasing industrial value, enhancing system maintainability, and supporting agile development. As the complexity of software systems continues to increase, automatic code generation will continue to play an important role and generate greater value, helping development teams more efficiently respond to rapidly changing market demands, helping software enterprises reduce software development costs, and promoting the continued development and progress of related technologies in the field of software engineering. Therefore, researching and applying automatic code generation technology has not only practical significance (improving software development efficiency, reducing software development costs, etc.), but also provides new perspectives and methods for future software development.

With the continuous expansion of software development scale and the increase of system complexity, software quality assurance has become a crucial task in software engineering. Unit testing, as a part of quality assurance [25, 45], can promptly detect and fix problems in the code in the early stages of software development. However, writing unit test cases is often a time-consuming and labor-intensive task, especially in complex software systems, where the comprehensiveness and effectiveness of test coverage face many challenges. Writing unit test cases with high coverage for complex software systems is very challenging for developers. To improve testing efficiency and coverage, automatic generation technology of unit test cases emerged, and has gradually become an important tool in the software development process [11, 38, 81]. Unit Test Case Generation automatically generates corresponding test cases by analyzing code structure, business logic, and input-output conditions, thereby ensuring that each functional module of the code is fully verified. This technology can not only greatly reduce the workload of developers manually writing test cases, but also generate test cases for dif-

ferent scenarios and boundary conditions, improving test comprehensiveness and code robustness. Through automatically generating unit test cases, development teams can test more efficiently, improve software quality, and ensure the long-term maintainability of the system. In recent years, with the wide application of development modes such as Agile Development [94], Continuous Integration (CI) [37], and Continuous Delivery (CD) [18], the demand for test automation has become increasingly urgent. Automatically generating unit test cases can not only be seamlessly integrated with CI/CD processes, but also help developers complete comprehensive code verification work in a short time, ensuring system stability after each code change. Whether for startup projects or complex large systems, unit test case automatic generation tools provide strong support for software quality assurance. The traditional manual writing of unit test cases is time-consuming and labor-intensive, especially in large-scale or frequently changing projects. Ensuring comprehensive test coverage and the effectiveness of test cases is a highly challenging task. Due to time and resource constraints, developers may overlook certain boundary conditions, exception cases, or special input scenarios, resulting in incomplete test coverage, which in turn affects the stability and reliability of the system.

The automatic generation technology of unit test cases is a solution proposed to address these challenges. By automatically analyzing the execution paths, logical branches, and possible inputs and outputs of the code, the generator can automatically generate a large number of test cases covering various scenarios in the code. This not only improves test coverage but also reduces the risks brought by human oversight. Developers can use automatic generation tools to create a comprehensive set of test cases in a short time, improving the depth and breadth of system testing, and quickly performing verification when code changes. The application of automatic generation of unit test cases can not only solve the inef-

efficiency problem of traditional manual writing of test cases, but also demonstrate its important significance and value in the following aspects: (1) Improve testing efficiency and coverage. Manually writing test cases often requires a lot of time, especially when the test scope is wide and the code complexity is high, making it more difficult to ensure comprehensive test coverage. Through automatic generation of unit test cases, developers can quickly generate a large number of test cases, thereby covering various logical paths and exception handling cases in the code in a shorter time. This not only greatly improves testing efficiency but also effectively improves test coverage, enabling potential defects in the code to be discovered as early as possible. (2) Reduce omissions and errors caused by human factors. When manually writing test cases, developers may overlook certain boundary conditions or exception scenarios due to lack of experience or cognitive bias. Automatically generated unit test cases can comprehensively analyze possible input-output situations based on the code structure, avoiding test blind spots caused by human oversight. In addition, automatic generators can ensure that the generated test cases follow established testing norms and standards, reducing inconsistencies and potential errors brought by manual writing of tests. (3) Promote the improvement of software quality. Automatically generated test cases can usually cover various boundary conditions, special scenarios, and exception handling paths of the code, ensuring that every part of the code is fully tested. This can not only improve the robustness of the code but also ensure that the system can still operate stably in various extreme situations. Through automatically generated test cases, development teams can discover potential defects in the code earlier, and fix them in time in the early stages of development, avoiding problems flowing into the production environment and improving the quality of the final product.

Although unit test case automatic generation technology has shown signifi-

cant advantages in improving testing efficiency and software quality, it still faces some challenges in practical applications. For example, how to generate efficient, comprehensively-covering test cases that reflect real scenarios is still a problem that researchers and developers need to solve. Although traditional unit test case generation tools can generate unit test cases with high coverage, they cannot find assertions for logical errors. Although deep-learning-based and large-language-model-based unit test case methods can generate assertions that find logical errors, the false positive rate of the assertions they generate is relatively high. How to generate assertions that can find logical errors while generating as few incorrect assertions as possible is a necessary condition for automatic test case generation methods to be practically usable. In addition, for complex systems and projects with complex business logic, how to ensure that the generated test cases do not cause additional performance burdens, and how to conduct precise testing for complex inputs and outputs, also require further research.

2.2 Research Questions and Difficulties

2.2.1 Large-Language-Model-Based Code Generation

The concept of automatic program generation can be traced back to 1957, when Alonzo Church proposed a problem of synthesizing a specified circuit based on mathematical requirements during the Summer Institute of Symbolic Logic at Cornell University. Although this problem is only limited to the circuit domain rather than program design, it is generally considered the beginning of program generation research, so the research community also refers to the program synthesis problem as the "Church problem". In the 1960s, researchers in the artificial intelligence field further explored the concept of "automatic programmer". Since 2010, deep-learning-based code generation has become a key research direction,

mainly including natural-language-based code generation and existing-code-based code completion. However, deep-learning-based code generation methods mainly rely on text similarity metrics (such as BLEU) or similarity metrics improved for code characteristics (such as CodeBLEU) in evaluation standards. This evaluation method has obvious flaws. For example, in the bubble sort algorithm, two pieces of code differ only in the judgment condition of the if statement in the for loop. Although the text similarity is extremely high, the functions they implement may be completely opposite. Since code generation problems pay more attention to the functional correctness of the generated code, similarity-based evaluation methods cannot measure the effectiveness of deep-learning-based code generation techniques in real code generation scenarios. In 2021, OpenAI used the massive code data on GitHub to train Codex, a model with hundreds of billions of parameters. To evaluate the effectiveness of Codex, the research team constructed the HumanEval test set containing 164 algorithm programming problems, and for the first time introduced test cases to evaluate the accuracy of the code generated by the model, and received wide recognition: subsequently, various code large language models proposed by academia and industry, such as PanGuCoder and StarCoder, all use HumanEval as the evaluation benchmark. Although test-case-based evaluation methods have received wide recognition, existing code generation evaluation benchmarks still have obvious deficiencies in evaluating the code generation capabilities of code large language models in real development scenarios. In particular, existing code generation evaluation benchmarks mainly focus on the generation effectiveness of code large language models on self-contained functions (i.e., functions that can be executed only using the given programming language’s standard environment), with relatively little research on the generation effectiveness of non-self-contained functions. This paper systematically conducts research on real-development-scenario-oriented code generation problems from

three aspects: evaluating the effectiveness of real-development-scenario code generation, collecting high-quality real-development-scenario fine-tuning code, and training real-development-scenario-oriented code generation models. It mainly solves the following two key problems:

(1) Existing code generation benchmarks cannot evaluate the effectiveness of code large language models in generating code for real development scenarios

The code generation benchmark HumanEval, widely used by the code generation research community, contains only self-contained functions, i.e., functions that can be successfully executed only by the given programming language’s environment. In real development scenarios, self-contained functions account for less than 20% [123], so the evaluation results on HumanEval do not represent the effectiveness of code large language models in code generation for real development scenarios. Researchers need to consider constructing code generation benchmarks for real development scenarios. Constructing real-development-scenario-oriented code generation benchmarks requires considering how to collect functions reflecting real development scenarios, how to collect contextual information needed by models to generate code through natural language, and how to construct an efficient evaluation environment.

(2) Existing code large language models lack targeted training for code generation in real development scenarios

Existing large language models focus more on generating self-contained functions, and most of the high-quality fine-tuning data used are self-contained functions [19, 23]. Researchers need to consider training code large language models for real development scenarios, especially for code generation with contextual dependencies. Training real-development-scenario-oriented code large language models requires considering how to collect high-quality fine-tuning functions for

real development scenarios, how to incorporate code-dependent contextual information during the training process, and how to combine the incorporated contextual information and natural language descriptions during the training process.

2.2.2 Large-Language-Model-Based Unit Test Case Assertion Generation

Although traditional unit test case generation methods [38, 81] can achieve high code coverage, because their test case generation processes depend on the specific implementation of the function under test, and the unit test case assertions are obtained by executing the function under test, it is difficult for them to discover defects at the program logic level [1, 93]. In contrast, although test case generation methods based on deep learning and large language models can identify logical-type errors, the accuracy of the unit test case assertions they generate is low, with a high proportion of false positive problems (i.e., incorrectly judging correct programs as incorrect). This paper focuses on the problem of generating high-quality unit test case assertions, and systematically conducts research from three aspects: generating high-quality unit test case assertions based on large language models, generating high-quality unit test case assertions based on a combination of information retrieval and deep learning, and detecting incorrect unit test case assertions. It mainly solves the following two key problems:

(1) Existing unit test case assertion generation methods have difficulty effectively generating assertions that can reveal program logic defects. The generation of unit test case assertions mainly depends on the requirement specifications of the code. Traditional unit test case generation tools generate unit test case assertions by executing the code. This implementation-based method is essentially difficult to discover logical defects in the program. Although deep-learning-based assertion generation methods can theoretically iden-

tify logical defects, the accuracy of their assertion generation can only reach about 30%. Compared with traditional test case generation tools, although such methods have potential in discovering logical defects, their actual effects still have a large gap from the requirements of engineering practice.

(2) Assertions generated by existing unit test case assertion generation methods have high false positives.

Through experimental verification, this research shows that even using the current state-of-the-art code large language models [92] to generate unit test case assertions for functions in the HumanEval benchmark test set, the false positive rate still exceeds 50%. Such a high false positive rate of assertions would incorrectly identify functionally correct programs as defective programs, which not only reduces the reliability of testing but also significantly affects developers' work efficiency and development experience.

2.3 Research Content of This Paper

Aiming at generating code and unit test cases, this paper focuses on how to better generate code and unit test cases based on large language models, mainly including constructing evaluation benchmarks for code large language models reflecting real development scenarios, designing and implementing unit test case assertion generation based on a combination of information retrieval and deep learning, and large-language-model-based unit test case assertion optimization. Overall, the main work and innovations of this paper include the following:

- This paper constructs CoderEval, a benchmark for validating the code generation capability of large language models, used to evaluate the ability of large language models to generate code for real development scenarios, supporting both Python and Java languages. This paper points out that

the currently used benchmark HumanEval for validating the code generation capability of large language models can only validate functions of algorithm-problem type (i.e., self-contained functions that can be executed without relying on any information outside the function), and cannot validate the ability of large language models to generate code for real development scenarios. This paper selects representative open-source projects by analyzing the tag and star count information of all Python and Java projects on GitHub, adds contextual information to the selected functions through program analysis techniques to add the functionality of evaluating the effectiveness of the contextual information of the code generated by the model to CoderEval, eliminates the data leakage problem by inviting developers to manually label natural language descriptions, and improves the usability of CoderEval by providing a Docker testing environment and incremental compilation technology.

- This paper proposes a unit test case assertion generation method based on a combination of information retrieval and deep learning, including information-retrieval-based unit test case generation techniques, automatic adaptation techniques based on heuristic search and deep neural networks, and techniques based on combining information retrieval and deep learning. This paper first attempts large-language-model-based unit test case generation methods, and through comparison with the unit test case assertion generation methods based on combining information retrieval and deep learning, points out that although large language models can achieve better results in unit test case assertion generation, they still have complementarity with the information-retrieval-and-deep-learning-based unit test case assertion generation methods, which consume fewer resources and have faster generation speed.

- This paper proposes a large-language-model-based method for detecting incorrect unit test case assertions, which can detect 64% of the incorrect unit test case assertions generated by large language models. This paper points out that although unit test case assertions generated by existing deep-learning-based methods can find logical errors in code compared to traditional unit test case assertion generation methods, they also bring a very high false positive rate, that is, judging correct code as incorrect. This paper first analyzes the proportion of incorrect unit test case assertions generated by existing large language models, and finds that more than 50% of the unit test case assertions generated by existing large language models are incorrect. To detect incorrect unit test case assertions generated by large language models, this paper provides the given function signature and docstring of the target problem to the large language model to generate candidate postconditions. In this step, this paper designs a prompt format for the large language model to generate formal and executable postconditions. Secondly, among the candidate postconditions, this paper uses the given I/O examples to filter out candidate postconditions that are violated by at least one of the given I/O examples. Finally, this paper uses the remaining postconditions to detect incorrect unit test case assertions.

2.4 Thesis Organization

Overall, this paper focuses on the two stages in the software development process that have the largest code volume and the highest development cost: coding and testing. First, aiming at the fact that existing large-model-based code generation methods focus on code generation benchmarks containing only self-contained-type code such as competition problems, this paper proposes a real-development-

scenario-oriented code generation benchmark in Chapter 3. Then, this paper proposes a unit test case assertion generation method in Chapter 4. Unit test case assertions can not only find errors in code, but also improve the quality of code generation. It is well known that test-driven software development can improve code quality, but because traditional test case generation tools cannot generate assertions that find logical errors, and writing test assertions is very time-consuming and labor-intensive for developers, test-driven software development has not been widely adopted. Since the generation of unit test case assertions is still in the early stages of research, the accuracy of assertions generated by the most advanced existing methods is only close to 50%, and incorrect assertions would misjudge correct functions under test. Therefore, this paper detects incorrect assertions in Chapter 5. Chapter 4 focuses on performance and Chapter 5 focuses on soundness, both of which are indispensable for current assertion generation. Finally, Chapter 6 summarizes the full paper.

Chapter 3

Function-Level Code Generation Evaluation Benchmark for Real Development Scenarios

This chapter first introduces the background of constructing a function-level code generation evaluation benchmark for real development scenarios, then introduces the construction process of the CoderEval benchmark, then introduces the performance of different code large language models on CoderEval and the usage status of CoderEval. Finally, a summary of this chapter is provided.

3.1 Introduction

In recent years, software engineering academia and industry have attempted to use machine learning techniques to solve code generation tasks, especially Transformer-based [108] large generative pre-trained language models [8, 12, 75, 86], such as Codex [19], AlphaCode [68], InCoder [39], CodeGen [80], PanGu-Coder [23], and ChatGPT [92]. Given natural language descriptions specifying

the functionality of the target function, these models can generate both self-contained functions (i.e., functions that only call or access built-in functions and standard libraries) and non-self-contained functions.

To fairly and comprehensively evaluate the effectiveness of the above models, researchers have proposed many representative benchmarks [3, 15, 16, 19, 44, 46, 51, 63, 111, 112], which have been widely used by existing code large language model work. HumanEval, released together with Codex [19], is a benchmark for evaluating the functional correctness of code generated by code large language models for the Python language. HumanEval consists of 164 developer-written problems, each including a function signature, a docstring described in natural language, and multiple unit test cases. Recently, DS-1000 [63] was proposed to evaluate the effectiveness of code generation models in generating code that depends on third-party data science libraries. In addition to benchmarks for the Python language, there are benchmarks for Java (AiXBench [46]) and other programming languages (MultiPL-E [15]).

Although these existing benchmarks are important and can validate the effectiveness of code large language models to a certain extent, there is a gap between these benchmarks and code generation in real development scenarios (e.g., code in open-source projects or code in actual development scenarios of companies). On the one hand, these benchmarks mainly focus on self-contained-type functions. Except for DS-1000, all of the above benchmarks include only self-contained functions. Although DS-1000 includes non-self-contained functions, these non-self-contained functions only call API functions from seven specific widely-used third-party libraries in data science. On the other hand, non-self-contained functions usually exist in real development scenarios. After analyzing the 100 most popular projects written in Java and Python respectively on GitHub, this chapter finds that non-self-contained functions account for more than 70% of open-source

project functions.

To solve the problem that existing benchmarks can only validate the effectiveness of code large language models in generating self-contained functions, this chapter proposes a context-aware benchmark `CoderEval`, used to evaluate the effectiveness of code large language models in generating code for real development scenarios. According to the dependency elements outside the target function, this chapter divides the code to be generated into six levels (i.e., self-contained, slib-runnable, plib-runnable, class-runnable, file-runnable, and project-runnable). Detailed information is described in Section 3.2.2. Considering that existing benchmarks such as `HumanEval` only cover the first two levels, `CoderEval` can provide a more practical and representative evaluation to assess the effectiveness of code large language models in generating code in actual development scenarios. This chapter defines functions belonging to the first two levels as self-contained functions, and functions at other levels as non-self-contained functions.

To make `CoderEval` more effective and more representative in validating the effectiveness of code large language models in generating code for real development scenarios, `CoderEval` selects functions from actual projects in three steps. `CoderEval` selected 230 functions each from 43 Python projects and 10 Java projects. First, `CoderEval` selects open-source projects from the 14 most common tags on GitHub. For each function in each open-source project (since in Java, functions are customarily called methods, this chapter uniformly uses "function" to represent functions in Python and methods in Java), `CoderEval` extracts the original docstring (i.e., the natural language description of the function), function name and signature, code implementation, and corresponding test code (if it exists), to form a function-level code generation problem. In addition, `CoderEval` uses program dependency analysis techniques to analyze detailed contextual information of each function (where context refers to code elements such

as types, APIs, variables, and constants defined outside the target function but in dependent third-party libraries, current class, file, or project), and provides it as `all_context` (all accessible contexts) and `oracle_context` (actually used contexts), enabling fine-grained evaluation of model effectiveness. Second, to reduce data leakage (i.e., the original docstring is likely to be used as training data by large language models), this chapter recruited 13 experienced engineers to provide manually labeled versions of natural language descriptions, serving as a second natural language description docstring (i.e., manually labeled docstrings) to supplement the original docstrings. Third, to improve the accuracy of evaluation, CoderEval checks the test coverage of existing tests provided by each project, and manually writes additional tests to achieve high test coverage.

To automatically evaluate the effectiveness of code large language models, this chapter uses the Pass@K metric [19] (evaluating whether the functional implementation of the function to be generated is correct) and a newly proposed metric named Acc@K (described in Section 3.3.3) to evaluate the generated code. Since CoderEval contains non-self-contained functions with contextual dependencies, CoderEval builds a project-level execution platform to provide a ready-to-use runtime environment that can automatically evaluate the functional correctness of the generated code. This chapter develops the validation platform of CoderEval based on Docker. CoderEval clones all projects and sets up project environments. Given multiple pieces of code generated by the model, the platform can automatically place the generated code at the appropriate file position in the project to be executed.

Table 3.1 shows the average cyclomatic complexity and average lines of code in CoderEval, HumanEval, and Concode. This section finds that the cyclomatic complexity of functions in CoderEval is similar to that in HumanEval, and the average lines of code in CoderEval is more than that in HumanEval.

Table 3.1: Statistical Comparison among CoderEval, HumanEval, and Concode

Statistical Benchmark	Cyclomatic Complexity	Line of Code
CoderEval-Python	4.71	32.0
CoderEval-Java	3.10	10.2
HumanEval	3.62	7.8
Concode	1.43	4.8

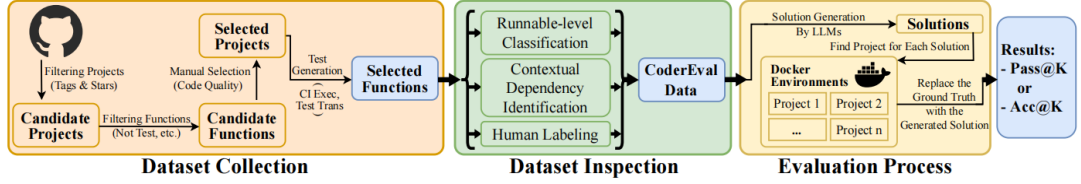


Figure 3.1: CoderEval construction flow chart

3.2 Construction of the CoderEval Benchmark

This section introduces the construction of CoderEval, with the construction process shown in Figure 3.1. The construction process of CoderEval includes three stages: data collection, data review, and evaluation process.

3.2.1 Project and Function Selection

To make CoderEval practical and diverse, this section selects functions from various open-source projects through four steps. (1) CoderEval selects candidate projects by crawling the tags of all projects on GitHub, selecting the 14 tags with the highest frequency and projects with high star counts. For each tag, CoderEval selects the top five projects with the highest star counts. The 14 types of tags are "gson", "music", "logging", "chat", "websocket", "mvc", "leetcode", "microservices", "jdbc", "json", "crud", "datastructures", "log4j", and "serialization". (2) CoderEval extracts all functions in the selected projects, keeping only those that are not test functions, interface functions, or deprecated functions, are accompa-

nied by function-level English comments, can successfully run in the validation platform (Section 3.2.4), and pass the original test cases. (3) CoderEval manually screens high-quality functions from the candidate functions. The main criterion for screening is whether a function often appears in actual development scenarios and whether developers would write this function in actual development. (4) CoderEval determines the final projects to select based on the number of selected functions each project contains. This process can help CoderEval compile fewer projects when selecting the same number of functions. Finally, CoderEval selected 230 functions from 10 Java projects. To keep consistent with the number of functions in Java’s CoderEval, this section also selected 230 functions from 43 Python projects.

The reason for manually selecting high-quality functions (the third step described above) in this section is that this section hopes CoderEval evaluates the helpfulness of model-generated code to developers. CoderEval hires experienced developers to select functions that may be written in real scenarios, mainly following the following five rules. (1) Contains less than 10 tokens of contextual dependencies. Too many contextual dependencies make it difficult for the model to generate correct functions. Functions that are difficult for all code large language models to correctly implement are not suitable to be included in the benchmark. (2) Functions often written in actual development scenarios, as judged by 13 recruited experienced engineers. The rationale behind this rule is that different developers have different preferences for commonly used functions. (3) Functions containing docstrings that can reflect the implementation of the function. (4) Functions with more than three lines of code implementation. (5) Non-test functions or deprecated functions.

3.2.2 Data Review

Manually labeled natural language descriptions. When using large language models (LLMs) to generate code, choosing what prompt to use has a great impact on the quality of the generated code. To study the impact of different prompts on the effectiveness of code generated by large language models, this section recruited 13 experienced engineers, all with at least 3 years of Python/Java experience, and had them provide manually labeled versions of the description (named docstring) for each problem in CoderEval. There are three main reasons for including manually labeled docstrings in CoderEval: (1) Including manually labeled docstrings helps alleviate the memorization effect (i.e., the original docstrings have a high probability of being used during the pre-training stage of large language models) and explore the impact of manually labeled prompts on the quality of code generated by large language models. (2) Including manually labeled docstrings helps study the performance of large language models on different docstrings. (3) Including manually labeled docstrings helps provide high-quality docstrings for functions in CoderEval. In the initial research of this section, this section found that the original docstrings of problems in CoderEval (from selected open-source projects) are very diverse, far beyond the scope of describing functionality, which are usually used as prompts for large language models. The original docstrings can play multiple different roles, such as explaining the internal logic of the function, introducing external usage and behavior, declaring effects and notes, etc.

Contextual dependency identification. One of the main differences between HumanEval [19] and CoderEval is that CoderEval considers the contextual dependencies of the target function, which refers to code elements defined outside the target function. These code elements are necessary to execute the target function. Table 3.2 shows the dependencies belonging to each type of dependency

Table 3.2: Dependencies belonging to each contextual dependency type

Dependency Type	Dependencies	Examples in Python
self-contained	built-in types/functions, no need to import	<code>min()</code> , <code>print()</code>
slib-runnable	standard libraries/modules, no need to install	<code>os</code> , <code>subprocess</code> , <code>sys</code>
pplib-runnable	publicly available libraries on pypi/maven	<code>unittest2</code> , <code>requests</code>
class-runnable	code outside the function but within class	<code>self.xxx</code> , <code>X.f()</code>
file-runnable	code outside the class but within the file	<code>func()</code> , <code>URL</code> , <code>name</code>
project-runnable	code in the other source files	<code>superclass</code> , <code>utils</code>

Each runnable level must depend on the dependencies defined at this level, must not depend on the dependencies defined at its subsequent levels, and may or may not depend on dependencies defined at its previous levels.

along with examples. This section determines the contextual dependencies of a function in three steps by performing program analysis on the project to which it belongs. (1) Before the analysis, this section first builds a knowledge base to help identify different references built into Python/Java, as well as different standard or public libraries imported from project-specific code. For this purpose, this section caches all built-in types/functions/variables/constants and standard library names for each Python version from 3.0.0 to 3.10.0 and Java versions from 1.8 to 17, as well as all publicly available libraries on GitHub, namely *pypi.org* and the *Maven* Central Repository. (2) After obtaining the knowledge base, given the function to be analyzed, this section parses the source file containing the function to obtain a list of type/function/variable/constant definitions. (3) This section uses static program analysis to identify all dependency elements, i.e., external references and calls defined outside the function under analysis, and classifies them into three categories: `TypeReference`, `VariableReference`, and `APIInvocation`. More specifically, `TypeReference` refers to user-defined classes or standard types (e.g., `"List"`, `"subprocess"`, and `"os"`), `VariableReference` refers to user-defined variables or objects, and `APIInvocation` refers to user-defined func-

tions or functions in standard libraries or third-party libraries. CoderEval uses the `oracle_context` information of a function to represent its contextual dependencies, which are determined by the previous three steps. `Oracle_context` information can not only be used to evaluate the accuracy of contextual dependencies in code generated by LLMs (as done in the experiments of this section), but can also be used as part of the input of code large models during the inference stage, exploring using `oracle_context` information as additional prompts to improve the effectiveness of code generated by code large models. However, it is usually difficult for users of code large models to know the `oracle_context` information in advance, so CoderEval also includes the `all_context` information of a function (i.e., all types, variables, and APIs defined/imported in the file to which the function belongs), which can additionally be used to prompt code large models to generate code containing part or all of the `all_context` information. Note that in the experiments of this section, this section does not use `all_context` information during the inference stage; here, this section only uses the original docstring, manually labeled docstring, function name, and function signature of the target function. This section hopes that `all_context` information is valuable to the research community, and this section plans to study the impact of `all_context` information (as part of the prompts used) on the effectiveness of code generation in future work.

Runnable level classification. As shown in Table 3.2, this section divides target functions into six runnable levels, each referring to the range in which the function can successfully run. These levels include self-contained, slib-runnable, plib-runnable, class-runnable, file-runnable, and project-runnable. Each runnable level must depend on the dependencies defined at this level, must not depend on the dependencies defined at its subsequent levels, and may or may not depend on the dependencies defined at its previous levels. For example, a function at the plib-runnable level depends on public libraries, so as long as the required Python

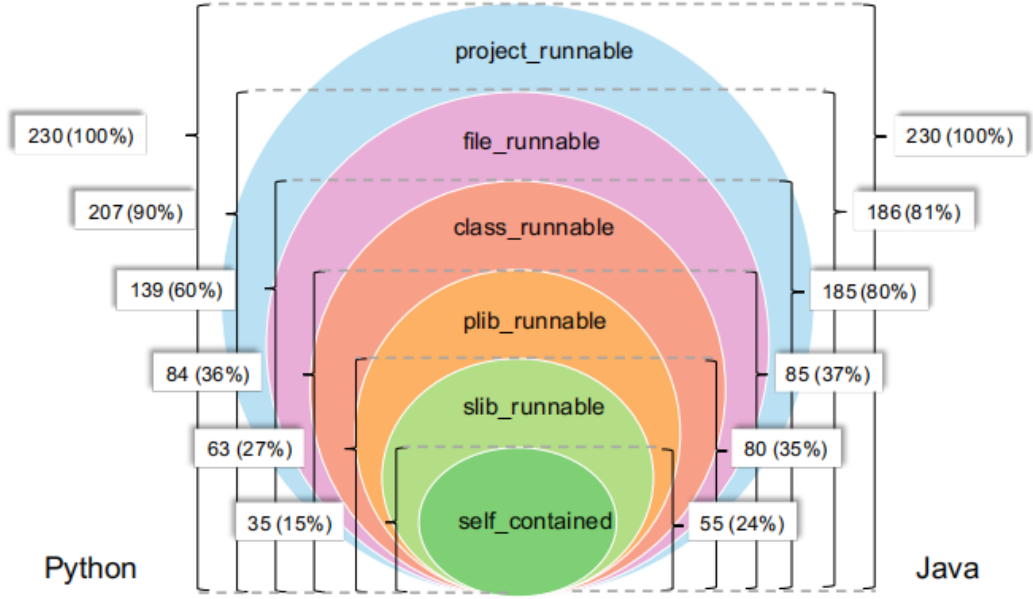


Figure 3.2: Distribution of runnable levels in CoderEval

version is used and the libraries are installed and imported, it can successfully run outside its class, source file, and project. The left and right parts of Figure 3.2 respectively show the distribution of runnable levels in Python’s CoderEval and Java’s CoderEval. Most (84%) of the functions collected in the Python version of CoderEval are file-runnable, indicating that the `oracle_context` information in their respective files is necessary and critical for LLMs to correctly generate the target functions. Functions belonging to class-runnable account for about half (54%) of all functions; considering that object-oriented programming is very popular in the current development environment, this result is reasonable. In contrast, functions belonging to self-contained and slib-runnable (self-contained functions) account for only 17% of the functions in the Python version of CoderEval, but these functions account for 100% in HumanEval. This result shows the limitations of HumanEval when used for model optimization and evaluation, especially in evaluating the effectiveness of code large language models in generating code for

actual development scenarios. Similar to Python’s CoderEval, self-contained and slib-runnable functions account for only 35% in the Java version of CoderEval.

3.2.3 Test Case Construction

For each project containing CoderEval, this section first runs the unit test cases included in the original project and measures the branch coverage of each function (from the project) included in CoderEval. When the branch coverage of a function does not reach 100%, the author of this paper manually wrote additional test cases for that function to achieve 100% branch coverage (if that function itself can reach 100% test coverage).

Based on the unit test cases in the original projects (except those handwritten by the authors), this section builds test cases in CoderEval through two steps. (1) To obtain the test cases corresponding to a function in CoderEval, this section constructs a static function call graph from the source code, and then uses the graph to select all test cases that can reach that function. In this way, when CoderEval evaluates the correctness of a function, this section avoids the high cost of executing all test cases in the project to which the function belongs. (2) To unify and simplify the interface for calling test cases in CoderEval, this section automatically converts the corresponding test cases of functions in CoderEval (which usually depend on testing frameworks such as JUnit and TestNG) into non-test functions in the "main" function of new Python/Java files. This section names these new Python/Java files "NewTestFile".

3.2.4 Experimental Environment Construction

To provide a ready-to-use runtime environment for automatically executing and evaluating functions generated by the model, this section built an evaluation

platform based on Linux Docker images, which can provide a virtual sandbox for safe distribution and execution of test threads.

Construction of Python execution environment for CoderEval. To evaluate the accuracy of code generated by code large language models, this section needs to clone all Python projects in CoderEval and set up executable environments for them. To avoid Python/library version conflicts, in the root directory of each repository, this section first uses pyenv to set the local Python version to the latest version specified in the CI configuration or documentation, and then uses venv to create a separate virtual environment for the project to which the target function belongs. After that, this section uses pip to install all dependencies and triggers the execution of test cases included in the original project to ensure the successful setup of the runtime environment. After the runtime environment is successfully established, our validation platform automatically replaces the target function (in the project to which it belongs) with the generated function, calls the generated function with the test input of the test case corresponding to the target function, and compares the actual test output with the expected output of the corresponding test case to determine whether the function under test passes all its corresponding test cases.

Construction of Java execution environment for CoderEval. Similar to Python’s evaluation platform, Java’s evaluation platform also first clones all Java projects in CoderEval. This section ensures that all Java projects in CoderEval can be executed in Java version 17. Unlike Python code, Java code needs to be compiled before execution, so the platform automatically compiles the test files (i.e., "NewTestFile") in advance (including the test cases corresponding to the target functions), and then replaces the target function with the generated function in the project to which it belongs. Then the platform runs the "javac" command to incrementally compile the changed files (whose changes are caused

by function replacement), and runs the "java" command to execute the bytecode of "NewTestFile". The return value of the "java" command indicates whether all corresponding test cases of the target function pass. CoderEval's test platform uses the "-cp" parameter of the "javac" and "java" commands to import the class files of the given test file and its dependent files/libraries. In addition, if compilation fails, CoderEval's Java test platform (without executing the test cases) directly considers the generated code to be incorrect.

3.3 Experimental Evaluation

3.3.1 Experimental Models

This chapter focuses on models that support both Python and Java code generation. CodeGen, PanGu-Coder, and ChatGPT all support Java and Python. Specifically, this chapter chose CodeGen because CodeGen has a dedicated Python language model (CodeGen-mono) and a model supporting multiple languages (CodeGen-multi). This chapter chose PanGu-Coder because PanGu-Coder has two models designed for Python and Java, respectively. This chapter chose ChatGPT because ChatGPT is the most effective and largest-parameter multi-language model. Choosing these models helps compare the effectiveness of monolingual code large language models and multilingual code large language models. The reason this chapter did not choose CodeGeeX [126] is that its model only has a 13B-sized version, and due to limited computational resources, this chapter cannot complete the experiment. This chapter plans to evaluate the effectiveness of CodeGeeX and other recent open-source models (such as StarCoder [67], WizardCoder [71], and MagiCoder [116]) on CoderEval in future work.

3.3.2 Experimental Setup

For PanGu-Coder, this chapter uses the PanGu-Coder model with its default setting of 300M parameters. For the Python version of CodeGen, this chapter uses the CodeGen-Mono model with its default setting of 350M parameters. Since CodeGen does not have a single-language version for Java, on Java’s CoderEval, this chapter uses the CodeGen-multi model instead. For ChatGPT, this chapter uses “gpt-3.5 turbo” in the experiment. It should be noted that the model parameter size of ChatGPT is much larger than that of the other two models.

In the inference stage, for all models, this chapter sets the maximum window length to 1024. This chapter adopts nucleus sampling [53]: the number of samples is 10 (i.e., 10 code snippets are generated for each function to be generated). This chapter sets the temperature parameter of the model to 0.8.

3.3.3 Evaluation Metrics

Similar to HumanEval, this chapter adopts the Pass@K metric for LLMs. Specifically, given 230 functions in CoderEval, the LLM’s Pass@K measures the percentage of functions (among the 230 functions) for which at least one of the top K samples (i.e., the number of generated codes) generated by the LLM correctly (judged by running the corresponding test cases) generates the function to be generated. In the experiment, this chapter sets the total number of samples generated by an LLM (denoted as n) to 10, and then calculates Pass@K for LLMs with K values of 1, 5, and 10, respectively. To avoid the problem of high sampling variance, this chapter uses the unbiased estimation metric Pass@K implemented in HumanEval for ChatGPT. Please note that although in previous work of PanGu-Coder [23] and CodeGen [80], n was set to 200, this chapter sets n to 10 as Copilot plug-in did, because this chapter believes that in actual set-

Table 3.3: Performance of three models on CoderEval and HumanEval

Benchmark	Model	Python			Java		
		Pass@1	Pass@5	Pass@10	Pass@1	Pass@5	Pass@10
CoderEval	CodeGen ¹	9.48%	19.58%	23.48%	13.91%	27.34%	33.48%
	PanGu-Coder ²	11.83%	20.93%	27.39%	25.43%	37.39%	43.04%
	ChatGPT ³	21.04%	27.31%	30.00%	35.39%	42.77%	46.09%
HumanEval	CodeGen ¹	10.20%	19.79%	22.80%	5.78%	9.00%	9.45%
	PanGu-Coder ²	13.42%	21.48%	22.73%	8.21%	15.88%	18.63%
	ChatGPT ³	39.21%	64.09%	72.96%	38.21%	59.35%	67.23%

¹ This section uses the CodeGen-Mono model of 350M size, which is specifically trained for the Python language. This section uses its default model parameter settings. Since CodeGen does not have a single-language version for Java, in the Java version of CoderEval experiments, this section uses the multilingual version of CodeGen-Multi instead.

² PanGu-Coder has two models for Python and Java. This section uses the default parameter settings of its 300M parameter-scale model.

³ This section uses "gpt-3.5-turbo" as the ChatGPT kernel model for experiments.

tings, sampling 10 times is more feasible and reasonable under acceptable cost and response time.

In addition to Pass@K, this chapter also proposes an Acc@K metric for LLMs based on oracle_context tokens (i.e., dependency elements contained in the oracle_context information) to evaluate the LLM’s ability to generate each individual oracle_context token in the top K samples generated by the LLM. In particular, the LLM’s Acc@K measures the percentage of target functions for which each of their oracle_context tokens is contained in at least one of the K samples generated by the LLM. Note that other metrics can be formulated to evaluate the LLM’s ability to generate functions containing all oracle_context tokens in the target function (considering and not considering the order of tokens respectively). These metrics pose more challenging requirements. This chapter plans to explore these metrics and other metrics in future work.

3.3.4 Experimental Results

(1) Performance of CodeGen, PanGu-Coder, and ChatGPT on CoderEval and HumanEval, especially in generating self-contained and non-self-contained functions.

Overall code generation performance. Table 3.3 shows the overall code generation performance of CodeGen, PanGu-Coder, and ChatGPT on CoderEval and HumanEval. On HumanEval, the overall performance reported by the three models on the Python version of HumanEval is the overall performance reported in the respective papers corresponding to these models. These papers do not report the effectiveness of their models on the Java version of HumanEval. This section evaluated the effectiveness of these models and listed the experimental results in Table 3.3. Since the functions in the Python version of CoderEval do not overlap with the functions in the Java version of CoderEval, this section cannot directly compare the effectiveness of the three models on the Python version of CoderEval and the Java version of CoderEval. On HumanEval and CoderEval (Python and Java), ChatGPT consistently outperforms the other two models. Analyzing the reasons, this section believes that the parameter scale of ChatGPT is much larger than the other two models. CodeGen and PanGu-Coder perform worse on HumanEval than on CoderEval, because although the functions in HumanEval are all self-contained functions (i.e., functions that can only call or access built-in functions and standard libraries), the complexity of these functions is higher than that of self-contained functions in CoderEval. The functions in HumanEval tend to be more algorithmic, while self-contained functions in CoderEval tend to be simple and practical (e.g., converting integers to strings or floats). ChatGPT is more effective on HumanEval than on CoderEval. ChatGPT’s strong algorithm learning ability makes ChatGPT perform well on HumanEval. However, when generating code that depends on context information,

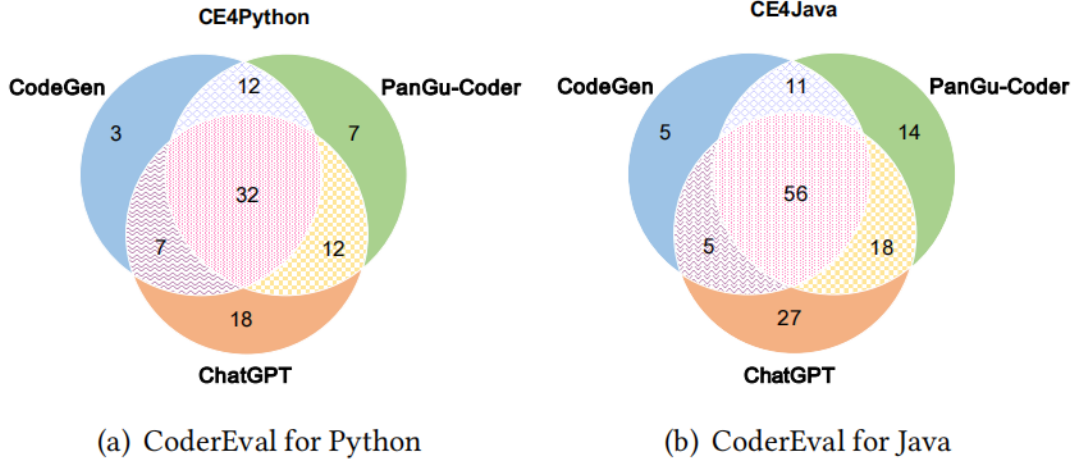


Figure 3.3: Number of problems solved by CodeGen, PanGuCoder, and ChatGPT on CoderEval respectively

the prompt given to ChatGPT does not include context information, resulting in ChatGPT’s poor performance on CoderEval.

Complementarity and overlap of the three models. Figure 3.3 shows the number of functions correctly generated by the three models on CoderEval, which have both differences and overlaps. This section uses problems with Pass@10 of 1 (i.e., the model generates 10 pieces of code for the function to be generated, and at least one piece of code has a correct implementation) to count the pass problems for each model. If the model can generate at least one code (out of 10 generated codes) that passes all test cases, this section considers this problem solvable by the model. Among the 230 functions in the Python version of CoderEval, the total number of functions that at least one of the three models can correctly generate is 91 (complementarity), while each of the three models can solve 32 problems (overlap). On the Java version of CoderEval, the total number of functions that at least one of the three models can correctly generate is 136 (complementarity), while each of the three models can correctly generate 56 functions (overlap). This high degree of complementarity indicates that different models

have their unique abilities, and the high degree of overlap indicates that the three models have consistent ability to generate correct problems on certain problems. This section analyzed the problems that all three models could correctly solve and found that more than half of them belong to self-contained functions. More specifically, the total number of overlapping problems in CoderEval is 88, of which 52 belong to self-contained functions. Note that self-contained functions account for only 30% in CoderEval. Therefore, all three models are good at generating self-contained functions (ChatGPT performs particularly well). The experimental results show that there are two important research directions in the field of function-level code generation for real development scenarios: (1) Extending the capabilities of code large language models to generate non-self-contained functions; (2) Exploring how to combine the code generation capabilities of different models.

Comparison of effectiveness of different models in generating self-contained and non-self-contained functions in CoderEval. Since all functions in HumanEval are self-contained functions, this section further compares the effectiveness of the three models in generating self-contained functions in CoderEval with their effectiveness in generating non-self-contained functions in CoderEval. The standalone and non-standalone rows in Table 3.4 show that in the Python version and Java version of CoderEval, the effectiveness of the three models in generating self-contained functions is significantly higher than in generating non-self-contained functions. Considering that more than 70% of functions in open-source projects are non-self-contained functions, improving the model’s ability to correctly generate `oracle_context` information is crucial for code generation in real development scenarios. This section evaluates self-contained and non-self-contained functions at detailed runnable levels. Self-contained functions include two runnable levels, self-contained and slib-runnable, while non-self-

Table 3.4: Comparison of effectiveness of different models in generating self-contained and non-self-contained functions in CoderEval

Runnable-Level	Model	Python			Java		
		Pass@1	Pass@5	Pass@10	Pass@1	Pass@5	Pass@10
standalone:	CodeGen	18.10%	33.38%	38.10%	26.25%	46.40%	52.50%
	PanGu-Coder	19.52%	31.30%	38.10%	43.88%	57.23%	62.50%
	ChatGPT	35.87%	43.56%	47.62%	64.88%	68.65%	70.00%
- self-contained	CodeGen	22.57%	36.75%	40.00%	27.27%	49.37%	56.36%
	PanGu-Coder	23.71%	34.93%	40.00%	48.36%	57.94%	61.82%
	ChatGPT	52.29%	60.08%	82.86%	61.82%	66.21%	67.27%
- slib-runnable	CodeGen	12.50%	29.17%	35.71%	24.00%	39.87%	44.00%
	PanGu-Coder	14.29%	26.76%	35.71%	34.00%	55.68%	64.00%
	ChatGPT	15.36%	22.92%	28.57%	71.60%	74.00%	76.00%
non-standalone:	CodeGen	6.23%	14.38%	17.96%	7.33%	17.17%	23.33%
	PanGu-Coder	8.92%	17.02%	23.35%	15.60%	26.81%	32.67%
	ChatGPT	15.45%	21.18%	23.35%	19.67%	28.97%	33.33%
- plib-runnable	CodeGen	4.76%	16.16%	23.81%	0%	0%	0%
	PanGu-Coder	13.33%	22.75%	28.57%	0%	0%	0%
	ChatGPT	21.43%	28.06%	28.57%	0%	0%	0%
- class-runnable	CodeGen	5.82%	9.05%	10.91%	8.30%	19.21%	26.00%
	PanGu-Coder	7.82%	15.04%	21.82%	19.90%	32.59%	40.00%
	ChatGPT	8.73%	12.57%	14.55%	22.40%	31.49%	36.00%
- file-runnable	CodeGen	7.79%	20.19%	25.00%	0%	0%	0%
	PanGu-Coder	9.41%	19.04%	26.47%	0%	0%	0%
	ChatGPT	21.03%	29.09%	32.35%	0%	0%	0%
- project-runnable	CodeGen	3.91%	8.33%	8.70%	6.14%	14.89%	20.45%
	PanGu-Coder	6.09%	10.51%	13.04%	7.95%	17.33%	20.45%
	ChatGPT	9.57%	12.08%	13.04%	16.14%	27.20%	31.82%

contained functions include four runnable levels, plib-runnable, class-runnable, file-runnable, and project-runnable. From Table 3.4, this section finds that except for Pass@5 and Pass@10 of class-runnable on Java’s CoderEval, ChatGPT’s code generation effectiveness is superior to the other two models on all runnable levels. This section analyzes that the main reason is that the parameter scale of ChatGPT is much larger than the other two models.

In conclusion, on the Python version and Java version of CoderEval, the effectiveness of the three models in generating self-contained functions in CoderEval is significantly higher than in generating non-self-contained functions. Different

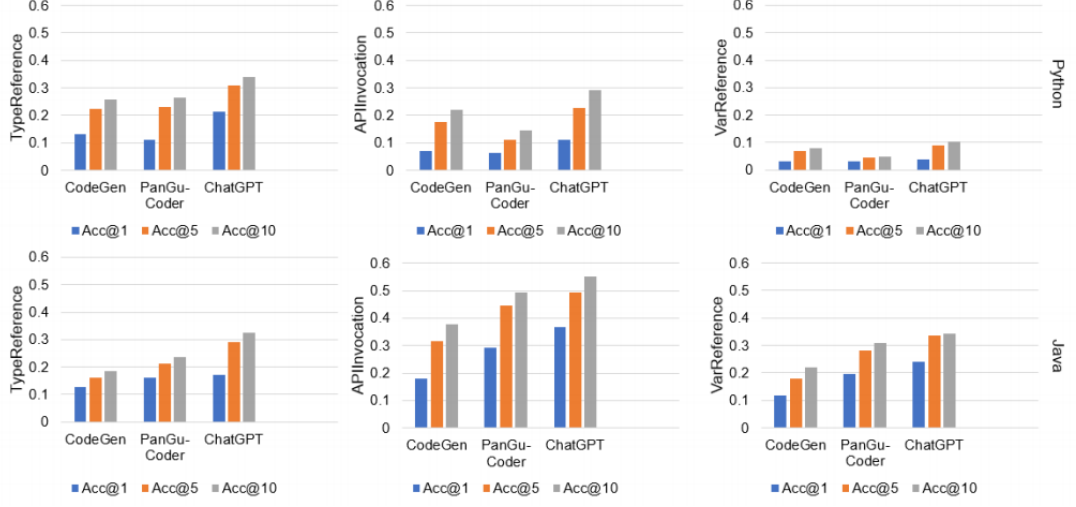


Figure 3.4: Accuracy of three models in generating oracle_context tokens

models have their unique capabilities in code generation. How to combine the code generation capabilities of different models is a direction worth studying.

(2) The effectiveness of correctly using oracle_context information in code generated by different models.

To further study the ability of large language models to generate oracle_context tokens, this section analyzes the oracle_context tokens in the generated code, focusing on three types of tokens: TypeReference, APIInvocation, and VarReference (note that VarReference does not include variables defined in the target function).

Figure 3.4 shows the Acc@K values of the three models on CoderEval (for a function to be generated, when the model generates one or multiple codes, the effectiveness of correctly generating various types of tokens in the groundtruth code); the results shown in the figure are the averages of 10 experiments. This section finds that the Acc@K values of the three models are consistent with Pass@K (reflecting that the generated functions can pass test cases), which indicates that the correctness of the function-level code generated by large language models is

proportional to their ability to generate context-related tokens. To further explore the ability of different models to generate different types of `oracle_context` tokens, this section divides `oracle_context` tokens into three categories: `TypeReference`, `APIInvocation`, and `VarReference`. Tokens belonging to these three categories usually constitute the most important part of the target function. As shown in Figure 3.4, on the Python version of `CoderEval`, the effectiveness of all models in correctly generating "VarReference" tokens is very poor, while the effectiveness of correctly generating "TypeReference" tokens is the best. However, on the Java version of `CoderEval`, the effectiveness of all models in correctly generating "TypeReference" tokens is very poor, while the effectiveness of correctly generating "APIInvocation" tokens is the best. The experimental results show that the ability of LLMs to generate different types of `oracle_context` tokens varies for different programming languages. Generating different types of tokens for different programming languages is an interesting future research direction.

In summary, as shown in Table 3.4 and Figure 3.4, the ability of large language models to generate `oracle_context` tokens (reflected by `Acc@K`) is consistent with the ability to generate correct code (reflected by `Pass@K`). As shown in Figure 3.4, the ability of large language models to generate different categories of `oracle_context` tokens (i.e., `TypeReference`, `APIInvocation`, and `VariableReference`) varies across programming languages.

(3) The impact of different prompts in `CoderEval` on the effectiveness of code generated by large language models

This section evaluated the three models using the original docstring and manually labeled docstring, and the experimental results are shown in Table 3.5 and Table 3.6. Table 3.5 shows the similarity between the original docstring and the manually labeled docstring. From Table 3.5, this section finds that the BLEU4 [83] value and Jaccard similarity between the original docstring and the

Table 3.5: Similarity between original docstring and manually labeled docstring in CoderEval

Metrics Language	BLEU-4	Jaccard Similarity
Python	54.7	55.2
Java	18.1	26.9

Table 3.6: Effectiveness of two prompt versions of different models on CoderEval

Model	Prompt	Python			Java		
		Pass@1	Pass@5	Pass@10	Pass@1	Pass@5	Pass@10
CodeGen	Original	9.48%	19.58%	23.48%	13.87%	27.12%	33.04%
	Human Label	12.26%	22.49%	25.65%	10.65%	21.36%	26.52%
PanGu-Coder	Original	11.83%	20.93%	27.39%	25.43%	37.39%	43.04%
	Human Label	13.74%	21.14%	24.78%	26.70%	40.33%	46.09%
ChatGPT	Original	21.13%	27.31%	30.00%	35.39%	42.77%	46.09%
	Human Label	26.61%	31.31%	32.61%	26.96%	34.85%	37.39%

manually labeled docstring of functions in the Python version of CoderEval are higher than those of functions in the Java version of CoderEval. Table 3.6 shows that for all models, the Pass@K values of the code generated using the original docstring and the manually labeled docstring of the three models are more similar on the Python version of CoderEval than on the Java version of CoderEval. On the Java version of CoderEval, the effectiveness of CodeGen and ChatGPT on the original docstring is higher than on the manually labeled docstring, while PanGu-Coder’s effectiveness on the original docstring is compared with its effectiveness on the manually labeled docstring, the result is exactly the opposite. There are two main reasons for the above results. First, as shown in Table 3.5, the similarity between the original docstring and the manually labeled docstring of Python in CoderEval is higher than that between the original docstring and the manually labeled docstring of Java. Second, comparing the generated code

with the two docstring versions, this section finds that the effectiveness of the model is related to the proportion of the target language corpus in the entire training data. For Python, the effectiveness of the model can be attributed to the specialized training or fine-tuning on the Python code corpus, while for Java, only PanGu-Coder is pre-trained on the Java code corpus. Therefore, although multilingual pre-training can transfer knowledge and bring better cross-language generalization [67], monolingual pre-training or fine-tuning is still a necessary condition for improving model effectiveness.

In summary, this section finds that choosing whether to use manually labeled docstrings or original docstrings in the problems affects the effectiveness of code generation. In monolingual code generation tasks, when given docstrings with the same semantics but different expressions, models trained with a single-language corpus perform better than models trained with a multilingual corpus.

3.4 Summary of This Chapter

This chapter proposes a new benchmark called CoderEval to evaluate the effectiveness of models in generating code for real development scenarios. Compared with the widely used HumanEval benchmark, CoderEval includes carefully selected problems from various open-source projects, to evaluate the effectiveness of models in code generation under real development scenarios. Experimental results show that CoderEval can reveal the strengths and weaknesses of three code large language models, highlighting the limitations of these existing code generation models in generating non-self-contained functions.

Chapter 4

Unit Test Case Assertion Generation Based on Information Retrieval and Deep Learning

This chapter first introduces the background of unit test case assertion generation, then introduces the large-language-model-based unit test case assertion generation method and experimental results, then introduces the unit test case assertion generation method based on the combination of information retrieval and deep learning and its experimental results, then introduces the comparison of the effectiveness of large-language-model-based and combined information-retrieval-and-deep-learning-based methods on unit test case assertion generation, and finally summarizes this chapter.

4.1 Introduction

Unit testing verifies the correctness of the software system under test by verifying the correctness of basic units. A basic unit refers to a small, functionally discrete

component in a software system (e.g., a method or a class). Compared with other levels of testing (such as integration testing and system testing), unit testing can help detect and diagnose faults faster, especially for complex software systems [24]. A typical unit test includes test inputs and unit test case assertions, with the latter serving as a specification of the desired behavior on the given test inputs. To reduce the manual work of writing unit tests, a large number of tools have been proposed to automatically generate test inputs, which fall into three main categories. (1) Random testing, such as Randoop [81], performs feedback-directed random testing by analyzing collected execution traces. (2) Dynamic symbolic execution, such as JBSE [11], dynamically performs symbolic execution using techniques designed to handle programs with complex heap input operations. (3) Search-based testing, such as EvoSuite [38], applies genetic algorithms to generate and optimize test inputs to meet coverage criteria.

In addition to generating test inputs, the above tools can also automatically generate unit test case assertions (assertions generated by the above tools can only detect crash errors or regression errors, and cannot detect non-crash errors in the current version without the previous version), mainly divided into two categories. (1) Capture and assert [119]. For example, Randoop [81] and EvoSuite [38] create assertions based on capturing and asserting the return values of all non-void return methods in the method sequences of the generated test inputs; EvoSuite further reduces these assertions based on mutation testing [57, 125]. (2) Differential testing [35]. For example, DiffGen [99] generates assertions when running on two different versions of a class by checking the equality/equivalence of method call return values and receiver object states of the two versions. To address the limitations of traditional assertion generation methods, based on other already-written unit tests, ATLAS [113] is a deep-learning (DL)-based method that generates assertions for test methods without any assertions (i.e., only test

inputs) and their methods under test. Specifically, the input of ATLAS consists of a pair of a test method without assertions and its method under test, both of which include their respective method names and method implementations. In the rest of this chapter, this chapter refers to such a pair as the input of unit test case assertion generation. The output of ATLAS is the assertion generated by the model (i.e., similar to assertions written by developers). Compared with traditional methods, ATLAS has two main advantages in the assertion generation task. First, ATLAS can generate meaningful assertions for detecting non-crash errors in the current version, while traditional methods usually only help detect crash/regression errors and generate meaningless assertions. For example, in real development scenarios, the evaluation of assertions generated by EvoSuite [1] shows that "in manually written tests, manually written assertions differ from assertions generated by traditional unit test case generation, and manually written assertions are meaningful and useful". Therefore, when developers write unit tests in the IDE, it is more ideal to examine the assertions generated by ATLAS [95]. Second, ATLAS can generate assertions for the code under test directly in the form of source code (even if the source code is not compilable), while traditional methods require the code under test to be compilable and executable. In other words, ATLAS can even be applied during ongoing test code development (e.g., during test-driven development [2, 5]). However, the effectiveness of ATLAS is limited by two main limitations inherent in deep learning methods. First, due to the interpretability of DL, assertions generated by ATLAS are not interpretable [43, 110]. Because the recommended assertion may not be the actual assertion applicable to the developer, the developer needs to check the recommended assertion. In this inspection process, providing an explanation of why an assertion is recommended is valuable for helping developers. Second, methods based on sequence-to-sequence deep learning models suffer from exposure bias [77]

and gradient vanishing [58, 62], resulting in poor effectiveness in generating long token sequences as assertions. In particular, this chapter calculated the accuracy of ATLAS in generating assertions of different lengths, and found that ATLAS performs well (46.3% accuracy) when generating short assertions (less than 15 tokens), but performs much worse (only 17.9% accuracy) when generating long assertions (more than 15 tokens).

This chapter first introduces large-language-model-based unit test case assertion generation. Then it introduces the unit test case assertion generation method based on the combination of information retrieval and deep learning. To address the limitations of deep-learning-based unit test case assertion generation methods, this chapter proposes an information retrieval (IR)-based method, including an information-retrieval-based assertion retrieval technique and a retrieved assertion adaptation technique. The input and output of the IR-based assertion retrieval technique are the same as those of ATLAS. This technique first retrieves the assertion whose corresponding test function (excluding assertions) and function under test are most similar to the given test function (excluding assertions) and function under test based on the Jaccard similarity coefficient. To improve the quality of retrieved assertions, this chapter proposes an adaptation technique that modifies tokens in the retrieved assertion according to the context. Since the test function corresponding to the assertion is also obtained when using IR to retrieve assertions, developers can obtain valuable reference when examining the assertion. In addition, since IR is based on similarity, our method can more easily generate assertions with longer token lengths. In addition to the IR-based method, this chapter also proposes an integration method that integrates the IR-based method with any DL-based method (e.g., ATLAS) to achieve a more powerful assertion generation method. In particular, our integration method carefully examines the compatibility between the retrieved assertion and the current test function: if the

compatibility is above a threshold (determined by the validation set), we directly return the retrieved assertion as the final result; otherwise, we further apply the DL-based method to generate the assertion.

Finally, this chapter discusses the advantages of unit test case assertion generation methods based on the combination of information retrieval and deep learning over large-language-model-based unit test case assertion generation methods.

4.2 Large-Language-Model-Based Unit Test Case Assertion Generation

4.2.1 Experimental Model

This section directly uses ChatGPT to generate unit test case assertions. The input part is the focal-test part of Listing 4.1, and the output part is the "Assertion to generate" part of Listing 4.1.

4.2.2 Experimental Setup

This section uses GPT-3.5-Turbo as the kernel model of ChatGPT to generate one assertion for each focal-test. Temperature and top-k are set to 0.8 and 0.95, respectively.

4.2.3 Evaluation Metrics

This section uses accuracy to evaluate the effectiveness of unit test case assertion generation. The criterion for this section to consider the generated assertion to be accurate is that the assertion is completely equal at the string level. String-level similarity may not fully reflect the effectiveness of unit test case assertion

Table 4.1: Distribution of assertion categories and numbers in two datasets

AssertType	Total	Equals	True	That	NotNull	False	Null	ArrayEquals	Same	other
Dataold	15,676	7,866(50%)	2,783(18%)	1,441(9%)	1,162(7%)	1,006(6%)	798(5%)	307(2%)	311(2%)	2(0%)
Datanew	26,542	12,557(47%)	3,652(14%)	3,532(13%)	1,284(5%)	1071(4%)	735(3%)	362(1%)	319(1%)	3,030(11%)

generation. The best way is to evaluate the bug-finding ability of the generated assertions. However, since the dataset used in this section is distributed across more than 9,000 projects, this section cannot evaluate the effectiveness of unit test case assertion generation using the bug-finding metric.

4.2.4 Experimental Dataset

Table 4.1 shows in detail the distribution of assertion categories and numbers in the two datasets used in this section.

This section refers to the dataset used by ATLAS as *Dataset_{old}*, which was mined from open-source projects on GitHub. *Dataset_{old}* removes all data that have tokens existing in all assertions but not appearing in the focal-test method, because ATLAS cannot successfully generate assertions for these data. Finally, *Dataset_{old}* contains 156,760 entries, and is divided into training, validation, and test sets with an 8:1:1 ratio.

In fact, the *Dataset_{old}* dataset used by ATLAS has removed challenging data, which simplifies the evaluation of the assertion generation task. Therefore, this section further constructs a more challenging dataset by adding the data removed by ATLAS back to the dataset. This section denotes this dataset as *Dataset_{new}*. Finally, *Dataset_{new}* contains 265,420 entries, and is divided into training, validation, and test sets with an 8:1:1 ratio.

4.2.5 Experimental Results

Listing 4.1: A TAP example in the test set

```

1 //focal-test:
2 testReportErrorOnWrongDateEffective(){
3     java.lang.String drl="rule X date-effective \"9-asbrdfh-1974\" when
        \n" + (("s: String()"+"then\n") + "end\n");
4     org.kie.internal.builder.KnowledgeBuilder kb = org.kie.internal.builder.
        KnowledgeBuilderFactory.newKnowledgeBuilder();
5     kb.add(new org.drools.core.io.impl.ByteArrayResource(drl.getBytes()),
        ResourceType.DRL);
6 }
7 hasErrors(){
8     return (errors) != null;
9 }
10 //Assertion to generate:
11 org.junit.Assert.assertTrue(kb.hasErrors())

```

Table 4.2 shows the accuracy of ChatGPT in generating unit test case assertions. Overall, on the dataset of real development scenarios, ChatGPT can generate more than half of correct unit test case assertions. ChatGPT’s effectiveness on the assertion generation task is significantly higher than that of existing deep-learning-based methods.

Table 4.2: Accuracy of ChatGPT method on each type of assertion

AssertType Approach	Total	Equals	True	That	NotNull	False	Null	ArrayEquals	Same	Other
ChatGPT-Dataold	8,058(51%)	4,061(52%)	1,521(55%)	581(40%)	762(66%)	396(39%)	411(52%)	159(52%)	165(53%)	2(100%)
ChatGPT-Datanew	12,670(48%)	5,711(45%)	1,942(53%)	1,217(34%)	801(62%)	472(44%)	356(48%)	193(53%)	177(55%)	1,801(59%)

4.2.6 Limitations of Directly Generating Unit Test Case Assertions Based on Large Language Models

Currently, large language models' capability in generating functional code is already relatively effective, but their capability in generating unit test assertions remains questionable. Models can learn token-level correspondences through functional code and their corresponding natural language descriptions, while the generation of unit test assertions differs from functional code generation. The generation of unit test assertions requires more learning and understanding of the structural information of the code under test, so as to generate good object sequences and inputs to satisfy branch coverage. However, existing large language models are all based on sequences and have insufficient ability to learn structural information. Therefore, on the assertion generation task, even using models with very large parameter scales, the effectiveness may not necessarily be better than that of non-large-language-model methods. On the assertion generation task, how to apply large language models in scenarios that cannot be solved by non-large-language-model methods will be a meaningful research direction.

4.3 Unit Test Case Assertion Generation Based on the Combination of Information Retrieval and Deep Learning

This section first proposes an assertion generation method based on similarity metrics (such as Jaccard, Dice, and Overlap) information retrieval. Then, through observation, this section finds that part of the incorrect assertions generated by the information-retrieval-based test case generation method are often caused by only a few inconsistent tokens. Next, this section proposes a heuristic-search-

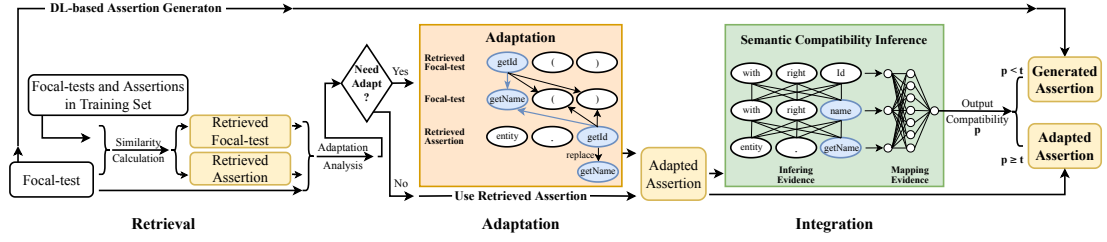


Figure 4.1: Assertion generation method based on information retrieval and its combination with deep learning

based adaptation technique. Then another adaptation technique based on neural networks (neural-network-based adaptation model) is proposed, which enhances the similarity between tokens through semantic information and computes the replacement value for code adaptation through the neural network structure. Next, this chapter finds that the complementarity of correct assertions generated by deep-learning-based methods and information-retrieval-based methods is relatively high, and this section proposes a combination method based on a fusion network. Specifically, ATLAS is a deep-learning (DL)-based method proposed to generate assertions for unit tests based on other already-written unit tests. Although ATLAS has certain effectiveness on the assertion generation task, its effectiveness is limited. To improve the effectiveness of the assertion generation task, this section first attempts to use information retrieval (IR) to generate assertions, and proposes an information-retrieval-based assertion method, including an information-retrieval-based assertion retrieval technique and a retrieved assertion adaptation technique. In addition, this section also proposes an integration method that combines information-retrieval-based methods with DL-based methods (such as ATLAS) to further improve effectiveness. The heuristic-based strategy is to replace based on token similarity. Experimental results show that our IR-based method outperforms the state-of-the-art deep-learning-based methods, and combining our IR-based method with deep-learning-based methods can

Algorithm 1: Information-retrieval-based assertion generation method
 IR_{ar}

Input : $Test_f$: the focal-test to generate assertion for.
Input : $Train_F$: all focal-tests in the training set.
Input : $Train_A$: all corresponding assertions in the training set.
Output: $Retrieved_f$: the focal-test retrieved by IR_{ar} .
Output: $Retrieved_a$: the assertion retrieved by IR_{ar} .

```

1  $max \leftarrow 0$ ;
2  $Retrieved_f \leftarrow ""$ ;
3  $Retrieved_a \leftarrow ""$ ;
4 for  $i = 1$  to  $len(Train_F)$  do
5    $jaccard \leftarrow compute(Test_f, Train_F[i])$ ;
6   if  $jaccard > max$  then
7      $max \leftarrow jaccard$ ;
8      $Retrieved_f \leftarrow Train_F[i]$ ;
9      $Retrieved_a \leftarrow Train_A[i]$ ;
10 return  $Retrieved_f, Retrieved_a$ ;
```

achieve better results on the unit test case assertion generation task.

4.3.1 Assertion Generation Method Based on IR

The basic idea of IR_{ar} is to retrieve the assertion whose corresponding method under test has the highest similarity to the given method under test. More specifically, IR_{ar} first tokenizes each method under test in the training set and test set using javalang [100], and removes duplicate tokens from each method under test to achieve more effective retrieval; then, given a method under test in the test set, IR_{ar} calculates the Jaccard similarity coefficient with all methods under test in the training set, and retrieves the one with the highest similarity coefficient; finally, it returns the assertion corresponding to the retrieved method under test as the expected assertion. When there is a tie in the similarity computation, IR_{ar} selects the first one that appears in the training set. As the input of the algorithm,

$Test_f$ is the input method under test, whose assertion will be generated by IR_{ar} ; $Train_F$ and $Train_A$ are lists of all methods under test and their assertions in the training set. IR_{ar} computes the Jaccard similarity between the input method under test and each method under test in the training set, selects the one with the highest similarity, and returns its assertion as the output. For example, given an input method under test to generate assertions, IR_{ar} compares it with each method under test in the training set, finds the most similar one, and returns its corresponding assertion. Algorithm 1 shows the detailed workflow of IR_{ar} . As the input of the algorithm, $Test_f$ is the input method under test and the test case without assertion, whose assertion will be generated by IR_{ar} ; $Train_F$ and $Train_A$ are lists of all methods under test and test cases without assertion and their corresponding assertions in the training set. IR_{ar} computes the Jaccard similarity between the input focal-test (i.e., method under test and test case without assertion) and each focal-test in the training set (line 3), selects the focal-test with the highest similarity (lines 4-7), and returns its assertion as the output (line 8).

Listing 4.2: A TAP example in the training set

```

1 //focal-test:
2 testFailedStaticImport (){
3     java.lang.String drl="package org.drools.test; \n" + (((((( "\"" + "
        import function org.does.not.exist.Foo; \n") + "" ) + "" ) + "
        rule X when\n") + "then\n" ) + "end");
4     org.kie.internal.builder.KnowledgeBuilder kb = org.kie.internal.builder .
        KnowledgeBuilderFactory.newKnowledgeBuilder();
5     kb.add(new org.drools.core.io.impl.ByteArrayResource(drl.getBytes()),
        ResourceType.DRL) ;
6 }
```

```

7  hasErrors () {
8      return ( errors ) != null ;
9  }
10 //Assertion:
11 org . junit . Assert . assertTrue ( kb . hasErrors () )

```

For example, to generate an assertion for the method under test and the test case without assertion in Listing 4.1, IR_{ar} compares it with each method under test and test case without assertion in the training set, finds the most similar one (as shown in Listing 4.2), and returns its corresponding assertion. IR_{ar} is based on an intuition that similar methods under test and test cases without assertion may have the same assertion, which is why IR_{ar} is applicable to the assertion generation task. However, it is also very common for two similar methods under test and test cases without assertion (but not identical) to have not completely identical corresponding assertions. For these cases, directly returning the retrieved assertion can only generate similar assertions, but cannot generate accurate assertions. To solve this problem, in the next section, this chapter will propose an automatic adaptation technique to modify the retrieved assertion.

4.3.2 Adaptation Model Based on Heuristic Search and Neural Networks

Listing 4.3: A false TAP example in the test set

```

1  //focal-test:
2  should_build_an_entity_with_the_right_name(){
3      builder .setName("name") ;
4      org . bonitasoft . engine . identity . model . SCustomUserInfoDefinition  entity =
        builder .done();

```

```

5  }
6  getName(){
7      return name;
8  }
9  //Assertion:
10 org.junit.Assert.assertEquals("name", entity.getName())

```

Although IR_{ar} cannot always retrieve completely correct assertions from the training set, it can return "almost correct" assertions that are very similar to correct assertions. Therefore, this section further proposes an automated adaptation technique RA_{adapt} , which modifies retrieved assertions into correct assertions based on contextual information. For the retrieved assertions, RA_{adapt} performs the following adaptation process.

- Step 1: Decide whether the retrieved assertion should be modified. Since not all retrieved assertions need to be adjusted, pre-identification should be done first to avoid incorrectly modifying correct assertions. If the assertion contains at least one token that does not appear in the input method under test and the test case excluding assertions, RA_{adapt} considers the assertion to need adaptation.
- Step 2: Decide which token (i.e., called method, variable, or constant) should be modified. RA_{adapt} considers tokens in the assertion that do not exist in the input method under test and the test case excluding assertions as candidate tokens for modification.
- Step 3: Decide what token the candidate token should be replaced with. RA_{adapt} determines the replacement value by analyzing the token correspondence between the methods under test and the test cases excluding

assertions of the test set and the training set. This section observes that most retrieved assertions have the same syntax structure as correct assertions, but with a few inconsistent special tokens (e.g., called method names, variable names, or constant values). Therefore, the current RA_{adapt} mainly considers replacement operations in the adaptation process. This section believes that in future work, RA_{adapt} can be further extended to add or remove more editing operations.

Algorithm 2 shows the details of RA_{adapt} . In lines 1-3, RA_{adapt} first extracts three categories of tokens (all called methods, variables, and constants) defined in t , t_{ir} , and a_{ir} (inputs to RA_{adapt}), storing these tokens in nine sets. Processing the categories of called methods, variables, and constants separately can make RA_{adapt} more effective, because there is no replacement relationship between these three categories, and grouping replacements by token type can narrow the replacement scope and thus improve replacement accuracy. For the example code snippets in Listing 4.3 and Listing 4.4, RA_{adapt} extracts nine sets of tokens: IM_t is [setName, getName, done], VAR_t is [builder, entity, name], CST_t is [name]; $IM_{t_{ir}}$ is [setId, getId, done], $VAR_{t_{ir}}$ is [builder, entity, id], $CST_{t_{ir}}$ is [1L]; $IM_{a_{ir}}$ is [getId], $VAR_{a_{ir}}$ is [entity], $CST_{a_{ir}}$ is [1L].

- Step 1: In lines 4-8 and lines 17-23, RA_{adapt} decides whether to adapt the assertion retrieved by IR_{ar} (in the training set). The criterion for requiring adaptation replacement is that at least one called method invoked-method does not appear in IM_t but appears in $IM_{a_{ir}}$; or at least one variable $VAR_{a_{ir}}$ does not appear in VAR_t but appears in $VAR_{t_{ir}}$; or at least one constant $CST_{a_{ir}}$ does not appear in CST_t but appears in $CST_{t_{ir}}$. For the example code snippets in Listing 4.3 and Listing 4.4, "getId" in $IM_{a_{ir}}$ does not appear in IM_t , and "1L" in $CST_{a_{ir}}$ does not appear in CST_t , so this section needs to perform replacement analysis for "1L" and "getId".

Algorithm 2: Retrieved-Assertion Adaptation RA_{adapt}

Input : t : the focal-test to adapt assertion for.
Input : t_{ir} : the focal-test retrieved by IR_{ar} .
Input : a_{ir} : the assertion retrieved by IR_{ar} .
Output: a'_{ir} : the assertion adapted by RA_{adapt} .

```

1 Function needAdapt ( $TK_t, TK_{t_{ir}}, TK_{a_{ir}}$ ) :
2    $needAdaptation \leftarrow False$ ;
3   for  $tk_{a_{ir}} \in TK_{a_{ir}}$  do
4     if  $tk_{a_{ir}} \notin TK_t$  and  $tk_{a_{ir}} \in TK_{t_{ir}}$  then
5        $needAdaptation \leftarrow True$ ;
6       break;
7   return  $needAdaptation$ ;

8 Function getReplaceMap ( $TK_t, TK_{t_{ir}}, TK_{a_{ir}}$ ) :
9    $replaceMap \leftarrow dict()$ ;
10  for  $tk_{t_{ir}} \in (TK_{t_{ir}} \cap TK_{a_{ir}})$  do
11     $replace \leftarrow ""$ ;
12     $max \leftarrow 0$ ;
13    for  $tk_t \in TK_t$  do
14       $temp \leftarrow computeSimilarity(tk_{t_{ir}}, tk_t)$ ;
15      if  $temp > max$  then
16         $max \leftarrow temp$ ;
17         $replace \leftarrow tk_t$ ;
18     $replaceMap[tk_{t_{ir}}] \leftarrow replace$ ;
19  return  $replaceMap$ ;

20  $IM_t, VAR_t, CST_t \leftarrow extractTokens(t)$ ;
21  $IM_{t_{ir}}, VAR_{t_{ir}}, CST_{t_{ir}} \leftarrow extractTokens(t_{ir})$ ;
22  $IM_{a_{ir}}, VAR_{a_{ir}}, CST_{a_{ir}} \leftarrow extractTokens(a_{ir})$ ;
23  $adaptIM \leftarrow needAdapt(IM_t, IM_{t_{ir}}, IM_{a_{ir}})$ ;
24  $adaptVAR \leftarrow needAdapt(VAR_t, VAR_{t_{ir}}, VAR_{a_{ir}})$ ;
25  $adaptCST \leftarrow needAdapt(CST_t, CST_{t_{ir}}, CST_{a_{ir}})$ ;
26 if  $!adaptIM$  and  $!adaptVAR$  and  $!adaptCST$  then
27   return  $a_{ir}$ ;

28  $IM'_{t_{ir}}, IM'_t \leftarrow trim(IM_{t_{ir}}, IM_t)$ ;
29  $VAR'_{t_{ir}}, VAR'_t \leftarrow trim(VAR_{t_{ir}}, VAR_t)$ ;
30  $CST'_{t_{ir}}, CST'_t \leftarrow trim(CST_{t_{ir}}, CST_t)$ ;
31  $replaceIM \leftarrow getReplaceMap(IM'_t, IM'_{t_{ir}}, IM_{a_{ir}})$ ;
32  $replaceVAR \leftarrow getReplaceMap(VAR'_t, VAR'_{t_{ir}}, VAR_{a_{ir}})$ ;
33  $replaceCST \leftarrow getReplaceMap(CST'_t, CST'_{t_{ir}}, CST_{a_{ir}})$ ;
34  $a'_{ir} \leftarrow replace(a_{ir}, replaceIM, replaceVAR, replaceCST)$ ;
35 return  $a'_{ir}$ ;
  
```

- Step 2: Line 9 of the algorithm removes the overlapping elements between $IM_{t_{ir}}$ and IM_t from these two sets, resulting in $IM'_{t_{ir}}$ and IM'_t , respectively. The elements in $IM'_{t_{ir}}$ are tokens (i.e., called methods here) that should be modified. Lines 10 and 11 of the algorithm use the same method as line 9 to deduplicate the token elements in $VAR_{t_{ir}}$ and VAR_t , and $CST_{t_{ir}}$ and CST_t , respectively. For the example code snippets, after removing duplicated token elements, $IM'_{t_{ir}}$ is [setId, getId], IM'_t is [setName, getName], $VAR_{t_{ir}}$ is [id], VAR_t is [name], $CST_{t_{ir}}$ is [1L], CST_t is ["name"].
- Step 3: In lines 12-14 of the algorithm, RA_{adapt} selects all replacement values for called methods, variables, and constants; the strategies for selecting replacement values are the heuristic strategy and the deep-neural-network-based adaptation strategy discussed below, respectively. In lines 15 and 16 of the algorithm, the algorithm replaces the called methods, variables, and constants to be replaced in a_{ir} , and returns the adapted modified assertion a'_{ir} .

For Step 3, this section designs two replacement strategies to determine replacement values: one is the heuristic-based approach (denoted as RA_{adapt}^H) and the other is the neural-network-based approach (denoted as RA_{adapt}^{NN}).

Heuristic method RA_{adapt}^H . RA_{adapt}^H computes sub-tokens based on camel case encoding through the functions *getReplaceMap* and *computeSimilarity*. Specifically, RA_{adapt}^H computes the similar positions of $tk_{t_{ir}}$ in $TK_{t_{ir}}$ and tk_t in TK_t . This chapter first determines whether $tk_{t_{ir}}$ and tk_t contain the same sub-tokens generated by camel case splitting. If $tk_{t_{ir}}$ and tk_t contain the same sub-tokens after camel case encoding, this chapter takes tk_t as a candidate. Then, it selects the replacement value based on the positions of $tk_{t_{ir}}$ in $TK_{t_{ir}}$ and tk_t in TK_t . Here, this section selects the token tk_t closest to $tk_{t_{ir}}$. For example, "getId" in Listing 4.4 needs to be compared with "setName" and "getName" in

Listing 4.3, respectively. The camel case split of "getId" is "get" and "Id", the camel case split of "getName" is "get" and "Name", and the camel case split of "setName" is "set" and "Name". The position of "getId" in t_{ir} is 34, the position of "getName" in t is 34, and the position of "setName" in t is 7. Therefore, "getId" is replaced with "getName". The constant 1L in Listing 4.4 needs to be replaced, and in $CST'_{t_{ir}}$, there is only one element "name", so "1L" is directly replaced with "name".

Deep-neural-network-based adaptation method RA_{adapt}^{NN} . The heuristic-based strategy RA_{adapt}^H replaces based on token similarity and position, which may ignore the semantic information embedded in the program. Then, this section proposes another neural-network-based replacement strategy RA_{adapt}^{NN} , which enhances token similarity through semantic information and computes replacement values through a neural network architecture for code adaptation. RA_{adapt}^{NN} consists of three parts: (1) embedding first-order semantic information; (2) enriching first-order information with high-order semantic information; (3) combining semantic information with the above token similarity.

Listing 4.4: A false TAP example in the training set

```

1 //focal-test:
2 should_build_an_entity_with_the_right_id() {
3     builder . setId (1L) ;
4     org . bonitasoft . engine . identity . model . SCustomUserInfoDefinition entity =
5         builder . done();
6 }
7 getId () {
8     return id ;
9 }
10 //Assertion:

```

10 `org . junit . Assert . assertEquals (1L, entity . getld ())`

In the first component, RA_{adapt}^{NN} uses Word2Vec [90] to capture first-order information, which maps each word to an n -dimensional vector space $\mathbb{R}$. This section uses an $m \times n$ matrix M for this semantic mapping, where m represents the size of the vocabulary (vocabulary usually refers to the set of unique tokens used in a text corpus). The second component captures high-order information by injecting contextual semantics into the current token. In particular, this section adopts a bidirectional recurrent neural network architecture (Bi-RNA), which passes information both forward and backward. At each step, Bi-RNA updates the representation of the current token I_i with its forward and backward context, enriching the initial first-order information of I_i with high-order information. Where R is the recurrent algorithm used to update information, and I_i^R is the semantically enhanced output of I_i .

$$I_i^R = [R_>(I_i, I_{i-1...0}), R_<(I_i, I_{i+1...l})]$$

Finally, this section uses a linear layer to combine semantic similarity with token similarity. Semantic similarity is computed using cosine coefficients; the higher the similarity, the larger the cosine coefficient. In line 30 of Algorithm 2, this section uses dot product to compute the cosine coefficient between tokens in $I_i^{Rt_{ir}}$ used to replace $tk_{t_{ir}}$ and I_i^{Rt} in its candidate tk_t (in the function *getReplaceMap*), as follows. Where S is the replacement score of $tk_{t_{ir}}$ and tk_t . $p1$ and $p2$ are parameters for training RA_{adapt}^{NN} .

$$S = \begin{cases} p1 \cdot I_i^{Rt_{ir}} \cdot I_i^{Rt} + p2 & I_i^{Rt_{ir}} \text{ and } I_i^{Rt} \text{ contain same sub-token(s)} \\ p1 \cdot I_i^{Rt_{ir}} \cdot I_i^{Rt} & \text{otherwise} \end{cases} \quad (4.1)$$

RA_{adapt}^{NN} optimizes by maximizing the negative log-likelihood over the replacement likelihood: S .

IR_{ar} is based on the following intuition: similar methods under test may have the same assertion. This is the reason why IR_{ar} works well in this example. However, it is also very common for two similar methods under test to share similar (but not identical) assertions. For these cases, directly returning the retrieved assertion cannot generate precise assertions. To solve this problem, this section proposes a neural-network-based adaptation model. The neural-network-based adaptation model consists of three parts: (1) embedding first-order semantic information, (2) enriching first-order information with high-order semantic information, and (3) combining semantic information with the above token similarity. In the first component, the neural-network-based adaptation model uses Word2Vec [90] to capture first-order information, which maps each word to an n -dimensional vector space R . This section uses an $m \times n$ matrix M for semantic mapping, where m represents the vocabulary size (vocabulary usually refers to the set of unique tokens used in a text corpus). The second component captures high-order information by injecting contextual semantics into the current token. In particular, this section adopts a bidirectional recurrent neural network architecture (Bi-RNA), which passes information both forward and backward. At each step, Bi-RNA updates the current token. The representation of I_i and its forward and backward context, enriches the initial first-order information of I_i with high-order information. ($I_i^R = [R > (I_i, I_i - 1...0), R < (I_i, I_i + 1...l)]$) Finally, this chapter uses a linear layer to combine semantic similarity with token similarity. Semantic similarity is calculated using cosine coefficients; the cosine coefficient between more similar tokens is larger. In line 30 of Algorithm 2, this chapter uses dot product to compute the cosine coefficient between tokens in $I_i^{(Rt_i r)}$ used to replace $tk_{(t_i r)}$ and $I_i^R t$ in its candidate tk_t (in the function `getReplaceMap`), as

follows. Here S represents the replacement score for $tk(t_i r)$ and tk_t . $p1$ and $p2$ are parameters trained based on RA_{adapt}^{NN} .

4.3.3 Combination of IR and DL Methods Based on Fusion Neural Networks

IR-based methods retrieve assertions from the training set, especially when similar cases exist in the training set, which can be very effective. On the other hand, DL-based methods learn to generate assertions based on the training set and can generate "new" assertions that do not exist in the training set. Intuitively, these two methods are complementary and can be further combined to achieve more powerful assertion generation. This section proposes a combination method based on a compatibility reasoning model (Integration), which computes the "compatibility" between the retrieved assertion and the current focal-test to determine whether to directly return the retrieved statement (with modification) or to apply a DL-based method to generate a new assertion. Figure 4.1 shows the workflow of our combination method. The first two steps are our IR-based method (including IR_{ar} and RA_{adapt} techniques), which retrieves assertions based on Jaccard similarity and adapts the retrieved assertion when necessary. After adaptation, this section uses a semantic compatibility reasoning model to compute the compatibility of the adapted assertion. If the compatibility is below a specified threshold (represented as t in Figure 4.1), this section turns to DL-based methods (such as ATLAS) to generate "new" assertions. In our method, t is determined based on the validation set. Next, this section will detail our compatibility reasoning model. The adaptation technique handles syntactic compatibility by replacing incorrect tokens. This chapter further utilizes semantic compatibility to find incompatible assertions. This problem is similar to the task of Neural Language Inference (NLI), which is usually solved with a binary neural reasoning model.

Usually, binary neural reasoning models infer evidence between a hypothesis and a premise to infer their relationship (i.e., entailment or contradiction). However, our problem is different, and the relationship is "ternary" rather than "binary". Since the assertion is retrieved from its training focal-test, the reasoning model should consider not only the compatibility between the retrieved assertion and the input focal-test, but also the compatibility between the retrieved focal-test and the input focal-test. Based on this insight, this section proposes a "ternary" neural reasoning model, which can be considered as an extension of the recently proposed binary neural reasoning model named ESIM [20]. This section uses an attention-based RNN model to model the local evidence of compatibility among the retrieved assertion, the retrieved focal-test, and the input focal-test. Then, this section combines data from three code sequences with a neural network, which maps data features to the final result.

4.3.4 Experimental Dataset

Consistent with the dataset used in the previous section, this chapter uses the DatasetOld and DatasetNew datasets from Section 4.2.4 for experiments.

4.3.5 Evaluation Metrics

The evaluation metrics used in this section are the same as ATLAS, using accuracy and BLEU value to evaluate the effectiveness of the methods proposed in this section. **Accuracy.** The criterion for this section to consider the generated assertion to be accurate is that the assertion is completely equal at the string level. **BLEU value.** BLEU value is widely used in the field of natural language processing, and it is used to evaluate the similarity of generated sentences to annotated sentences. This section uses the BLEU value to evaluate the similarity

Table 4.3: Accuracy of different methods

Approach Dataset	ATLAS	IR	RA_{adapt}^H	RA_{adapt}^{NN}	Intergration
Dataold	31.42	36.26	40.97	43.63	46.54
Datanew	21.66	37.90	39.65	40.53	42.20

Table 4.4: BLEU values of different methods

Approach Dataset	ATLAS	IR	RA_{adapt}^H	RA_{adapt}^{NN}	Intergration
Dataold	68.51	71.48	73.28	73.95	78.86
Datanew	37.91	57.98	59.81	59.81	60.92

between the assertions generated by our method and the annotated assertions.

4.3.6 Experimental Results

(1) The effectiveness of the information-retrieval-based assertion generation method IR_{ar} on the assertion generation task

Overall effectiveness. As shown in Table 4.3 and Table 4.4, in terms of accuracy and BLEU, the proposed IR_{ar} technique significantly outperforms the DL-based comparison method ATLAS on both datasets. In addition, this section observes that on the more challenging dataset (i.e., $Data_{new}$ and unknown token), this difference is much more significant. The reason may be that existing DL-based methods have weak ability to generate assertions with unknowns, and thus perform much worse on $Data_{new}$ (i.e., accuracy drops by nearly 10%).

Table 4.5: Accuracy of different methods on various types of assertions

AssertType Approach	Total	Equals	True	That	NotNull	False	Null	ArrayEquals	Same	Other
ATLAS-Dataold	4,925(31%)	2,501(32%)	966(35%)	248(17%)	598(51%)	229(23%)	236(30%)	100(33%)	47(15%)	0(0%)
IR-Dataold	5,684(36%)	2,957(38%)	1,039(37%)	449(31%)	439(38%)	314(31%)	285(36%)	111(36%)	89(29%)	1(50%)
ATLAS-Datanew	5,749(22%)	2,900(23%)	619(17%)	537(15%)	388(30%)	126(12%)	85(12%)	47(13%)	37(12%)	1,010(33%)
IR-Datanew	10,059(38%)	4,664(37%)	1,436(39%)	1,070(30%)	600(47%)	394(37%)	286(39%)	147(41%)	113(35%)	1,349(45%)

Effectiveness on different assertion types. This section further compares the effectiveness of IR_{ar} and ATLAS on different types of assertions. Each column in Table 4.5 represents an assertion type, and each cell shows the number of correctly generated assertions and the corresponding proportion in parentheses. As shown in the table, this section observes that IR_{ar} consistently outperforms ATLAS on all assertion types, indicating the universality of IR_{ar} in generating different types of assertions. The third column of Table 4.3 and Table 4.4 shows the accuracy and BLEU values of the IR_{ar} method on the two datasets. From the tables, it can be seen that the performance of IR_{ar} method on both datasets in terms of accuracy and BLEU values is significantly better than ATLAS.

Effectiveness of different similarity coefficients. IR_{ar} uses Jaccard [102] as its default similarity coefficient. This section studies the impact of different IR similarity coefficients on the effectiveness of IR_{ar} . In particular, this section implements two other variants of IR_{ar} with two widely used similarity coefficients, namely Overlap [118] and Dice [28], which compute the similarity of two given sets (i.e., X and Y) as follows.

$$Overlap(X, Y) = |X \cap Y| / \min(|X|, |Y|)$$

$$DSC(X, Y) = |X \cap Y| / (|X| + |Y|)$$

Table 4.6 shows the accuracy of IR_{ar} with different similarity coefficients on the two datasets. The results show that the similarity coefficient has little impact on the effectiveness of IR_{ar} , indicating the universality of the IR_{ar} technique in assertion generation. In summary, our results show that the proposed IR_{ar} method outperforms the existing DL-based method ATLAS, including on different assertion types and different IR similarity coefficients. To further analyze these results, this section next analyzes the correct and incorrect assertions generated

Table 4.6: Accuracy of retrieval using different similarity coefficients

Approach Dataset	Jaccard	DICE	Overlap
Dataold	36.26	36.26	36.12
Datanew	37.90	37.90	37.74

by the IR_{ar} method respectively.

Correct assertions generated by ATLAS and IR_{ar} . To further investigate the assertions generated by ATLAS and IR_{ar} , this section first analyzes the assertions successfully generated by ATLAS and IR_{ar} . To further investigate the correct assertions generated by IR_{ar} and ATLAS, this section compares the length of correct assertions (i.e., the number of tokens in each assertion). Figure 4.2 shows the distribution of the lengths of correctly generated assertions, where the X-axis represents the length of the assertion and the Y-axis represents the number of corresponding assertions. Interestingly, as shown in the figure, this section can observe that IR_{ar} tends to outperform ATLAS on longer assertions. For example, on $Data_{old}$, when the target assertion contains less than 8 tokens, ATLAS is more effective than IR_{ar} , while when the target assertion contains more than 8 tokens, ATLAS is less effective than IR_{ar} . This section also observes a similar trend on $Data_{new}$: although IR_{ar} is always more effective than ATLAS, the difference becomes more prominent as the assertion length increases. To further confirm our observation, this section computes the average length of correct assertions generated by ATLAS and IR_{ar} , respectively. On $Data_{old}$, the average lengths of correct assertions generated by ATLAS and IR_{ar} are 7.98 and 8.63, respectively; on $Data_{new}$, the average lengths of ATLAS and IR_{ar} are 9.74 and 10.74, respectively. Our experimental results show that IR_{ar} has a stronger ability to generate correct long assertions than ATLAS. A potential reason may be that it is still very challenging for existing DL-based methods to generate long

sequences from scratch, while IR-based methods retrieve long sequences from existing data is feasible. In addition, our research findings also indicate that there is complementarity between DL-based and IR-based methods, especially for assertions of different lengths, which also motivates the combination of these two method categories to generate more powerful assertions.

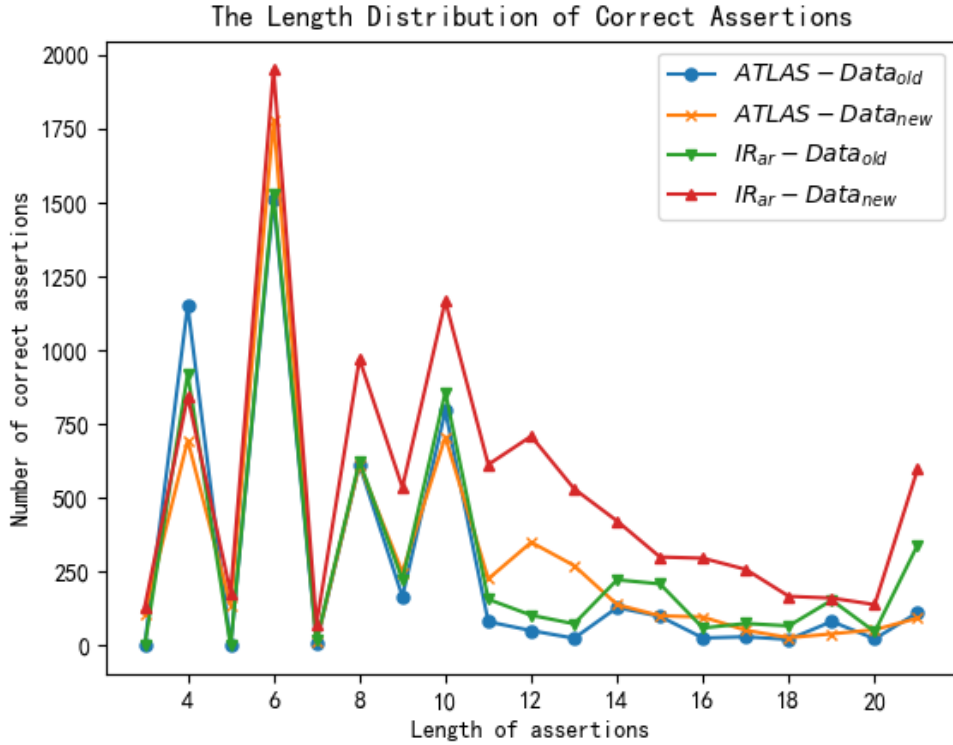


Figure 4.2: Distribution of lengths of correctly generated assertions by ATLAS and IR_{ar}

This section further studies the correct assertions generated by ATLAS. Interestingly, this section finds that most of them exist in the training set. More specifically, in $Data_{old}$, among the 4,925 correct assertions that ATLAS can generate, 4,560 (92.59%) assertions exist in the training set if the abstraction strategy is applied. The abstraction strategy in ATLAS replaces specific tokens (such as

Table 4.7: Edit distance between assertions that ATLAS and IR_{ar} cannot successfully generate and labeled assertions

Dataset \ Edit	1	2	3
ATLAS-Dataold	2,840(26%)	763(7%)	914(9%)
IR_{ar} -Dataold	2,966(30%)	1,198(12%)	572(6%)
ATLAS-Datanew	2,614(13%)	1,595(8%)	1,522(7%)
IR_{ar} -Datanew	4,532(31%)	1,912(13%)	1,091(8%)

identifier names and variable names) with abstract tokens to reduce infrequent tokens and the size of the vocabulary. Similarly, in $Data_{new}$, this section also observes that most correctly generated assertions (i.e., 98.76%) are assertions that exist in the training set. Intuitively, compared with IR-based methods, the advantage of DL-based methods lies in "intelligently" creating new assertions that have never appeared in the training set. However, our results show that most assertions generated by ATLAS exist in the training set, and only a few correct assertions are new and "created" by ATLAS. The previous observation explains why even simple IR methods can outperform DL-based methods, which also shows that existing DL-based assertion generation has limited ability in generating correct assertions.

Incorrect assertions generated by ATLAS and IR_{ar} . After studying the assertions that ATLAS and IR_{ar} can successfully generate, this section next analyzes the assertions that ATLAS and IR_{ar} cannot successfully generate. In particular, this section calculates the edit distance between incorrectly generated assertions and token assertions (i.e., groundtruth) in Table 4.7. Interestingly, although ATLAS or IR_{ar} fails to successfully generate a large number of assertions, a significant portion of these incorrectly generated assertions are very similar to correct assertions, i.e., their edit distances are quite small. For example, among the 10,751 and 20,793 incorrect assertions generated by ATLAS on $Data_{old}$ and

$Data_{new}$ respectively, 2,840 (i.e., 26.4%) and 2,641 (i.e., 12.7%) assertions differ from correct assertions by only one token, while 4,517 (i.e., 42.0%) and 5,731 (i.e., 27.6%) assertions differ from correct assertions by no more than three tokens. Similarly, among the 9,992 and 14,483 incorrect assertions retrieved by IR_{ar} on $Data_{old}$ and $Data_{new}$ respectively, 2,966 (i.e., 29.7%) and 4,532 (i.e., 31.3%) assertions differ from correct assertions by only one token, while 4,736 (i.e., 47.4%) and 7,535 (i.e., 52.0%) assertions differ from correct assertions by no more than three tokens. Inspired by the above observations, this section further classifies the incorrectly generated assertions with only one token edit distance from correct assertions according to the type of incorrect token. Table 4.7 shows the number of assertions in each token category. From the table, this section can find that compared with correct assertions, most incorrect assertions generated by ATLAS or IR_{ar} have incorrect constant values, indicating that generating constant values may be a major challenge in assertion generation. Such a challenge is not surprising, because the candidate space for constant values is usually very large, so it is difficult to correctly predict constant values. In addition, this section also finds that a large portion of generated incorrect assertions are due to incorrect assertion types. For example, on $Data_{old}$, if the assertion type generated by ATLAS (IR_{ar}) is fixed, 738 (677) incorrect assertions can be modified into correct assertions. Compared with incorrect constant values, fixing incorrect assertion types is usually simpler because the number of fixed types is limited. In summary, a large number of incorrect assertions generated by ATLAS and IR_{ar} are very similar to correct assertions. These incorrect assertions are still helpful to developers, or provide useful information for assertion generation. In addition, many incorrect assertions may become correct after modifying only one token (e.g., related to assertion types). In addition, our research findings also motivate the intuition of the adaptation technique proposed in this section.

Table 4.8: Number of different token categories when the edit distance is 1

Token Dataset	total	api	variable	constant	assertType
ATLAS-Dataold	2,840	27	865	1,171	738
IR_{ar} -Dataold	2,966	145	998	1,042	677
ATLAS-Datanew	2,614	225	482	1,051	274
IR_{ar} -Datanew	4,532	448	1,014	2,100	395

Table 4.9: Accuracy of RA_{adapt}^H and RA_{adapt}^{NN} on each assertion type

AssertType Approach	Total	Equals	True	That	NotNull	False	Null	ArrayEquals	Same	Other
RA_{adapt}^H -Dataold	6,423(41%)	3,300(42%)	1,151(41%)	536(37%)	553(48%)	335(33%)	316(40%)	120(39%)	111(36%)	1(50%)
RA_{adapt}^{NN} -Dataold	6,839(44%)	3,509(45%)	1,225(44%)	551(38%)	610(52%)	342(34%)	341(43%)	134(44%)	126(41%)	1(50%)
RA_{adapt}^H -Datanew	10,525(40%)	4,882(39%)	1,487(41%)	1,142(32%)	651(51%)	403(38%)	297(40%)	154(43%)	121(38%)	1,388(46%)
RA_{adapt}^{NN} -Datanew	10,758(41%)	4,988(40%)	1,526(42%)	1,161(33%)	691(54%)	401(37%)	308(42%)	162(45%)	126(39%)	1,395(46%)

(2) Effectiveness of the adaptation technique

Table 4.9 shows the accuracy of RA_{adapt}^H and RA_{adapt}^{NN} on different assertion types. In Table 4.9, the "Total" column shows the overall accuracy of RA_{adapt}^H and RA_{adapt}^{NN} on $Data_{old}$ and $Data_{new}$. This section finds that both adaptation techniques can significantly improve IR_{ar} , and RA_{adapt}^{NN} can achieve higher improvement effect than RA_{adapt}^H . For example, RA_{adapt}^H and RA_{adapt}^{NN} improve IR_{ar} by 4.71% and 7.37% respectively on $Data_{old}$, and by 1.75% and 2.63% respectively on $Data_{new}$. In addition, RA_{adapt}^H (RA_{adapt}^{NN}) can further significantly outperform the DL-based method ATLAS, namely, improve by 9.55% (12.21%) on $Data_{old}$ and improve by 17.99% (18.87%) on $Data_{new}$. In addition, the other columns in Table 4.9 show the effectiveness of adaptation for different types of assertions. From the table, this section can observe consistent improvement by these two automatic adaptation techniques on all assertion types. This section further analyzes the success and failure cases of our adaptation technique. In particular, this section finds that on $Data_{old}$, RA_{adapt}^H and RA_{adapt}^{NN} modify 419 and 737 incorrect retrieved assertions into correct assertions, respectively; similarly, on $Data_{new}$,

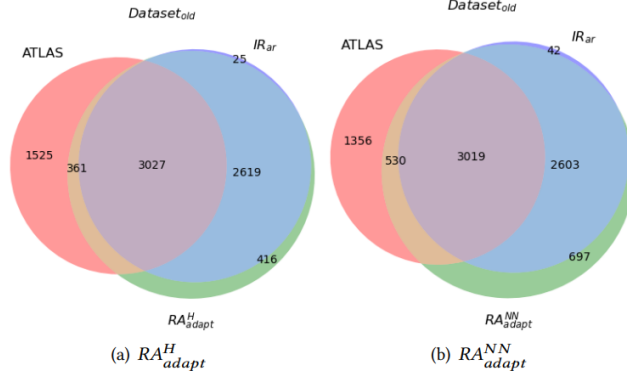


Figure 4.3: Complementarity between information-retrieval-based methods and deep-learning-based methods on the old dataset

RA_{adapt}^H and RA_{adapt}^{NN} can modify 563 and 679 incorrect retrieved assertions into correct assertions, respectively. On the other hand, our adaptation technique may modify correct assertions into incorrect ones. On $Data_{old}$, a small number of correctly retrieved assertions (i.e., 25 and 42) are incorrectly identified as incorrect assertions by RA_{adapt}^H and RA_{adapt}^{NN} respectively, and further modified into incorrect assertions. On $Data_{new}$, similarly, a small number of correctly retrieved assertions (i.e., 68 and 84) are incorrectly identified as incorrect assertions by RA_{adapt}^H and RA_{adapt}^{NN} respectively, and further modified into incorrect assertions. In summary, our adaptation technique successfully modifies a large number of incorrect assertions into correct assertions, at the cost of incorrectly modifying a small number of correct retrieved assertions, which explains the overall effectiveness of the adaptation technique.

(3) Effectiveness of the Integration technique

Figures 4.3 and 4.4 show the complementarity of information-retrieval-based methods and deep-learning-based methods on the old and new datasets respectively. From the figures, it can be seen that the two methods have strong complementarity. Therefore, this section further proposes a combination method.

Table 4.10: Accuracy of the Integration method on each type of assertion

AssertType Approach	Total	Equals	True	That	NotNull	False	Null	ArrayEquals	Same	Other
Integration-Dataold	7,295(47%)	3,714(47%)	1,333(48%)	546(38%)	724(62%)	348(35%)	352(44%)	148(48%)	129(41%)	1(50%)
Integration-Datanew	11,201(42%)	5,248(42%)	1,566(43%)	1,196(34%)	711(55%)	401(37%)	313(43%)	162(45%)	128(40%)	1,476(49%)

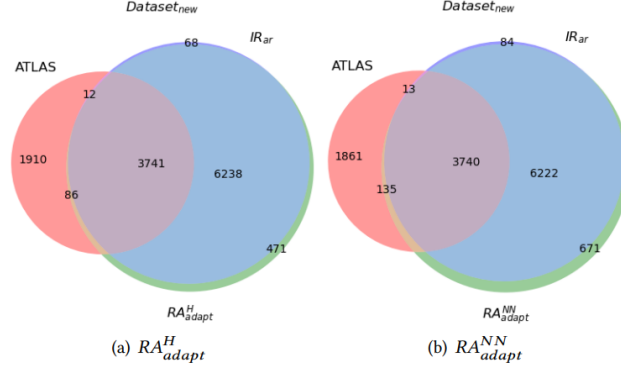


Figure 4.4: Complementarity between information-retrieval-based methods and deep-learning-based methods on the new dataset

Figures 4.3 and 4.4 show the overlap between correct assertions generated by DL-based methods (i.e., ATLAS) and IR-based methods (such as IR_{ar} and RA_{adapt}). From the figures, it can be seen that DL-based and IR-based methods are complementary. For example, on *Data_{old}*, between ATLAS and RA_{adapt}^H , 1,525 correct assertions are uniquely generated by ATLAS, while 3,035 correct assertions are uniquely generated by RA_{adapt}^H ; between ATLAS and RA_{adapt}^{NN} , 1,356 correct assertions are uniquely generated by ATLAS, while 3,300 correct assertions are uniquely generated by RA_{adapt}^{NN} . This complementarity is consistent on *Data_{new}*. The results show that fusing these two methods can achieve even more powerful unit test case assertion generation effectiveness. Table 4.10 shows the effectiveness of Integration on all assertions and each type of assertion. From the table, this section observes that Integration consistently outperforms DL-based and IR-based methods on all assertions. The results show that the fusion method proposed in this section is effective and also indicate that integrating DL-based

and IR-based methods may be a promising direction for automatic assertion generation.

4.4 Comparison Between Large-Language-Model-Based Methods and Methods Combining Information Retrieval and Deep Learning

Although the overall effectiveness of large-language-model-based methods is relatively good, they require a lot of computational resources and inference time. This paper proposes a method that combines information retrieval and deep learning for unit test case assertion generation through program analysis techniques and building deep learning models. As shown in Table 4.3 and Table 4.10, the method proposed in this section is more effective than existing deep-learning-based unit test case assertion generation methods (ATLAS) on the unit test case assertion generation task.

Although the overall effectiveness of methods combining information retrieval and deep learning is not as good as that of large-language-model-based methods, they consume far less computing resources than large-language-model-based methods. Specifically, for large-language-model-based methods, the parameter scale of the model is above 1B, while for the method proposed in this paper, the parameter scale of the model is only about 1M. This section analyzes the complementarity of large-language-model-based methods and methods combining information retrieval and deep learning, and points out the significance of methods combining information retrieval and deep learning in the era of large language models.

Figures 4.5(a) and 4.5(b) show the complementarity of large-language-model-

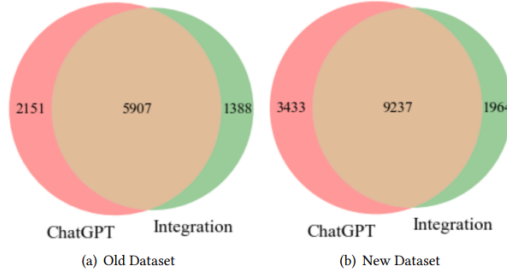


Figure 4.5: Complementarity between large-language-model-based methods and methods combining information retrieval and deep learning

based methods and methods combining information retrieval and deep learning on the old and new datasets. From the figures, it can be seen that the two methods have strong complementarity. The method combining information retrieval and deep learning can successfully generate 1,388 and 1,964 assertions on the old and new datasets respectively, all of which ChatGPT cannot successfully generate.

4.5 Summary of This Chapter

This chapter first uses a large language model (ChatGPT) to generate unit test case assertions. Then this chapter first attempts to use Information Retrieval (IR) to generate assertions, and proposes an information-retrieval-based assertion generation method, including IR techniques and Adaptation techniques. This chapter also proposes an integration method that integrates IR-based methods with DL-based methods (such as ATLAS) to further improve unit test case assertion generation effectiveness. Experimental results of this chapter show that the IR method significantly outperforms ATLAS on the assertion generation task, achieving 36.26% and 37.90% accuracy on the two datasets respectively. In addition, the Adaptation technique proposed in this chapter achieves 43.63% and 40.53% accuracy on the two datasets. The Integration technique proposed in this chapter

achieves 46.54% and 42.20% accuracy on the two datasets respectively. Finally, this chapter analyzes the complementarity of large-language-model-based methods and methods combining information retrieval and deep learning. The work in this chapter conveys an important message that non-large-language-model-based methods are competitive for software engineering tasks (such as assertion generation) and worth pursuing, and should be seriously considered by the research community, because in recent years, the research community has adopted large language model solutions too widely in software engineering tasks.

Chapter 5

Large-Language-Model-Based Detection of Incorrect Unit Test Case Assertions

The previous chapter introduced unit test case assertion generation methods based on deep learning and information retrieval. Although the method proposed in the previous chapter can surpass existing other advanced assertion generation methods, the method still produces nearly 50% incorrect assertions. Incorrect assertions would judge correct functions under test as incorrect, which is unacceptable for developers. Therefore, this chapter focuses on how to detect incorrect assertions. This chapter first introduces the background of large-language-model-based detection of incorrect unit test case assertions, then introduces DeCon, a large-language-model-based method for detecting incorrect unit test case assertions, then introduces experimental results and analysis, and finally summarizes this chapter.

5.1 Introduction

Software testing can be used to verify the correctness of the program under test. To perform software testing, developers write test cases consisting of test inputs and assertions to detect faults, thereby preventing software failures at deployment time [61, 107]. To reduce the manual work involved in writing test cases, various tools have been proposed to automatically generate test cases, such as EvoSuite [38], JBSE [11], and Randoop [81]. However, test cases generated by these tools can only help detect crash errors or regression errors, but cannot detect non-crash errors such as logic-related errors. For example, Randoop [81] and EvoSuite [38] create assertions based on capturing and asserting the return values of all non-void return methods in the test inputs. EvoSuite further reduces these assertions based on mutation testing [57, 125].

Although recent deep-learning (DL)-based methods [29, 106, 113, 122] can generate assertions that detect non-crash and non-regression faults, these methods face three main limitations. First, these methods have limited generalization ability. These methods are trained on limited training sets, limited training data, and programming languages, so they often perform poorly when applied to other datasets. Second, these methods have limited effectiveness. A study [106] shows that the top-1 accuracy of these methods is only 26.40%. Third, the false positive rate is high. A recent study [54] shows that more than 47% of the assertions generated by TOGA [29] (a state-of-the-art assertion generation method) are false positives, i.e., incorrect assertions. Incorrect assertions can lead to reporting correct programs under test as erroneous, greatly increasing the debugging burden of developers.

To attempt to address the above limitations, recent research [106] has used large language models (LLMs) to generate assertions, achieving better effectiveness than DL-based methods (in this chapter, DL-based methods are referred to

as non-large-language-model deep learning methods), but still suffer from high false positives (i.e., a high proportion of incorrect assertions among the generated assertions). For example, this chapter conducted an empirical study on the quality of assertions generated by four common code large language models (CodeGen [80], InCoder [39], Codex [19], and GPT-3.5 [92]) on the widely used code generation benchmark HumanEval. This chapter first evaluates the percentage of correctly executed, incorrectly executed (i.e., judging correct programs as incorrect), and non-executable assertions (e.g., cannot be successfully compiled and executed) among all generated assertions. Incorrectly executed and non-executable assertions are incorrect assertions. Our statistics show that among all generated assertions, the percentage of incorrectly executed assertions is 54.1%, and the percentage of non-executable assertions is 8.3%. In other words, among all generated assertions, the percentage of incorrect assertions is 62.4%.

To effectively detect incorrect assertions, this chapter proposes a method named DeCon. This method uses large language models to generate postconditions to detect incorrect assertions. In addition to docstrings and target function signatures for target problems, our method requires a small set of I/O examples (i.e., samples of example inputs and outputs) for target problems; these I/O examples can be manually written and prepared, or automatically generated by tools and then obtained through manual confirmation by developers. Developers can easily write some additional I/O examples for target problems (e.g., I/O examples contained in docstrings for target problems in HumanEval). Note that these I/O examples cannot be directly used to detect incorrect assertions, because discrete input-output pairs cannot detect the correctness of other discrete output-outputs. Therefore, this chapter first uses the I/O examples in docstrings written by developers to reduce incorrect postconditions generated by large language models, and then uses the remaining postconditions to detect incorrect

assertions. Specifically, DeCon includes three steps. First, DeCon provides the given function signature and docstring of the target problem to the large language model to generate candidate postconditions. In this step, this chapter designs a prompt format for the large language model to generate formal and executable postconditions. Second, among the candidate postconditions, this chapter uses the given I/O examples to filter out candidate postconditions that are violated by at least one given I/O example. Third, this chapter uses the remaining postconditions to detect incorrect assertions, because these assertions violate at least one remaining postcondition.

Experimental results show that using postconditions generated by GPT-3.5 and GPT-4, DeCon can detect more than 63.0% (precision) and 65.5% (precision) of incorrect assertions generated by four code large language models (CodeGen [80], InCoder [39], Codex [19], and GPT-3.5 [92]) respectively. Meanwhile, only 9.7% (recall rate 90.3%) and 6.5% (recall rate 93.5%) of correct assertions are misjudged as incorrect. Combining recall and precision, the F1 scores for GPT-3.5 and GPT-4 are 73.8% and 76.5% respectively. After removing detected incorrect assertions, using postconditions generated by GPT-3.5 and GPT-4, DeCon can improve the average Pass@1 of four large language models in code generation by 3.2% and 4.4% respectively. Although DeCon may filter out correct assertions, the remaining correct assertions can still retain 99.3% of the bug-finding ability. Compared with not using I/O examples to filter out incorrect postconditions generated by large language models, using I/O examples can help improve the bug-finding ability by 73.9% (96.4%-22.5%) and 25% (99.3%-74.3%).

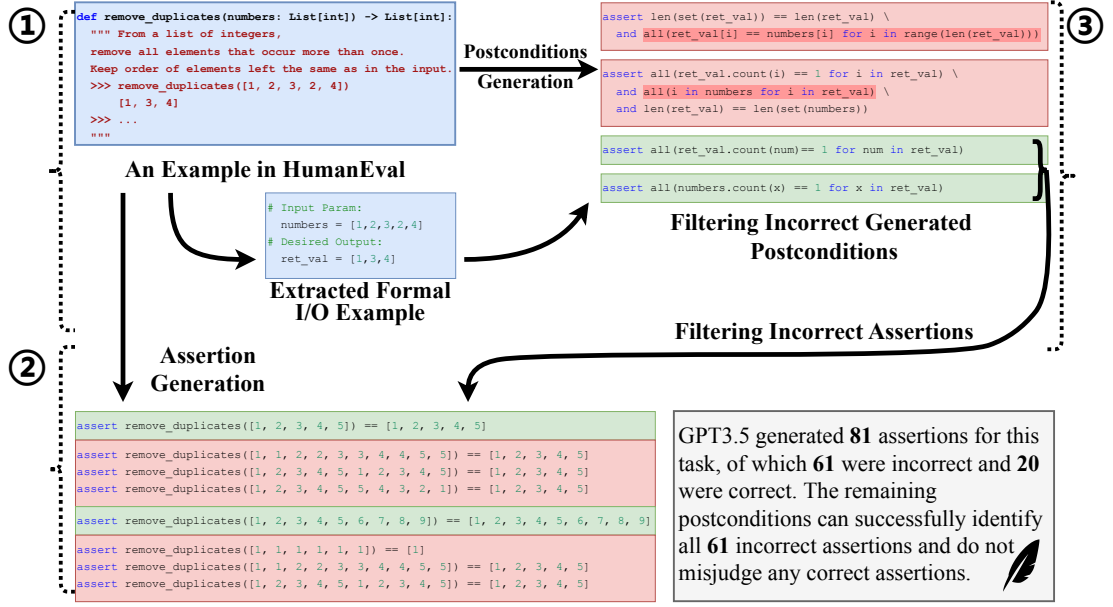


Figure 5.1: Specific example of DeCon detecting incorrect unit test case assertions

5.2 Large-Language-Model-Based Method for Detecting Incorrect Unit Test Case Assertions

This chapter first outlines DeCon (Section 5.2.1), then describes the detailed work in two parts: postcondition generation and filtering (Section 5.2.2) and incorrect assertion detection (Section 5.2.3).

5.2.1 Introduction to the Overall Method Flow

Figure 5.1 shows a specific example of DeCon. Overall, the core idea of DeCon is to use postconditions to detect incorrect assertions. As shown in Figure 5.1, when developers provide a function signature and function description, along with one or two input and output examples for the given function (Part 1), the assertions generated by the assertion generation tool for the given function signature and description are shown in Part 2. The postconditions generated by the large lan-

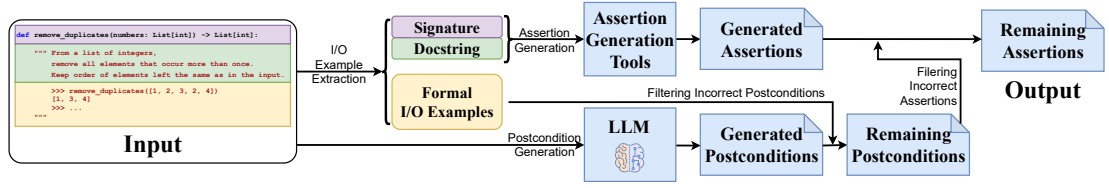


Figure 5.2: Overall flow chart of DeCon

guage model for the given function signature and description are shown in Part 3. After generating assertions and postconditions, this chapter automatically extracts formal input and output examples from the function description. Then, this chapter uses formal input and output examples to filter out incorrectly generated postconditions. The postconditions on the red background (filtered out) in Part 2 violate the I/O examples, while the postconditions on the green background (retained) in Part 2 do not violate the I/O examples. It should be noted that this chapter can guarantee that the filtered-out postconditions are all incorrect, but this chapter cannot guarantee that the remaining postconditions are all correct. The remaining postconditions may also misjudge correct assertions as incorrect. After generating postconditions that do not violate the I/O examples, this chapter uses these postconditions to filter assertions that violate these postconditions. Assertions with red background in Part 3 are filtered out, while assertions with green background are retained. Finally, GPT-3.5 generated 81 assertions for this problem, 61 of which are incorrect and 20 are correct. The remaining postconditions can successfully identify all 61 incorrect assertions without misjudging any correct assertions.

Figure 5.2 shows the overall flow chart of DeCon. The key idea of DeCon is to use postconditions to detect incorrect assertions. As shown in Figure 5.2, given a function signature and a functional description of the function (which contains one or two examples of inputs and expected outputs for the given function), the assertion generation tool generates assertions for the given function signature

and description. This chapter first extracts formal inputs and expected outputs from the docstring. This chapter extracts an average of 2.87 I/O examples from the docstring of each problem in HumanEval. Then, this chapter uses a large language model to generate postconditions for the given function signature and description. Next, this chapter uses formal input and output examples to filter out incorrect postconditions generated by the large language model. It should be noted that this chapter can guarantee that the filtered-out postconditions are all incorrect, but this chapter cannot guarantee that the remaining postconditions are all correct. The remaining postconditions may also misjudge correct assertions as incorrect. Finally, after generating postconditions that do not violate the I/O examples, this chapter uses these postconditions to detect assertions that violate these postconditions. The assertions generated by traditional unit test case generation tools (such as EvoSuite and Randoop) are based on the execution of the program implementation; assertions generated by traditional unit test case generation tools can never find logical errors. This section only focuses on methods that generate assertions based on large language models. This section does not use deep-learning-based assertion generation methods to generate assertions for two reasons. First, current deep-learning-based assertion generation methods are based on limited training sets (both training data and programming languages are limited), and cannot generalize to the HumanEval dataset. Second, a study [106] shows that the effectiveness of LLM-based unit test case assertion generation is higher than that of deep-learning-based methods.

5.2.2 Generation and Filtering of Postconditions

Figure 5.3 shows the processing of I/O example formalization. To automatically verify whether postconditions generated by LLMs are applicable to I/O examples, this section extracts I/O examples into the format of variable declarations (line

Algorithm 3: Algorithm flow chart for detecting incorrect assertions

```
Input : signature, an input method's signature.
Input : doc, the docstring of this method.
Input : AssertionGen, an LLM that generates assertions.
Input : PostGen, an LLM that generates postconditions.
Output: assertions, a list of assertions for this method

// check whether one assertion and one postcondition are compatible
1 Function compatible (assertion, condition) :
    // an assertion is an (input, output) pair
2     evalInput  $\leftarrow$  assertion.input;
3     evalOutput  $\leftarrow$  assertion.output;
4     postMethod  $\leftarrow$  “function(input, output){condition}”
5     return eval(postMethod(evalInput, evalOutput))

// Parsing docstring
6 description  $\leftarrow$  doc.descriptions;
7 caseAssertions  $\leftarrow$  doc.extractAssertions();

// generate assertions and postconditions
8 candidateAsserts  $\leftarrow$  TestGen(signature, description);
9 candidateConditions  $\leftarrow$  PostGen(caseAssertions);

// filtering postconditions
10 incConditions  $\leftarrow$   $\emptyset$ ;
11 for condition  $\in$  candidateConditions do
12     for assertion  $\in$  caseAssertions do
13         if not compatible(assertion, condition) then
14             incConditions.add(condition);
15 conditions  $\leftarrow$  candidateConditions – incConditions;

// filtering assertions
16 incAssertions  $\leftarrow$   $\emptyset$ ;
17 for assertion  $\in$  candidateAsserts do
18     for condition  $\in$  conditions do
19         if not compatible(assertion, condition) then
20             incAssertions.add(assertion);
21 assertions  $\leftarrow$  candidateAsserts – incAssertions;
22 return assertions;
```

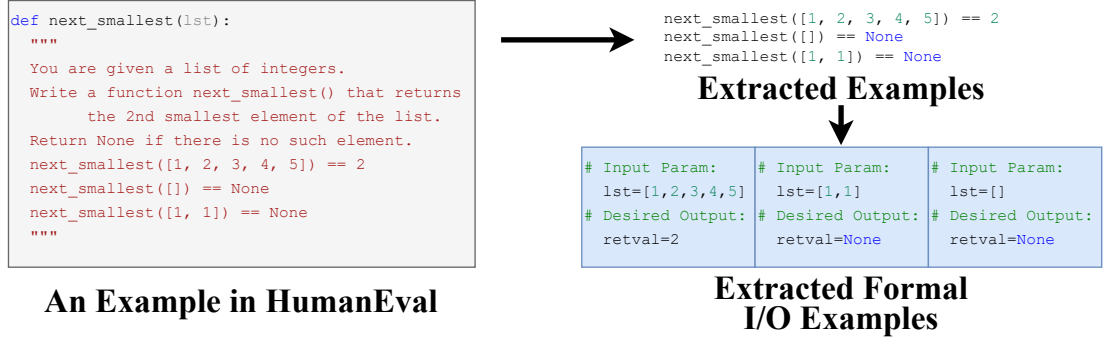


Figure 5.3: Example of formalization processing of I/O instances

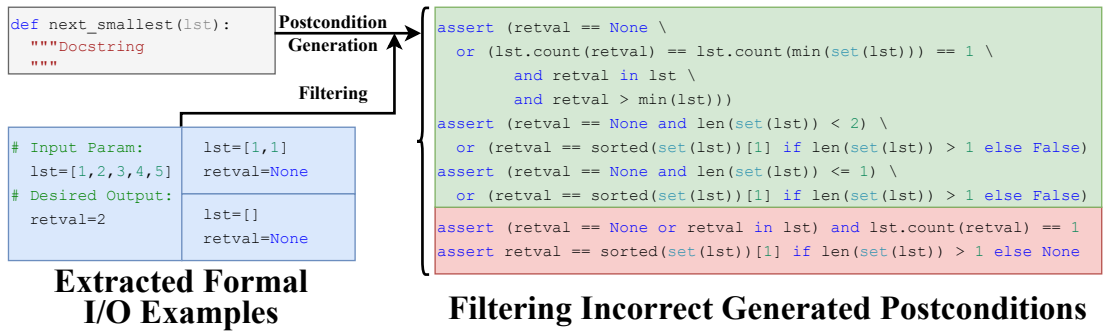


Figure 5.4: Example of DeCon filtering incorrect postconditions

6 in Algorithm 3). Similar to existing work [34], this chapter uses the prompts shown in Fig 5.5 for GPT-3.5 and GPT-4 to generate postconditions:

In the previous prompt, the bold parts represent input specific to different problems and programming languages, while the rest are fixed templates. Since the experiments in this section only focus on the Python version of HumanEval, "[PROGRAMMING LANGUAGE]" is "Python", and "[PROGRAMMING-LANGUAGE-SPECIFIC EXAMPLE]" is "def fun(num1, num2): x=(num1*num2)/num2 return x". For the example in the overview, "[FUNCTION NAME]" is "remove duplicates", and "[PROGRAM CONTEXT, FUNCTION STUB, AND DOCSTRING HERE]" is the first part of the overview. Since the experimental scenario in this section focuses on HumanEval, the target functions are all self-contained functions. For the experimental scenario in this section, the "[PRO-

Prompt Template

*You have the following code context, function stub and natural language specification (in the form of a code comment) for **[FUNCTION NAME]**. When implemented, the function should comply with this natural language specification: **[PROGRAM CONTEXT, FUNCTION STUB, AND DOCSTRING HERE]** Write a symbolic postcondition for **[FUNCTION NAME]** consisting of exactly one assert statement. For variables, use only the function input parameters and a hypothetical return value, which we'll assume is stored in a variable `return_val`. If the post condition calls any functions external to the program context, they should only be those from the functional subset of **[PROGRAMMING LANGUAGE]**. By this, we mean functions that are pure (i.e., no side effects) such as **[PROGRAMMING LANGUAGE-SPECIFIC EXAMPLE]**. Although the post condition should be less complex than the function itself, it should not be trivial. It should encapsulate an aspect of the function without implementing the function. The format of your response should be: code for exactly one postcondition with assert here.*

Figure 5.5: prompt used to generate postconditions.

GRAM CONTEXT]” program context is empty. For real development scenarios, “[PROGRAM CONTEXT]” is the context information that the function may depend on.

Figure 5.4 shows an example of filtering incorrect postconditions. The three postconditions on the green background satisfy all three I/O examples, and the two postconditions on the red background do not satisfy at least one of the three I/O instances. This section extracts each formal input and output from the docstring as variable declarations in the program, and places the generated postconditions after the variable declarations. This section ensures that after combining the extracted input and output examples with the postconditions to be filtered into a Python file, the combined Python file is executable. For any postcondition, if it fails to pass the formal input-output examples, the generated postcondition is considered incorrect (lines 10 to 15 in Algorithm 3). For the example in Fig-

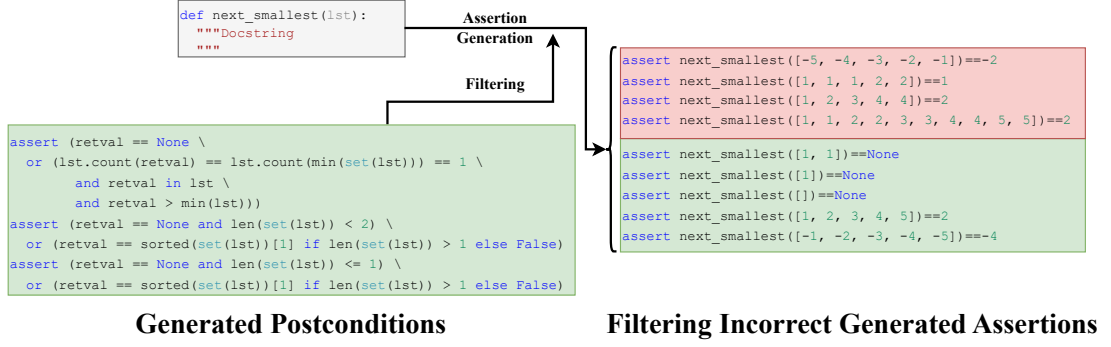


Figure 5.6: Example of DeCon detecting incorrect assertions

Figure 5.4, DeCon deletes postconditions with red background that do not satisfy three I/O instances, and retains postconditions with green background. Finally, DeCon uses the retained postconditions to detect incorrect assertion statements generated by the model or handwritten by developers.

5.2.3 Incorrect Assertion Detection

This section extracts parameters in each assertion as variable declarations in the program, and places all filtered postconditions after the variable declarations. This section ensures that after combining variables extracted from generated assertions with filtered postconditions into a Python file, the combined Python file is executable. For each assertion, if it fails to satisfy the postcondition, the generated assertion is considered incorrect (lines 16 to 21 in Algorithm 3). Figure 5.6 shows an example of detecting incorrect assertions. The five assertions on the green background satisfy all remaining three postconditions, and the four assertions on the red background do not satisfy at least one of the three remaining postconditions. Finally, DeCon judges assertions with green background as correct assertions, and assertions with red background as incorrect assertions. It should be noted that limited by the completeness and accuracy of postconditions, DeCon cannot guarantee not to misjudge correct assertions as incorrect, nor can

it guarantee to detect all incorrect assertions. Therefore, in the experimental evaluation section of this chapter, this section uses precision, recall, and F1 value to evaluate the effectiveness of DeCon.

5.3 Experimental Evaluation

5.3.1 Experimental Datasets

Decon-AssertData. This section sets the number of samples generated for each LLM to 100; each sample may contain one or more unit test case assertions. Similar to existing work [17], this section uses the following prompt to generate unit test cases for large language models:

”description of the function and check the correctness of *function_name* assert”.

This section uses four large language models in total (i.e., CodeGen, InCoder, Codex, and GPT3.5) to generate assertions (lines 5 and 7 in Algorithm 3).

This section statistically finds that CodeGen, InCoder, Codex, and GPT-3.5 generate 16,326, 126,529, 85,507, and 89,728 assertion statements respectively. After removing duplicate assertion statements, CodeGen, InCoder, Codex, and GPT-3.5 generate 13,970, 90,065, 54,643, and 29,452 non-duplicate assertions respectively.

HumanEval. HumanEval, released together with Codex [19], is a benchmark for measuring code generation methods, used to measure the functional correctness of programs synthesized from docstrings. It consists of 164 Python handwritten programming problems and solutions, each including a function signature, docstring, body, and several unit tests (averaging 7.7 tests per problem). For each problem, the input contains two parts: one is the natural language description of the problem, and the other is the function signature of the solution

containing function parameter types and expected return types. HumanEval is widely used as a benchmark by existing code large language model work to validate their effectiveness. Although functions in HumanEval are all self-contained functions and cannot reflect code generation effectiveness in real development scenarios, since DeCon can currently only reduce false positives of unit test case assertions generated by large language models for functions with basic input-output types, and functions in HumanEval are more complex than self-contained functions in real development scenarios, functions in HumanEval are very suitable for validating the effectiveness of DeCon’s work.

HumanEvalFix. Muennighoff et al. [76] manually injected faults into the standard implementation of each problem in HumanEval, resulting in obtaining HumanEvalFix’s incorrect implementations. All problems in HumanEvalFix have injected errors. HumanEvalFix was originally designed to evaluate the effectiveness of program repair, but since it contains both correct and error-prone code implementations, this section uses it to evaluate the fault-finding ability of assertions.

5.3.2 Experimental Setup

Assertion generation. For all models (i.e., CodeGen, InCoder, Codex, and GPT-3.5), this section generates 100 samples for each problem in HumanEval ¹. Temperature and top-k are 0.8 and 0.95, respectively. The kernel models of Codex, GPT-3.5, and GPT-4 used in this chapter are code-davinci-002, GPT-3.5-Turbo, and GPT-4-Turbo. It should be noted that although this paper uses DeCon to detect incorrect assertions generated by the above models to demonstrate the effectiveness of DeCon in the task of detecting incorrect assertions, DeCon can detect not only incorrect assertions generated by any model, but also incorrect

¹each sample may contain one or more assertions

assertions written by developers.

Postcondition generation. For postcondition generation models (i.e., GPT-3.5 and GPT-4), this section generates five samples for each problem in HumanEval². The temperature and top-k are the same as the assertion generation model. To avoid randomness in the experiments, this section conducted three experiments when using GPT-3.5 and GPT-4 to generate postconditions. Finally, this section finds that the results of the three experiments are relatively similar. Based on the weighted F1 score, this section selected the postconditions with the middle performance ranking.

5.3.3 Evaluation Metrics

Metrics for incorrect assertions. This section uses the built-in correct implementations in HumanEval to mark correct and incorrect assertions. If DeCon detects an incorrectly marked assertion, the incorrect assertion is considered successfully detected. Finally, the proportion of successfully detected incorrect assertions to the total number of incorrect assertions is calculated.

Compilability metric. This section uses the "return_value" of "process" in Python to determine whether there is a syntax error at runtime. If there is a syntax error, it is considered incompatible.

Metrics for detecting incorrect assertions. Since this chapter cannot guarantee that the generated postconditions and the remaining postconditions (after filtering out incorrect postconditions through I/O examples) are completely correct, postconditions may classify correct assertions as incorrect, or classify incorrect assertions as correct. This chapter selects precision, recall, and F1 score to evaluate the effectiveness of DeCon in detecting incorrect assertions. These metrics are also adopted by existing work on generating assertions. These metrics

²each sample may contain one or more assertion statements

are defined as follows:

$$\begin{aligned} Precision &= \frac{TP}{TP + FP}, & Recall &= \frac{TP}{TP + FN} \\ F1 &= \frac{2 \times Precision \times Recall}{Precision + Recall} \end{aligned} \quad (5.1)$$

Where TP and FP are correct and incorrect assertions marked as correct, respectively, and TN and FN are incorrect and correct assertions marked as incorrect, respectively.

Metrics for improving code generation effectiveness. This chapter adopts Pass@K as the metric for evaluating the behavioral correctness of generated code snippets based on assertions. When this chapter sets n (number of samples) to 100 and calculates Pass@K for K values of 1, 2, and 10, to avoid the problem of high sampling variance, this chapter uses Pass@K proposed by Codex in HumanEval.

Metrics for finding errors. HumanEvalFix is modified from the HumanEval dataset. The implementation of each problem in the HumanEvalFix dataset has errors. The more error problems found by the generated assertions, the higher the quality of the generated assertions is considered.

5.3.4 Experimental Results

(1) Number and proportion of different types of assertions (non-compilable, correctly executed, and incorrectly executed) generated by different models

This section classifies the assertions generated by LLMs into the following three steps. First, this section removes duplicate assertions from all generated assertions. Second, this section classifies assertions generated by LLMs into compilable and non-compilable parts based on whether the generated assertions can

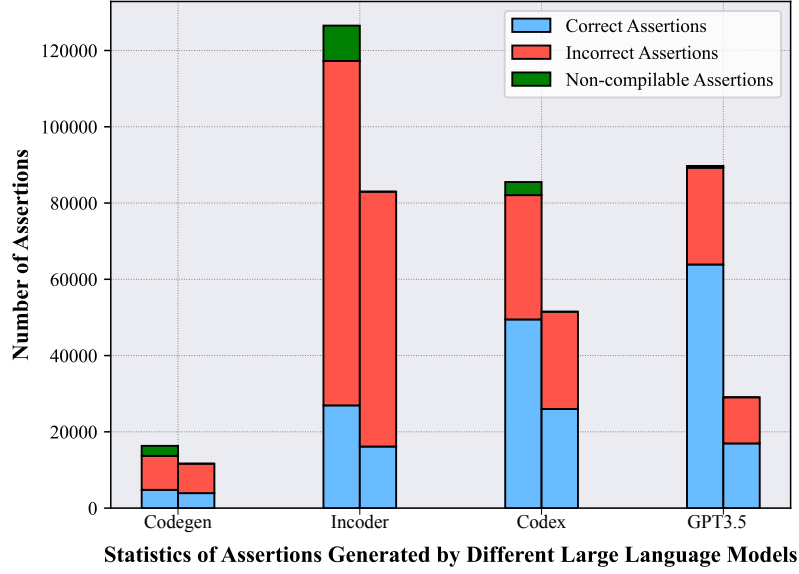


Figure 5.7: Number and proportion of different types of assertions generated by different models

be successfully compiled. Third, this section classifies compilable assertions into correct assertions and incorrect assertions (based on whether the groundtruth solutions provided by the HumanEval dataset satisfy them). Figure 5.7 shows the number and proportion of three types of assertions generated by different LLMs. For each LLM, the left column represents assertion results before deduplication, and the right column represents assertion results after deduplication. For each column, the green sub-column represents the number of non-compilable assertions, the red sub-column represents the number of incorrect assertions, and the blue sub-column represents the number of correct assertions.

From Figure 5.7, this section finds: (1) **Duplication**. The proportions of duplicate assertions in CodeGen, InCoder, Codex, and GPT-3.5 are 14.4%, 28.8%, 36.1%, and 67.2% respectively. (2) **Compilability**. Before deduplication, non-compilable assertions generated by three LLMs (except CodeGen) account for

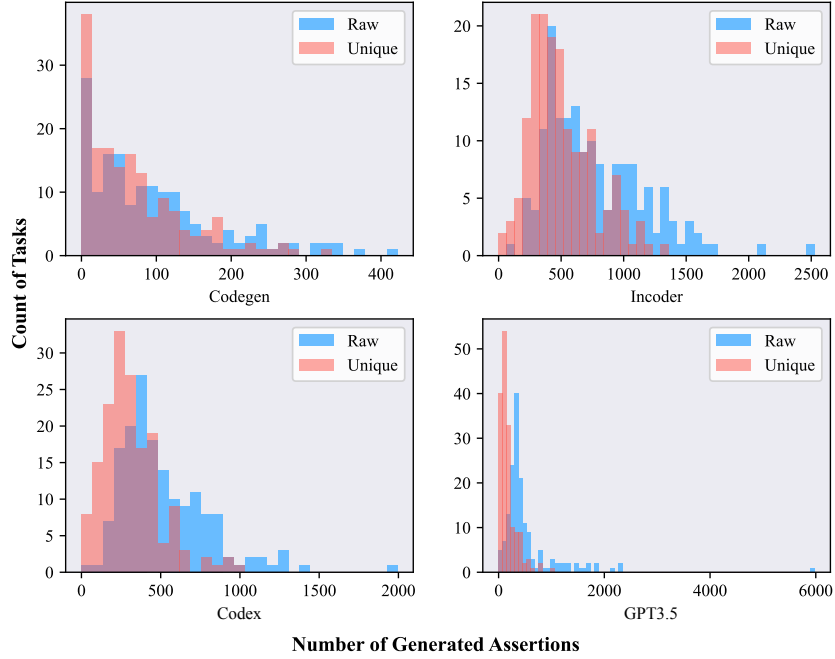


Figure 5.8: Distribution of the number of assertions generated by different models on HumanEval problems

8.3%, while the proportion of non-compilable assertions in all LLMs drops significantly to less than 0.5%, indicating that most non-compilable assertions are duplicates and easy to remove. (3) **Correctness.** After deduplication, the overall proportion of incorrect assertions among executable assertions on four LLMs is 59.6% (54.1% for all generated assertions). Specifically, the correctness proportions of CodeGen, Incoder, Codex, and GPT-3.5 are 55.3%, 74.2%, 46.7%, and 41.1%, respectively. Based on careful analysis of assertions generated by four LLMs, this section finds that (1) deduplication and compilation can help generate unique and correct assertions. (2) LLMs still generate about 62.4% incorrect assertions (8.3% non-compilable assertions and 54.1% compilable assertions), indicating the importance of detecting incorrect assertions. In addition, Figure 5.8 shows the distribution of the number of assertions generated on HumanEval prob-

lems. From the figure, it can be seen that the distribution of the number of unit test case assertions generated by the four models varies greatly. For example, the distribution of CodeGen is like an exponential distribution (only decreasing), while the distributions of the other three models are close to a normal distribution (first increasing and then decreasing). Considering the substantial differences among various problems in RQ3 and RQ4, in addition to directly evaluating all assertions, this section also designs another evaluation setting, evaluating the weighted results of each problem in HumanEval.

(2) Correctness of postconditions generated by large language models for functions in HumanEval.

Since each postcondition can contain one or more postconditions in the form of assert statements, GPT-3.5 generated 1,900 postconditions in total in the form of assert statements for 164 problems. In the rest of this section, this section refers to postconditions in the form of assert statements simply as postconditions. After deduplication, GPT-3.5 generated 1,698 postconditions in total. When filtering using the built-in test cases of the HumanEval dataset, 694 incorrect postconditions were filtered out. For GPT-4, it generated 820 postconditions for 164 problems. After deduplication, GPT-4 generated 671 postconditions in total. When filtering using the built-in test cases of the HumanEval dataset, 275 incorrect postconditions were filtered out. Usually, this section uses I/O examples in docstrings to filter out 539 incorrect postconditions among the 671 incorrect postconditions generated by GPT-4 (detected by built-in test cases in HumanEval), and 194 incorrect postconditions among the 275 incorrect postconditions (judged by built-in test cases in HumanEval). At the problem level, this section finds that GPT-3.5 did not generate any correct postconditions for 5 problems and generated completely correct postconditions for 16 problems. For GPT-4, GPT-4 did not generate any correct postconditions for 19 problems, and generated

completely correct postconditions for 87 problems. To explain the reasons for GPT’s poor effectiveness in generating postconditions, this section manually investigated the postconditions generated in problems (19-4=15 problems) where GPT-3.5 could generate correct postconditions when GPT-4 failed. This section finds that in these 15 problems, the postconditions generated by GPT-3.5 are very simple, such as "assert instance(return_val, bool)" and "assert return_val in ['yes', 'no']".

Table 5.1: Overall effectiveness of DeCon

AssertGen Model	PostGen Model	Number of assertions				Raw	Unweighted Metrics			Weighted Metrics		
		TP	FP	TN	FN	Prec.	Prec.	Rec.	F1	Prec.	Rec.	F1
CodeGen	GPT-3.5	3507	2465	5262	417	0.337	0.587	0.894	0.709	0.591	0.734	0.655
	GPT-4	3656	2257	5470	268	0.337	0.618	0.932	0.743	0.647	0.779	0.707
InCoder	GPT-3.5	14317	15901	50932	1829	0.195	0.474	0.887	0.618	0.575	0.824	0.677
	GPT-4	14601	15729	51104	1545	0.195	0.481	0.904	0.628	0.665	0.874	0.755
Codex	GPT-3.5	23493	10391	15111	2499	0.505	0.693	0.904	0.785	0.732	0.871	0.795
	GPT-4	24544	9823	15679	1448	0.505	0.714	0.944	0.813	0.788	0.917	0.848
GPT-3.5	GPT-3.5	15715	4684	7407	1250	0.584	0.770	0.926	0.841	0.780	0.902	0.836
	GPT-4	16325	3950	8141	640	0.584	0.805	0.962	0.877	0.815	0.941	0.873

- Column "Raw Prec." is defined as $(TP + FN) / \text{Total}$, the precision of the large language model directly generating assertions

(3) Effectiveness of DeCon in detecting incorrect assertions

Analysis method. Since this chapter cannot guarantee that the generated postconditions and the remaining postconditions (after filtering out incorrect postconditions through I/O examples) are completely correct, postconditions may classify correct assertions as incorrect, or classify incorrect assertions as correct. This section evaluates the effectiveness of DeCon in detecting incorrect assertions based on the postconditions generated in this section from the perspective of three metrics: precision, recall, and F1 score. As shown in Section 5.3.3, this section also averages these three metrics as weighted metrics across various problems to alleviate the number of assertions in different problems (as shown in Figure 5.8).

This section evaluates each combination of AssertGen model (generating assertions) and PostGen model (generating postconditions). Specifically, this section uses CodeGen, InCoder, Codex, and GPT-3.5 as our AssertGen models, and uses GPT-3.5 and GPT-4 as our PostGen models.

Improvement in precision. Table 5.1 shows the precision, recall, and F1 score of DeCon in identifying correct assertions. The results show that for each combination of AssertGen and PostGen models, DeCon can effectively improve the precision of assertions. Notably, DeCon improves the precision of assertions by at least 31.8%, indicating that DeCon can effectively improve the precision of assertions generated by LLMs by detecting incorrect assertions.

Trade-off between precision and recall. In addition to the improvement in precision, Table 5.1 also lists the recall results and F1 scores of DeCon. Notably, all recall values exceed 0.887, demonstrating the effectiveness of DeCon in preserving correct assertions. To reflect the trade-off between precision and recall, this section also evaluates using F1 scores, and DeCon achieves an average F1 value of 0.752. In addition, models with higher raw precision scores (such as Codex and GPT-3.5) can also benefit from DeCon, thus demonstrating the generalization ability of DeCon.

Weighted metric results. Table 5.1 also shows the weighted results of DeCon in detecting incorrect assertions generated by large language models for functions in HumanEval. Compared with unweighted metrics, precision slightly improves (less than 0.1), while recall slightly decreases (less than 0.15). This difference indicates that DeCon maintains consistent performance across different problems in the HumanEval dataset.

(3) The effectiveness of the incorrect postcondition filtering technique for detecting incorrect assertions.

In the previous section, this section demonstrated the effectiveness of DeCon

Table 5.2: Effectiveness of DeCon without I/O filtering of incorrect postconditions

AssertGen Model	PostGen Model	Number of assertions				Raw	Unweighted Metrics			Weighted Metrics		
		TP	FP	TN	FN	Prec.	Prec.	Rec.	F1	Prec.	Rec.	F1
CodeGen	GPT-3.5	749	253	7474	3175	0.337	0.748	0.191	0.304	0.220	0.138	0.169
	GPT-4	2523	784	6943	1401	0.337	0.763	0.643	0.698	0.601	0.55	0.574
InCoder	GPT-3.5	3131	904	65929	13015	0.195	0.776	0.194	0.310	0.305	0.166	0.215
	GPT-4	9841	5952	60881	6305	0.195	0.623	0.610	0.616	0.673	0.625	0.648
Codex	GPT-3.5	5154	642	24860	20838	0.505	0.889	0.198	0.324	0.320	0.161	0.214
	GPT-4	16609	4189	21313	9383	0.505	0.799	0.639	0.710	0.733	0.635	0.681
GPT-3.5	GPT-3.5	3582	211	11880	13383	0.584	0.944	0.211	0.345	0.303	0.172	0.220
	GPT-4	10724	1509	10582	6241	0.584	0.877	0.632	0.735	0.749	0.643	0.692

- Column "Raw Prec." is defined as $(TP + FN) / \text{Total}$, the precision of the large language model directly generating assertions

in detecting incorrect assertions. In this section, this section conducts an ablation experiment to evaluate the contribution of incorrect postcondition filtering (i.e., filtering incorrect postconditions based on I/O examples) to implementation effectiveness.

Method. To evaluate the contribution of incorrect postconditions filtered by I/O examples, this section conducts an ablation experiment that does not filter candidate postconditions, and the study is based on I/O examples extracted from docstrings (lines 9-14 in Algorithm 3). This section uses precision, recall, and F1 score in both weighted and unweighted settings.

Trade-off between precision and recall. Table 5.2 shows the effectiveness of DeCon without I/O filtering of incorrect postconditions. Compared with the results of I/O example filtering of incorrect postconditions (Table 5.1), this section observes that the maximum precision increases by 0.302. The only difference between these two tables is that Table 5.2 retains many incorrect postconditions that were originally filtered out during I/O example filtering. These incorrect postconditions lead to the removal of both correct and incorrect assertions. Specifically, the reduction ratio of false positive (FP) assertions ranges

from 62% (1 - 1509 / 3950) to 79% (1 - 211 / 4684), while the reduction ratio of true positive assertions ranges from 32% to 79%. Although the reduction of TP and FP assertions leads to an improvement in precision, it also greatly increases FN assertions, resulting in low recall values. For example, with GPT-3.5 as the PostGen model, the highest recall rate is only 0.211, indicating that most correct assertions are removed. The comparison shows that I/O example filtering of incorrect postconditions greatly reduces FN assertions (ranging from 39% to 368%) while increasing a small number of FP assertions. Using F1 score as an evaluation metric for the trade-off, I/O example filtering of incorrect postconditions can improve the F1 values of GPT-3.5 and GPT-4 by an average of 70%. Specifically, when the PostGen model is GPT-3.5 and GPT-4, it improves by 130% and 11%, respectively.

Impact on weighted metrics. Table 5.2 also shows results weighted over various problems in the HumanEval dataset. This section finds that in most combinations of AssertGen and PostGen models, weighted metrics are lower than unweighted scores (except for the combination of InCoder and GPT-4). This difference stems from the distribution of assertion numbers. As shown in Figure 5.8, these models generate more assertions for problems with higher confidence, thus making unweighted metrics perform better. In addition, this section notices that the effectiveness of CodeGen decreases more than that of other models. The investigation in this chapter shows that CodeGen failed to generate any correct assertions in 26 problems, while the other three models only failed in 5, 3, and 3 problems respectively. This imbalance in CodeGen’s results mainly led to the decline of CodeGen’s ineffectiveness on weighted metrics.

(4) DeCon’s improvement on the effectiveness of code generated by large language models

To further explore the role of detecting incorrect assertions, this section studies

Table 5.3: Effectiveness of DeCon in improving code generation effectiveness

Model	Baseline			CodeT (raw)			CodeT + DeCon					
							GPT-3.5 as PostGen			GPT-4 as PostGen		
	@1	@10	@100	@1	@2	@10	@1	@2	@10	@1	@2	@10
CodeGen	29.7%	50.3%	73.7%	35.8%	43.0%	61.6%	39.4%	47.5%	62.0%	40.8%	47.5%	64.4%
InCoder	16.4%	28.3%	47.5%	17.7%	22.3%	36.4%	23.8%	29.5%	39.5%	24.5%	30.3%	41.7%
Codex	47.0%	74.9%	92.1%	62.9%	75.2%	85.0%	64.4%	77.4%	87.1%	67.0%	77.6%	87.3%
GPT-3.5	51.2%	77.1%	92.0%	65.2%	72.7%	85.7%	66.8%	74.9%	85.7%	66.9%	75.7%	86.8%

the help of detecting incorrect assertions in improving code generation effectiveness. Here, this section applies the idea of CodeT [17]. The idea of CodeT is to generate assertions while large language models generate code. By designing a voting mechanism (for the same problem, multiple assertions and generated code execute each other to see the number of passes), the effectiveness of code generated by large language models can be improved. The core idea of CodeT is that correct answers are all the same, while incorrect answers are in various forms.

Method. For each large language model that generates code, this section generates 1, 2, and 10 candidate programs for each problem in HumanEval. This section first uses CodeT to directly use the assertions generated by the large language model to improve the effectiveness of code generation by the large language model. Then, within the framework of CodeT, this section uses DeCon to remove incorrect assertions from all assertions generated by the large language model to see the improvement in the effectiveness of code generation by the large language model after applying DeCon.

Overall results. As shown in Table 5.3, using DeCon-filtered assertions can improve the effectiveness of code generated by CodeT on all large language models. Specifically, for postconditions generated by GPT-3.5, DeCon improves the average Pass@K of the four large language models by 3.2%, 4.0%, and 1.4%

in Pass@1, Pass@2, and Pass@10, respectively. For postconditions generated by GPT-4, DeCon improves the average Pass@K of the four models by 4.4%, 4.5%, and 2.9% in Pass@1, Pass@2, and Pass@10, respectively. Comparing the Pass@K results of CodeT alone and CodeT+DeCon, this section finds that: (1) DeCon can effectively improve the effectiveness of code generated by LLMs; (2) using GPT-4 as the PostGen model for DeCon to generate postconditions is more effective than using GPT-3.5 as the PostGen model for DeCon to generate postconditions.

Table 5.4: Effectiveness of DeCon on fault finding tasks

TestGen	PostGen	Total	With E.F.	Without E.F.
CodeGen	GPT-3.5	112	106	23
	GPT-4	112	110	80
InCoder	GPT-3.5	144	139	36
	GPT-4	144	143	110
Codex	GPT-3.5	153	150	36
	GPT-4	153	152	114
GPT-3.5	GPT-3.5	152	146	31
	GPT-4	152	152	113
Average	GPT-3.5	140.25	135.25	31.5
	GPT-4	140.25	139.25	104.25

- E.F. refers to "Example Filtering" (i.e., how many postconditions can be filtered by I/O examples)

(5) Effectiveness of DeCon in finding errors.

To further demonstrate the effectiveness of DeCon, this section conducted an error-finding experiment to show the fault-finding ability after detecting and removing incorrect assertions. The fault-finding experiment shows that although DeCon may filter out correct assertions, the fault-finding ability of the remaining correct assertions decreases only slightly. Compared with the fault-finding ability of assertions filtered by postconditions (without I/O example filtering), the fault-

finding ability of assertions filtered by postconditions (with I/O example filtering) is improved.

Method. This section compares the effectiveness of assertions generated by large language models without DeCon filtering and assertions filtered by DeCon from the perspective of fault finding. For all four large language models, this section marks errors discovered through correct assertions (based on our labeling in the HumanEvalFix dataset) as erroneous programs. Then, this section evaluates how many problems (out of 164 total problems) in the HumanEvalFix dataset can find their true errors through assertions with/without DeCon example filtering (i.e., filtering incorrect postconditions based on I/O examples).

Overall results. Table 5.4 shows the overall effectiveness of DeCon in fault finding. The "Total" column refers to the total number of errors that can be found in correct assertions generated by each LLM. For some problems, assertions generated by LLMs cannot find errors. This chapter finds that correct assertions generated by CodeGen have the worst fault detection ability, finding 114 errors out of 164 problems, while the other three models have better fault detection abilities, with Codex and GPT-3.5 finding 153 and 152 problems out of 164 respectively. The "With E.F." column refers to the effectiveness of postcondition-filtered assertions; these postconditions are filtered by I/O examples. On average, across various AssertGen models, 96.4% (135.25/140.25) and 99.3% (139.25/140.25) of groundtruth fault-problems respectively can be found by assertions generated by DeCon with I/O example filtering on postconditions generated by GPT-3.5 and GPT-4.

Comparison with I/O examples in docstrings. Since our method DeCon uses I/O examples when detecting incorrect assertions, this section needs to conduct fault-finding experiments directly using I/O examples. This section extracts I/O examples from the docstring of the function and automatically constructs

them as executable assertion statements. Then, experiments are conducted on data in HumanEvalFix. The experiment found that using only the extracted I/O examples can find 152 faulty problems, which is consistent with DeCon. Although the experimental results in this section confirm the consistency of DeCon in the number of faults, they also highlight the complementarity of using I/O examples and DeCon. Notably, among the 12 problems that cannot be found using I/O examples in docstrings, DeCon manages to identify 10 of them.

5.4 Summary of This Chapter

This chapter first conducts an empirical study on the quality of assertions generated by four large language models on HumanEval, and finds that 62.4% of the generated assertions are incorrect. In addition to docstrings and target function signatures of target problems, based only on a few I/O examples, this chapter proposes a method named DeCon, which detects whether the generated assertions are incorrect by generating postconditions. Experimental results show that DeCon can detect more than 64% of the incorrect assertions generated by LLMs (63% and 65.5% for GPT-3.5 and GPT-4 respectively), and DeCon can also improve the effectiveness of large language models in code generation and bug-finding tasks.

Chapter 6

Summary

Automatic code generation, as an important research branch in the field of software engineering, has self-evident importance and value. This paper aims at generating code and unit test case assertions, focusing on how to better generate code and unit test case assertions based on large language models, mainly including constructing benchmarks reflecting real development scenarios, designing and training a code large language model Wenwang for real development scenarios, generating unit test case assertions based on large language models and a combination of deep learning and information retrieval, and detecting incorrect unit test case assertions based on large language models. Overall, the main work and innovations of this paper include the following:

- This paper constructs a benchmark, CoderEval, for validating the code generation capability of large language models, used to validate the ability of large language models to generate code for real development scenarios, supporting both Python and Java languages. This paper points out that the currently used benchmark HumanEval for validating the code generation capability of large language models can only validate functions of algorithm-problem type (i.e., self-contained functions that can be executed without

relying on any information outside the function), and cannot validate the ability of large language models to generate code for real development scenarios. This paper selects representative open-source projects by analyzing the tag and star count information of all Python and Java projects on GitHub, adds contextual information to the selected functions through program analysis techniques to add the functionality of evaluating the effectiveness of the contextual information of the code generated by the model to CoderEval, eliminates the data leakage problem by inviting developers to manually label natural language descriptions, and improves the usability of CoderEval by providing a Docker testing environment and incremental compilation technology.

- This paper proposes a unit test case assertion generation method based on large language models and a combination of information retrieval and deep learning, including information-retrieval-based unit test case generation techniques, automatic adaptation techniques based on heuristic search and deep neural networks, and techniques based on combining information retrieval and deep learning. This paper first attempts large-language-model-based unit test case assertion generation methods, and through comparison with the unit test case assertion generation methods based on combining information retrieval and deep learning, points out that although large language models can achieve better results in test case assertion generation, they still have complementarity with the information-retrieval-and-deep-learning-based unit test case assertion generation methods, which consume fewer resources and have faster generation speed.
- This paper proposes a large-language-model-based method for detecting incorrect test case assertions, which can detect 64% of the incorrect unit

test case assertions generated by large language models. This paper points out that although the unit test cases generated by existing deep-learning-based methods can find logical errors in code compared to traditional unit test case generation methods, they also bring a very high false positive rate, that is, judging correct code as incorrect. This paper first analyzes the proportion of incorrect unit test case assertions generated by existing large language models, and finds that more than 50% of the unit test case assertions generated by existing large language models are incorrect. To detect incorrect unit test case assertions generated by large language models, this paper provides the given function signature and docstring of the target problem to the large language model to generate candidate postconditions. In this step, this paper designs a prompt format for the large language model to generate formal and executable postconditions. Secondly, among the candidate postconditions, this paper uses the given I/O examples to filter out candidate postconditions that are violated by at least one of the given I/O examples. Finally, this paper uses the remaining postconditions to detect incorrect assertions.

Bibliography

- [1] M Moein Almasi, Hadi Hemmati, Gordon Fraser, Andrea Arcuri, and Janis Benefelds. An industrial evaluation of unit test generation: Finding real faults in a financial application. In *2017 IEEE/ACM 39th International Conference on Software Engineering: Software Engineering in Practice Track (ICSE-SEIP)*, pages 263–272. IEEE, 2017. 33, 63
- [2] Dave Astels. *Test driven development: A practical guide*. Prentice Hall Professional Technical Reference, 2003. 23, 63
- [3] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021. 14, 39
- [4] Christian Bauer. *Hibernate in action*. "Manning Pubns Co", 2005. 24
- [5] Kent Beck. *Test-driven Development: By Example*. Addison-Wesley Professional; 1st edition (November 8, 2002), 2003. 63
- [6] Islam Beltagy and Chris Quirk. Improved semantic parsers for if-then statements. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 726–736, 2016.

- [7] Avishkar Bhoopchand, Tim Rocktäschel, Earl Barr, and Sebastian Riedel. Learning python code suggestion with a sparse pointer network. *arXiv preprint arXiv:1611.08307*, 2016. 4
- [8] Sid Black, Leo Gao, Phil Wang, Connor Leahy, and Stella Biderman. GPT-Neo: Large Scale Autoregressive Language Modeling with Mesh-Tensorflow. March 2021. 11, 38
- [9] Amiangshu Bosu, Anindya Iqbal, Rifat Shahriyar, and Partha Chakraborty. Understanding the motivations, challenges and needs of blockchain software developers: A survey. *Empirical Software Engineering*, 24(4):2636–2673, 2019. 23
- [10] Luc Bougé, Nicole Choquet, Laurent Fribourg, and M-C Gaudel. Test sets generation from algebraic specifications using logic programming. *Journal of Systems and Software*, 6(4):343–360, 1986. 18
- [11] Pietro Braione, Giovanni Denaro, and Mauro Pezzè. Jbse: A symbolic executor for java programs with complex heap inputs. In *Proceedings of the 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (ESEC/FSE)*, page 1018–1022, 2016. 27, 62, 95
- [12] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, pages 1877–1901, 2020. 38
- [13] Ruichu Cai, Boyan Xu, Xiaoyan Yang, Zhenjie Zhang, Zijian Li, and Zhihao Liang. An encoder-decoder framework translating natural language to database queries. *arXiv preprint arXiv:1711.06061*, 2017. 2

- [14] Eugenio Capra, Chiara Francalanci, and Francesco Merlo. An empirical study on the relationship between software design quality, development effort and governance in open source projects. *IEEE Transactions on Software Engineering*, 34(6):765–782, 2008. 23
- [15] Federico Cassano, John Gouwar, Daniel Nguyen, Sydney Nguyen, Luna Phipps-Costin, Donald Pinckney, Ming-Ho Yee, Yangtian Zi, Carolyn Jane Anderson, Molly Q Feldman, Arjun Guha, Michael Greenberg, and Abhinav Jangda. Multipl-e: A scalable and polyglot approach to benchmarking neural code generation. *IEEE Transactions on Software Engineering*, pages 3675–3691, 2023. 13, 14, 39
- [16] Shubham Chandel, Colin B Clement, Guillermo Serrato, and Neel Sundaresan. Training and evaluating a jupyter notebook data science assistant. *arXiv preprint arXiv:2201.12901*, 2022. 39
- [17] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. Codet: Code generation with generated tests. *arXiv preprint arXiv:2207.10397*, 2022. 105, 116
- [18] Lianping Chen. Continuous delivery: Huge benefits, but challenges too. *IEEE software*, 32(2):50–54, 2015. 28
- [19] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. 7, 13, 20, 24, 32, 38, 39, 41, 44, 96, 97, 105
- [20] Qian Chen, Xiaodan Zhu, Zhen-Hua Ling, Si Wei, Hui Jiang, and Diana Inkpen. Enhanced LSTM for natural language inference. In *Proceedings of*

- the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1657–1668, 2017. 81
- [21] Tsong Yueh Chen, F-C Kuo, Robert G Merkel, and Sebastian P Ng. Mirror adaptive random testing. *Information and Software Technology*, 46(15): 1001–1010, 2004. 16
- [22] Tsong Yueh Chen, Fei-Ching Kuo, Robert G Merkel, and TH Tse. Adaptive random testing: The art of test case diversity. *Journal of Systems and Software*, 83(1):60–66, 2010. 16
- [23] Fenia Christopoulou, Gerasimos Lampouras, Milan Gritta, Guchun Zhang, Yinpeng Guo, Zhongqi Li, Qi Zhang, Meng Xiao, Bo Shen, Lin Li, Hao Yu, Li Yan, Pingyi Zhou, Xin Wang, Yuchi Ma, Ignacio Iacobacci, Yasheng Wang, Guangtai Liang, Jiansheng Wei, Xin Jiang, Qianxiang Wang, and Qun Liu. Pangu-coder: Program synthesis with function-level language modeling. *arXiv preprint arXiv:2207.11280*, 2022. vii, 11, 12, 32, 38, 51
- [24] Mike Cohn. *Succeeding with Agile: Software Development Using Scrum (1st ed.)*. Addison-Wesley Professional; 1st edition (October 26, 2009), 2010. 62
- [25] Lisa Crispin. Driving software quality: How test-driven development impacts software quality. *IEEE software*, 23(6):70–71, 2006. 27
- [26] Leonardo De Moura and Nikolaj Bjørner. Z3: An efficient smt solver. In *International conference on Tools and Algorithms for the Construction and Analysis of Systems*, pages 337–340. Springer, 2008. 17
- [27] Karnig Derderian, Robert M Hierons, Mark Harman, and Qiang Guo. Automated unique input output sequence generation for conformance testing of fsms. *The Computer Journal*, 49(3):331–344, 2006. 16

- [28] Lee R. Dice. Measures of the amount of ecologic association between species. *Ecological Society of America*, 1945. 83
- [29] Elizabeth Dinella, Gabriel Ryan, Todd Mytkowicz, and Shuvendu K. Lahiri. Toga: A neural method for test oracle generation. In *Proceedings of the 44th International Conference on Software Engineering*, page 2130–2141, 2022. 21, 95
- [30] Li Dong and Mirella Lapata. Language to logical form with neural attention. *arXiv preprint arXiv:1601.01280*, 2016. 2, 6
- [31] Li Dong and Mirella Lapata. Coarse-to-fine decoding for neural semantic parsing. *arXiv preprint arXiv:1805.04793*, 2018. 6
- [32] Bruno Dutertre and Leonardo De Moura. A fast linear-arithmetic solver for dpll (t). In *International Conference on Computer Aided Verification*, pages 81–94. Springer, 2006. 17
- [33] Hartmut Ehrig and Bernd Mahr. *Fundamentals of algebraic specification 1: Equations and initial semantics*, volume 6. Springer Science & Business Media, 2012. 18
- [34] Madeline Endres, Sarah Fakhoury, Saikat Chakraborty, and Shuvendu K Lahiri. Formalizing natural language intent into program specifications via large language models. *arXiv preprint arXiv:2310.01831*, 2023. 102
- [35] Robert B. Evans and Alberto Savoia. Differential testing: A new approach to change detection. In *Proceedings of the 6th Joint Meeting on European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering: Companion Papers (ESEC/FSE)*, page 549–552, 2007. 62

- [36] R. A. Fisher. *The design of experiments*. Oliver & Boyd, 1935. 17
- [37] Martin Fowler and Matthew Foemmel. Continuous integration. 2006. 28
- [38] Gordon Fraser and Andrea Arcuri. Evosuite: Automatic test suite generation for object-oriented software. In *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE)*, pages 416–419, 2011. 27, 33, 62, 95
- [39] Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Wen-tau Yih, Luke Zettlemoyer, and Mike Lewis. Incoder: A generative model for code infilling and synthesis. *arXiv preprint arXiv:2204.05999*, 2022. 9, 38, 96, 97
- [40] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. The pile: An 800gb dataset of diverse text for language modeling. *CoRR*, abs/2101.00027, 2021. *arXiv preprint arXiv:2101.00027*. 10
- [41] Bobby George and Laurie Williams. An initial investigation of test driven development in industry. In *Proceedings of the 2003 ACM symposium on Applied computing*, pages 1135–1139, 2003. 23
- [42] Xiaodong Gu, Hongyu Zhang, Dongmei Zhang, and Sunghun Kim. Deep api learning. In *Proceedings of the 2016 24th ACM SIGSOFT international symposium on foundations of software engineering*, pages 631–642, 2016. 3
- [43] Riccardo Guidotti, Anna Monreale, Salvatore Ruggieri, Franco Turini, Fosca Giannotti, and Dino Pedreschi. A survey of methods for explaining black box models. *ACM Computing Surveys*, 51(5):93:1–93:42, 2019. 63

- [44] Patrick Haluptzok, Matthew Bowers, and Adam Tauman Kalai. Language models can teach themselves to program better. *arXiv preprint arXiv:2207.14502*, 2022. 39
- [45] Paul Hamill. *Unit test frameworks: tools for high-quality software development*. " O'Reilly Media, Inc.", 2004. 27
- [46] Yiyang Hao, Ge Li, Yongqiang Liu, Xiaowei Miao, He Zong, Siyuan Jiang, Yang Liu, and He Wei. Aixbench: A code generation benchmark dataset. 2022. 13, 14, 39
- [47] Mark Harman. Software engineering meets evolutionary computation. *Computer*, 44(10):31–39, 2011. 16
- [48] Mark Harman and Bryan F Jones. Search-based software engineering. *Information and software Technology*, 43(14):833–839, 2001. 16
- [49] Mark Harman and Phil McMinn. A theoretical and empirical study of search-based testing: Local, global, and hybrid search. *IEEE Transactions on Software Engineering*, 36(2):226–247, 2009. 16
- [50] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016. 24
- [51] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. Measuring coding challenge competence with apps. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, volume 1, 2021. 39

- [52] Abram Hindle, Earl T. Barr, Zhendong Su, Mark Gabel, and Premkumar Devanbu. On the naturalness of software. In *2012 34th International Conference on Software Engineering (ICSE)*, pages 837–847, 2012. 2
- [53] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. *arXiv preprint arXiv:1904.09751*, 2019. 51
- [54] Soneya Binta Hossain, Antonio Filieri, Matthew B Dwyer, Sebastian Elbaum, and Willem Visser. Neural-based test oracle generation: A large-scale evaluation and lessons learned. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 120–132, 2023. 95
- [55] Xing Hu, Ge Li, Xin Xia, David Lo, and Zhi Jin. Deep code comment generation. In *Proceedings of the 26th International Conference on Program Comprehension (ICPC)*, page 200–210, 2018. 24
- [56] Srinivasan Iyer, Ioannis Konstas, Alvin Cheung, and Luke Zettlemoyer. Mapping language to code in programmatic context. *arXiv preprint arXiv:1808.09588*, 2018. 13, 15
- [57] Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5): 649–678, 2011. 62, 95
- [58] Rafal Józefowicz, Oriol Vinyals, Mike Schuster, Noam Shazeer, and Yonghui Wu. Exploring the limits of language modeling. *CoRR*, abs/1602.02410, 2016. 64
- [59] James C King. A new approach to program testing. *ACM Sigplan Notices*, 10(6):228–233, 1975. 17

- [60] Roger E Kirk. Experimental design. *Sage handbook of quantitative methods in psychology*, pages 23–45, 2009. 17
- [61] Pavneet Singh Kochhar, Ferdian Thung, and David Lo. Code coverage and test suite effectiveness: Empirical study with real bugs in large systems. In *2015 IEEE 22nd international conference on software analysis, evolution, and reengineering (SANER)*, pages 560–564. IEEE, 2015. 95
- [62] Oleksii Kuchaiev and Boris Ginsburg. Factorization tricks for LSTM networks. In *Workshop Track Proceedings of the 5th International Conference on Learning Representations (ICLR)*, 2017. 64
- [63] Yuhang Lai, Chengxi Li, Yiming Wang, Tianyi Zhang, Ruiqi Zhong, Luke Zettlemoyer, Scott Wen tau Yih, Daniel Fried, Sida Wang, and Tao Yu. Ds-1000: A natural and reliable benchmark for data science code generation. *arXiv preprint arXiv:2211.11501*, 2022. 13, 14, 39
- [64] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K Lahiri, and Siddhartha Sen. Codamosa: Escaping coverage plateaus in test generation with pre-trained large language models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pages 919–931. IEEE, 2023. 20
- [65] Julia Lerman and Rowan Miller. *Programming entity framework: code first*. ” O’Reilly Media, Inc.”, 2012. 24
- [66] Jian Li, Yue Wang, Michael R Lyu, and Irwin King. Code completion with neural attention and pointer networks. *arXiv preprint arXiv:1711.09573*, 2017. 4, 5
- [67] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny

- Chim, et al. Starcoder: May the source be with you! *arXiv preprint arXiv:2305.06161*, 2023. 12, 50, 60
- [68] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d’Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. Competition-level Code Generation with AlphaCode. 2022. 8, 38
- [69] Chang Liu, Xinyun Chen, Eui Chul Shin, Mingcheng Chen, and Dawn Song. Latent attention for if-then program synthesis. *Advances in Neural Information Processing Systems*, 29, 2016. 2
- [70] Chang Liu, Xin Wang, Richard Shin, Joseph E. Gonzalez, and Dawn Song. Neural code completion. 2017. 5
- [71] Ziyang Luo, Can Xu, Pu Zhao, Qingfeng Sun, Xiubo Geng, Wenxiang Hu, Chongyang Tao, Jing Ma, Qingwei Lin, and Daxin Jiang. Wizardcoder: Empowering code large language models with evol-instruct. *arXiv preprint arXiv:2306.08568*, 2023. 12, 50
- [72] Yashwant K Malaiya. Antirandom testing: Getting the most out of black-box testing. In *Proceedings of Sixth International Symposium on Software Reliability Engineering. ISSRE’95*, pages 86–95. IEEE, 1995. 17
- [73] Phil McMinn, Muzammil Shahbaz, and Mark Stevenson. Search-based test input generation for string data types using the results of web queries. In

2012 IEEE Fifth international conference on software testing, verification and validation, pages 141–150. IEEE, 2012. 16

- [74] Christoph C. Michael, Gary McGraw, and Michael A Schatz. Generating software test data by evolution. *IEEE transactions on software engineering*, 27(12):1085–1110, 2001. 16
- [75] Naman Goyal Mike Lewis, Yinhan Liu. Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. 2019. 38
- [76] Niklas Muennighoff, Qian Liu, Armel Zebaze, Qinkai Zheng, Binyuan Hui, Terry Yue Zhuo, Swayam Singh, Xiangru Tang, Leandro von Werra, and Shayne Longpre. Octopack: Instruction tuning code large language models. *arXiv preprint arXiv:2308.07124*, 2023. 106
- [77] Kenton Murray and David Chiang. Correcting length bias in neural machine translation. In *Proceedings of the 3rd Conference on Machine Translation (WMT)*, pages 212–223, 2018. 63
- [78] Trong Duc Nguyen, Anh Tuan Nguyen, Hung Dang Phan, and Tien N Nguyen. Exploring api embedding for api usages and applications. In *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*, pages 438–449. IEEE, 2017. 3
- [79] Tung Thanh Nguyen, Anh Tuan Nguyen, Hoan Anh Nguyen, and Tien N Nguyen. A statistical semantic language model for source code. In *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering*, pages 532–542, 2013. 3
- [80] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. A conversational paradigm for

program synthesis. *arXiv preprint arXiv:2203.13474*, 2022. vii, 8, 38, 51, 96, 97

- [81] Carlos Pacheco and Michael D. Ernst. Randoop: Feedback-directed random testing for java. In *Companion to the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems and Applications Companion (OOP-SLA)*, page 815–816, 2007. 27, 33, 62, 95
- [82] Michael P Papazoglou and Willem-Jan Van Den Heuvel. Service-oriented design and development methodology. *International Journal of Web Engineering and Technology*, 2(4):412–442, 2006. 23
- [83] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 311–318, 2002. 58
- [84] Chris Quirk, Raymond Mooney, and Michel Galley. Language to code: Learning semantic parsers for if-this-then-that recipes. In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 878–888, 2015. 2
- [85] Maxim Rabinovich, Mitchell Stern, and Dan Klein. Abstract syntax networks for code generation and semantic parsing. *arXiv preprint arXiv:1704.07535*, 2017. 6
- [86] Alec Radford and Karthik Narasimhan. Improving language understanding by generative pre-training. 2018. 38
- [87] Ervin Ramollari, Dimitris Dranidis, and Anthony JH Simons. A survey of service oriented development methodologies. In *The 2nd European young*

- researchers workshop on service oriented computing*, volume 75. Citeseer, 2007. 23
- [88] Veselin Raychev, Martin Vechev, and Eran Yahav. Code completion with statistical language models. In *Proceedings of the 35th ACM SIGPLAN conference on programming language design and implementation*, pages 419–428, 2014. 3
- [89] Veselin Raychev, Pavol Bielik, and Martin Vechev. Probabilistic model for code with decision trees. *ACM SIGPLAN Notices*, 51(10):731–747, 2016. 6
- [90] Radim Řehůřek and Petr Sojka. Word2vec embeddings. <https://radimrehurek.com/gensim/models/word2vec.html>, 2013. 78, 79
- [91] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. An empirical evaluation of using large language models for automated unit test generation. *IEEE Transactions on Software Engineering*, 2023. vii, 18, 19
- [92] John Schulman, Barret Zoph, Christina Kim, Jacob Hilton, Jacob Menick, Jiayi Weng, Juan Felipe Ceron Uribe, Liam Fedus, Luke Metz, Michael Pokorný, et al. Chatgpt: Optimizing language models for dialogue. *OpenAI blog*, 2022. 34, 38, 96, 97
- [93] Sina Shamshiri. Automated unit test generation for evolving software. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, pages 1038–1041, 2015. 33
- [94] James Shore and Shane Warden. *The art of agile development*. ” O’Reilly Media, Inc.”, 2021. 28
- [95] Yoonki Song, Suresh Thummalapenta, and Tao Xie. Unitplus: Assisting

- developer testing in eclipse. In *Proceedings of the OOPSLA Workshop on Eclipse Technology EXchange*, page 26–30, 2007. 63
- [96] Leon Sterling and Ehud Y Shapiro. *The art of Prolog: advanced programming techniques*. MIT press, 1994. 18
- [97] Zeyu Sun, Qihao Zhu, Lili Mou, Yingfei Xiong, Ge Li, and Lu Zhang. A grammar-based structural cnn decoder for code generation. *Proceedings of the AAAI Conference on Artificial Intelligence*, 33(01):7055–7062, Jul. 2019. 5, 6
- [98] Zeyu Sun, Qihao Zhu, Yingfei Xiong, Yican Sun, Lili Mou, and Lu Zhang. Treegen: A tree-based transformer architecture for code generation. *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(05):8984–8991, Apr. 2020. 6
- [99] Kunal Taneja and Tao Xie. Automated regression unit-test generation. In *Proceedings of the 23rd IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 407–410, 2008. 62
- [100] Chris Thunes. Javalang. 2019. 70
- [101] Paolo Tonella. Evolutionary testing of classes. *ACM SIGSOFT Software Engineering Notes*, 29(4):119–128, 2004. 16
- [102] Tanimoto TT. An elementary mathematical theory of classification and prediction. *Internal IBM Technical Report*, 1957. 83
- [103] Zhaopeng Tu, Zhendong Su, and Premkumar Devanbu. On the localness of software. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, pages 269–280, 2014. 3

- [104] Michele Tufano, Dawn Drain, Alexey Svyatkovskiy, Shao Kun Deng, and Neel Sundaresan. Unit test case generation with transformers and focal context. *arXiv preprint arXiv:2009.05617*, 2020. 19
- [105] Michele Tufano, Shao Kun Deng, Neel Sundaresan, and Alexey Svyatkovskiy. Methods2test: A dataset of focal methods mapped to test cases. In *Proceedings of the 19th International Conference on Mining Software Repositories*, pages 299–303, 2022. 20
- [106] Michele Tufano, Dawn Drain, Alexey Svyatkovskiy, and Neel Sundaresan. Generating accurate assert statements for unit test cases using pretrained transformers. In *Proceedings of the 3rd ACM/IEEE International Conference on Automation of Software Test*, page 54–64, New York, NY, USA, 2022. Association for Computing Machinery. 21, 95, 100
- [107] Arash Vahabzadeh, Amin Milani Fard, and Ali Mesbah. An empirical study of bugs in test code. In *2015 IEEE international conference on software maintenance and evolution (ICSME)*, pages 101–110. IEEE, 2015. 95
- [108] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017. 38
- [109] Oriol Vinyals, Meire Fortunato, and Navdeep Jaitly. Pointer networks. *Advances in neural information processing systems*, 28, 2015. 3, 6
- [110] Jingyuan Wang, Yufan Wu, Mingxuan Li, Xin Lin, Junjie Wu, and Chao Li. Interpretability is a kind of safety: An interpreter-based ensemble for adversary defense. In *Proceedings of the 26th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, pages 15–24, 2020. 63

- [111] Thomas Wang, Adam Roberts, Daniel Hesslow, Teven Le Scao, Hyung Won Chung, Iz Beltagy, Julien Launay, and Colin Raffel. What language model architecture and pretraining objective work best for zero-shot generalization? *arXiv preprint arXiv:2204.05832*, 2022. 39
- [112] Zhiruo Wang, Shuyan Zhou, Daniel Fried, and Graham Neubig. Execution-based evaluation for open-domain code generation. *arXiv preprint arXiv:2212.10481*, 2022. 39
- [113] Cody Watson, Michele Tufano, Kevin Moran, Gabriele Bavota, and Denys Poshyvanyk. On learning meaningful assert statements for unit test cases. In *Proceedings of the 42th IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 1398–1409, 2020. 21, 62, 95
- [114] Joachim Wegener and Oliver Bühler. Evaluation of different fitness functions for the evolutionary testing of an autonomous parking system. In *Genetic and Evolutionary Computation—GECCO 2004: Genetic and Evolutionary Computation Conference, Seattle, WA, USA, June 26–30, 2004. Proceedings, Part II*, pages 1400–1412. Springer, 2004. 16
- [115] Joachim Wegener and Matthias Grochtmann. Verifying timing constraints of real-time systems by means of evolutionary testing. *Real-Time Systems*, 15:275–298, 1998. 16
- [116] Yuxiang Wei, Zhe Wang, Jiawei Liu, Yifeng Ding, and Lingming Zhang. Magicoder: Source code is all you need. *arXiv preprint arXiv:2312.02120*, 2023. 50
- [117] L.J. White and E.I. Cohen. A domain strategy for computer program testing. *IEEE Transactions on Software Engineering*, SE-6(3):247–257, 1980. 16

- [118] Wikipedia contributors. Overlap — Wikipedia, the free encyclopedia. 2021. [Online; accessed 12-March-2022]. 83
- [119] Tao Xie. Augmenting automatically generated unit-test suites with regression oracle checking. In *Proceedings of the 20th European Conference on Object-Oriented Programming (ECOOP)*, pages 380–403, 2006. 62
- [120] Frank F. Xu, Uri Alon, Graham Neubig, and Vincent Josua Hellendoorn. A systematic evaluation of large language models of code. In *Deep Learning for Code Workshop*, 2022. 3
- [121] Pengcheng Yin and Graham Neubig. A syntactic neural model for general-purpose code generation. *arXiv preprint arXiv:1704.01696*, 2017. 2
- [122] Hao Yu, Yiling Lou, Ke Sun, Dezhi Ran, Tao Xie, Dan Hao, Ying Li, Ge Li, and Qianxiang Wang. Automated assertion generation via information retrieval and its integration with deep learning. In *Proceedings of the 44th International Conference on Software Engineering*, pages 163–174, 2022. 21, 95
- [123] Hao Yu, Bo Shen, Dezhi Ran, Jiaxin Zhang, Qi Zhang, Yuchi Ma, Guangtai Liang, Ying Li, Qianxiang Wang, and Tao Xie. Codereval: A benchmark of pragmatic code generation with generative pretrained models. In *2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE)*, pages 428–439, 2024. 32
- [124] Zhiqiang Yuan, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, Xin Peng, and Yiling Lou. Evaluating and improving chatgpt for unit test generation. *Proceedings of the ACM on Software Engineering*, 1(FSE): 1703–1726, 2024. vii, 19, 20

- [125] Jie Zhang, Lingming Zhang, Mark Harman, Dan Hao, Yue Jia, and Lu Zhang. Predictive mutation testing. *IEEE Transactions on Software Engineering*, 45(9):898–918, 2019. 62, 95
- [126] Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shan Wang, Yufei Xue, Lei Shen, Zihan Wang, Andi Wang, Yang Li, et al. Codegeex: A pre-trained model for code generation with multilingual benchmarking on humaneval-x. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 5673–5684, 2023. 10, 50
- [127] Victor Zhong, Caiming Xiong, and Richard Socher. Seq2sql: Generating structured queries from natural language using reinforcement learning. arxiv 2017. *arXiv preprint arXiv:1709.00103*, 2017. 2